开发者社区

DTK

DTK 26.10
hipprof 使用手册

DTK-261000-007-0915-922

目　录

1 hipprof 简介 1
2 hipprof 指令 1
2.1 hipprof 参数 1
2.2 通用指令 5
2.3 tracing 指令 5
2.4 PMC 指令 11
2.5 LeakCheck 指令 16
2.6 寄存器压力分析指令 17
2.7 CoreDump 分析指令 17
2.8 集群指令 17
3 hipprof 功能演示 19
3.1 单进程 HIP 接口跟踪可视化 19
3.2 多进程 HIP 接口跟踪可视化 22
3.3 HIP 接口跟踪信息打印 24
3.4 HIP 接口跟踪过滤分析 26
3.5 MPI 多节点多进程 HIP 接口跟踪可视化 28
3.6 HIPTX 接口跟踪可视化 33
3.7 OpenMP/OpenACC 接口跟踪可视化 34
3.8 RCCL 接口跟踪可视化 39
3.9 LIBC 接口跟踪可视化 42
3.10 HSA 接口跟踪可视化 43
3.11 OpenCL 接口跟踪可视化 44
3.12 实时导出 Trace 数据 45
3.13 分段实时导出 Trace 数据 46
3.14 PMC 性能分析 47
3.15 寄存器压力分析 49
3.16 CoreDump 分析 49
4 泄漏检测功能演示 51

开发者社区

4.1 泄漏检测.....51
4.2 基于时间过滤泄漏堆栈.....55
4.3 查找内存波动原因.....58
4.4 会话控制泄漏检测.....61
4.5 泄漏堆栈排序方法.....62
5 集群场景功能演示.....65
5.1 集群追踪可视化.....65
5.2 集群追踪控制.....70
5.3 集群传输控制.....76
5.4 集群网络控制.....78
5.5 展示信息过滤控制.....79

开发者社区

版本更新：

- DTK-26.10

  增加--realtime-trace <type code>指令，指定实时导出 trace 数据。

  增加--orderby-totalsize 指令，设置根据泄漏大小排列泄漏函数堆栈。

  修复内存泄漏检测，函数堆栈缺失问题，全局静态变量内的内存泄漏误报问题。

  调整--pmc <type>指令，支持 default|read|write|compute|wave|util|memory 多种模式。

  支持 pmc 集群模式。

- DTK-26.04

  增加--flush-interval 指令，调整性能数据缓存刷新时间。

  增加--no-export 指令，支持选项控制只采集，不导出性能数据（自定义导出）。

  增加--exit-cleanup 指令，支持进程非正常退出场景，强制性能数据缓存刷新。

  增加--follow-fork 指令，跟踪子进程。

  增加--show-pid 指令，根据 pid 分组打印和导出统计数据。

  扩充--session 会话功能，增加--show-nodes、--exit 功能，控制集群模式会话机制动态分析性能数据。

  增加--master 指令，启动主节点服务端。

  增加--port 指令，指定通信端口。

  增加--node-list 指令，指定可以启动客户端的子节点。

  增加--data-mode 指令，指定数据传输模式。

  增加--slave 指令，启动子节点客户端。

  增加--master-address 指令，指定主节点服务端 IP 地址。

  增加--netcard 指令，指定通信网卡。

  增加--pid-file 指令，指定 pid 文件。

  增加--kernel-profile-mode 指令，支持 OpenCL PMC 功能。

  完善--mpi-trace 指令支持 intelmpi 环境。

  完善--hsa-trace 指令显示 kernel 信息。

  优化 leakcheck 性能，--check-minsize 由默认的 1024 调整为 1。

开发者社区

恢复--input-core 及 --codeobj-analyze 指令。

- DTK-25.04.2

  增加--cl-trace 指令，支持对 OpenCL API 进行跟踪分析。

  增加--trace-args 指令，显示跟踪参数信息，默认情况下参数信息保持隐藏。

  增加--mpi-trace 指令在 openmpi 环境 fortran 版本支持。

  修复--omp-trace 与--hip-trace 联用时无法导出 hip 数据问题。

  修复--leak-check 部分场景死锁问题。

  修复--pmc 多线程部分场景崩溃问题。

  废弃旧的--hid-args 指令，默认隐藏参数，需要参数则使用--trace-args 指令。

  该版本暂时废弃 --input-core 指令。

  该版本暂时废弃 --codeobj-analyze 指令。

- DTK-25.04.1

  增加--lib-trace 指令，支持对 LIBC API 进行跟踪分析。

  增加--lib-trace-level 指令，支持对 LIBC API 进行等级过滤。

  增加--hsa-trace 指令，支持对 HSA API 进行跟踪分析。

  修改-o <output file>指令，指定输出文件的路径和文件名。

  修改-d <data directory>指令，改变临时目录，默认/tmp。

  修复 rccl 传入 ncclAvg 操作时，hipprof 显示结果与实际调用 ncclkernel 不一致问题。

  修复 hipprof 无法将同时间段所有卡信息放在同一个 json 内问题。

  修复在 Ubuntu 系统中内存泄漏检测时，创建新进程出现死锁的问题。

  修复 hipprof 命令参数中包含转义字符时解析失败的问题。

  修复--pmc 显示的核函数耗时与 trace 数据中耗时不一致问题。

  优化 hip 接口性能数据采集，改善 hip 采集接口性能。

- DTK-25.04

  增加--kernel-stack 指令，支持对核函数主机端调用栈追踪显式。

  增加--omp-trace 指令，支持对 OpenMP、OpenACC 程序做性能分析。

  增加--rccl-trace 指令，支持对 RCCL 程序做性能分析。

  增加--pmc-off 指令，支持设置启动时不开启 pmc 功能。

开发者社区

增加--codeobj-analyze <elf file> 指令，支持打开寄存器压力分析功能。

增加--record-memfreeinfo 指令，支持记录内存释放信息，默认不记录。

增加--show-limit <limit_num>指令，支持设置显示泄漏堆栈的数量，默认数量为 5。

增加显示泄漏内存类型(host、device0、device1...)、各类型内存泄漏总大小、内存泄漏文件名及行号等信息。

修复多线程场景下因 hipprof 导致的空泡耗时过大问题，优化后空泡耗时减少 50%以上。

json 可视化数据中增加 memcpy 类型、大小、吞吐率的输出显示。

- DTK-24.04

  增加--devices 指令，可以针对某一张或多张卡做性能分析。

  增加--print-trace 指令，将性能分析过程数据 print 在终端上。

  增加--mask-trace 过滤功能，可以指定 API 进行分析，缩减分析数据量。

  增加--session 会话功能，进行会话机制动态分析性能数据。

  增加--trace-off 功能，默认关闭跟踪功能，由 roctracer API、或者 session 控制性能分析范围。

  增加--segment-size 功能，用于指定用于生成 json 文件的 hipAPI 条目数。

  增加--merge-type 功能，用于选择合并类型。

  增加--output-type 导出选项功能，可以导出 json 和 html 类型输出文件。

  将 hiprtc 相关函数对接 hipprof，加入 API 跟踪。

  优化内存、显存、流、事件的泄漏检测，增加泄漏信息：函数堆栈、次数、大小信息。

  优化内存泄漏曲线图，区分内存、显存、总体曲线。

  增加内存堆栈图，可用来可视化泄漏函数调用关系。

  增加--check-minsize 参数，小于该值的内存申请，不记录函数栈信息。

  增加--time-range 参数，检测指定时间范围内的泄漏信息。

- DTK-23.10

  增加了内存，流和事件的泄漏检查提示。

- DTK-23.04

开发者社区

修改 Performance Counter（PMC）输出方式，增加 Performance Counter 输出格式选项。

- DTK-22.10.1

  增加了显存使用情况的跟踪和可视化输出。

- DTK-22.10

  性能分析可视化是通过 chrome 浏览器 tracing 前端工具，由于工具的限制，如果跟踪数据较多，产生的文件较大，会造成无法正常展示的情况，因此，在 DTK-22.10 版本中，如果数据内容较多，hipprof 输出的可视化数据会自动分割成多个文件，分组进行可视化。

开发者社区

1 hipprof 简介

hipprof 是 DTK 的性能分析工具，用于对 HIP 应用程序提供可视化的性能分析。提供的功能有：单进程、多进程、多节点的 HIP API 跟踪，ROCTX 跟踪，OpenMP/OpenACC API 跟踪，RCCL API 跟踪，OpenCL API 跟踪，MPI 日志解析，PMC 硬件计数器性能数据的统计输出，内存、流和事件泄漏检查输出以及相关的辅助功能。下面从指令和功能演示两个角度分别对 hipprof 的使用进行介绍。

2 hipprof 指令

hipprof 指令的基本使用格式是：

hipprof [options] <app command line>

hipprof 所有的指令执行过程是先解析后执行，因此指令的设置没有先后顺序的区分。但是有优先级的区分，hipprof 的分析原理是通过运行 hip 程序，收集过程中产生的性能数据，这个过程可能会耗时较长，因此优先级最高的是不需要运行 hip 程序的指令。比如：

```
hipprof --hip-trace -h ./testhip
```

其中同时包含了--hip-trace 和-h 的指令，但是-h 是显示帮助信息，并不需要执行 hip 程序，因此优先级最高，这里忽略--hip-trace 指令只执行-h 的指令。而需要运行应用做性能分析的指令具有相同的优先级并且可以同时使用，但是应该尽量避免同时使用，因为有些指令会造成性能上的影响，降低其他指令操作的性能数据准确度。

2.1 hipprof 参数

-h	用于显示帮助信息，罗列并提示指令作用。
-o <output file>	指定输出文件的路径和文件名。
-d <data directory>	改变临时目录，默认/tmp。
--output-type <type code>	设置 3（默认）输出显示文件为 perfetto，0 为 json 格式，1 为 html 格式，2 为 perfetto 格式， 3 为 pftrace.gz 格式

开发者社区

--hiptx-trace	启动 ROCTX 应用的跟踪分析，可参考 NVTX。
--hiptx-kernel	将 ROCTX 的标注范围扩展至对应的 kernel 上。
--hip-trace	启动 HIP API、Memory copy、kernel 时间线跟踪。
--libc-trace	启动 LIBC API 时间线的跟踪功能。
--hsa-trace	启动 HSA API、Memory copy、kernel 时间线的跟踪功能
--cl-trace	启动 OpenCL API、kernel 时间线跟踪。
--omp-trace	启动 OpenMP/OpenACC API、Memory copy、kernel 时间线跟踪。
--rccl-trace	启动 RCCL API、kernel 时间线跟踪。
--mpi-trace	启动 MPI 日志解析，跟踪。
--barrier	在 trace 文件中增加 event 和隐式同步的信息。
--kernel-stack	输出核函数的堆栈信息，输出 csv 文件。
--no-export	只生成 db 数据文件，不导出性能数据。
--stats	只打印和导出统计数据，不输出 trace 文件。
--show-pid	根据 pid 分组打印和导出统计数据。
--segment-size <size>	用于分割 trace 文件条目数，至少 5000 ，默认 500000。
--group-stream	将 HIP TRACE 中的 memcpy、kernel 按照 stream 合并显示。
--trace-args	跟踪分析时收集参数，db 数据库文件中 args 含数据。
--realtime-trace <type code>	指定实时导出 trace 数据，0：经 db 导出 trace 数据（默认），1：实时导出，2：实时导出且生成 db 文件。
--devices <device id>	只输出指定设备的异步活动（OPS 和 COPY）。
--print-trace	在终端上打印性能分析流程数据。
--mask-trace <config file>	指定配置文件的 HIP API 进行数据收集。
--libc-trace-level <level>	启动 LIBC API 的追踪等级。
--trace-off	在启动时禁用跟踪。

开发者社区

--db <db file name>	对指定的数据库文件进行解析，生成对应的性能数据文件。
--db-path <db path>	对指定目录的所有数据库文件进行解析并生成对应的性能数据文件。
--db-merge <db file path>	指定目录,将目录内多个数据库文件合并为一个数据库文件，并导出对应的性能数据文件。
--merge-type <type id>	选择合并类型，0 为默认原始时间，1 为时间对齐。
--show-trace-id	展示指定数据库文件中的 traces-ID。
--set-trace-id <trace_id>	根据指定的 trace-ID 导出可视化文件。
--index-range <start:end>	导出部分性能数据，index 为 HIP API 的索引号。
--flush-interval <ms>	调整性能数据缓存刷新时间。
--buffer-size <size>	调整性能数据缓存大小，达到指定值后自动回传数据，默认 100000。
--exit-cleanup	进程退出，强制性能数据缓存刷新。
--follow-fork	强制跟踪子进程状态。
--master	启动主节点服务端。
--port <port num>	指定通信端口；默认：1234。
--node-list <ip|name[,...]>	指定需要启动客户端的子节点。
--data-mode <mode id>	指定数据传输模式；0:回传 db，1:实时回传，2:不回传；默认 0。
--slave	启动子节点客户端。
--master-addres <ip|ip:port>	指定主节点服务端 IP 地址。
--netcard <card name>	指定通信网卡，默认自动识别，识别出错时可指定。
--pid-file <pid file name>	指定 hipprof 与采集进程的通信文件；默认 /tmp/hipprof/hipprof_pid_[uid]_[pid].pid。

开发者社区

--leak-check	启动内存、显存、流和事件的泄漏检查。。
--check-minsize <size>	小于 size 的内存泄漏，不提取函数堆栈信息，size 默认值 1。
--memory-usage	相比于--leak-check，减少了提取函数堆栈功能。
--time-range <start:end>	使用指定时间范围过滤内存泄露信息，单位：秒。
--record-memfreeinfo	记录内存释放信息，默认不记录。
--orderby-totalsize	设置根据泄漏大小排列泄漏函数堆栈，默认根据泄漏次数排列泄漏函数堆栈。
--show-limit <limit_num>	限制泄漏函数栈的数量，默认最多展示 5 个。
--session <name>	动态控制性能分析(服务端)。
--session-client <name>	启动 session 客户端。
--show-nodes	展示集群节点信息指令。
--start	开始控制指令。
--stop	停止控制指令。
--flush	刷新控制指令。
--exit	通过 session 控制集群强制退出指令。
--pmc <type>	根据类型输出 PMC 性能数据。
--pmc-read	输出缓存读相关的 PMC 数据（将弃用，请使用--pmc read）。
--pmc-write	输出缓存写相关的 PMC 数据（将弃用，请使用--pmc write）。
--pmc-type <number>	切换 PMC 输出格式。
--pmc-off	启动时关闭 PMC 功能。
--kernel-name <kernel>	PMC 滤出设定的核函数名称。
--kernel-profile-mode <mode>	设置 kernel 指令数据收集模式，默认 0，设置 1 用于支持 OpenCL。
--input-core <core file>	分析指定的核心转储（core dump）文件。
--codeobj-analyze	用于对给定的程序执行寄存器压力分析功能。

开发者社区

2.2 通用指令

➢-h

用于显示帮助信息，罗列并提示指令作用。

➢-o <output file>

设置输出文件的存储路径和文件名前缀。参数值<output file>按以下规则解析

- ●参数值以斜杠（/）结尾：整个字符串被识别为输出目录路径，不再添加额外的文件前缀。
- ●参数值不以斜杠（/）结尾：从字符串最后一个斜杠处分割为文件路径和文件前缀两部分。其中文件前缀添加到程序生成文件文件名中。

➢-d <data directory>

指定修改设置路径，默认/tmp。

➢--output-type <type code>

设置输出文件格式。默认值为 2，即输出 Perfetto 文件；设为 0 生成 JSON 文件，设为 1 生成 HTML 文件。

2.3 tracing 指令

➢--hiptx-trace

用于开启 hiptx 程序的性能分析。如果用户想通过加入一些 mark 或者 range 信息来确定程序内部和 HIP API 之间的关系，可以在用户 HIP 程序中加入 roctx.h 的接口，并通过这条指令开启 HIPTX 的跟踪分析，将用户调用 roctx.h 中的接口情况一起显示出来。

➢--hiptx-kernel

用于开启 hiptx 程序，根据 tx 接口来标注信息来确定程序内部和 HIP Kernel 之间的关系。

developer.sourcefind.cn

➢--hip-trace

用于开启 HIP API 以及核函数、memcpy 的跟踪分析，如果性能数据内容较多，可视化 trace 文件会自动分割为多个脚本文件，以支持前端显示。

➢--libc-trace

用于开启 LIBC API 进行跟踪分析，如果性能数据内容较多，可视化 trace 文件会自动分割为多个脚本文件，以支持前端显示。

➢--hsa-trace

用于开启 HSA API 以及 copy，kernel 相关的跟踪分析，和--hip-trace 类似。

➢--cl-trace

用于开启 OpenCL API 以及 copy，kernel 相关的跟踪分析，和--hip-trace 类似。

➢--omp-trace

用于开启 OpenMP/OpenACC API 以及核函数、memcpy 的跟踪分析，如果性能数据内容较多，可视化 trace 文件会自动分割为多个脚本文件，以支持前端显示。

➢--rccl-trace

用于开启 RCCL API、核函数的跟踪分析以及开启 HIP API 以及核函数、memcpy 的跟踪分析，可视化页面中核函数的详细信息增加带宽、数据量等显示。

➢--mpi-trace

使用 MPI 做多进程、多结点的分析，在开启 MPI 日志的同时、使用--mpi-trace 功能，hipprof 会解析 MPI 的日志，并与 HIP API 做对应，同时显示出来。

开发者社区

➢--barrier

用于开启 HIP API 以及核函数、memcpy 的跟踪分析后将同步屏障点信息增加到 trace 文件，在可视化页面里面增加 event 和隐式同步的信息。

➢--kernel-stack

通过该指令来获取核函数主机端调用栈信息。

➢--no-export

只产生 db，不导出性能数据。

➢--stats

用于开启 HIP API 以及核函数、memcpy 的跟踪分析后打印和导出统计数据时不输出可视化 trace 文件。

➢--show-pid

默认情况下，多进程程序的性能数据统计表不根据 pid 进行区分，使用这个指令，可以根据 pid 分组打印和导出统计数据。

➢--segment-size <size>

设置条目大小间接控制 trace 文件大小，perfetto 格式的 trace 默认 5000000±200000 条。

➢--group-stream

默认情况下，HIP API 的跟踪脚本是将 memcpy 和 stream 可视化目录分开显示的，如果希望 memcpy 的内容显示在 stream 中，使用--group-stream 来转换目录结构。这个指令可以和--db 联合使用。

➢--trace-args

用户开启 HIP API 以及核函数、memcpy 的跟踪分析时采集各种函数的参数

开发者社区

具体信息。

➢--realtime-trace <type code>

开启实时导出 trace 数据的模式，(options: 0=db, 1=rt, 2=rt-db default: 0)，0：经 db 导出 trace 数据（默认），1：实时导出，2：实时导出且生成 db 文件。

➢--devices <device id>

通过指定设备 ID，对异步活动 COPY 和 OPS 进行输出显示。

➢--print-trace

将性能分析数据输出到终端。

➢--mask-trace <config file>

通过指定含有要分析的 HIP API 的配置文件，进行 HIP API 过滤，只输出统计相关 HIP API 信息。

➢--libc-trace-level

开启 LIBC API 等级，默认跟踪所有支持的 libc api 接口。0：跟踪部分常用的 libc api；1：跟踪所有支持的 libc api 接口。

➢--trace-off

通过该指令来禁用性能数据跟踪。

➢--db <db file name>

如果用户已经做了性能分析并生成了一个数据库文件（.db 结尾），但是对输出的可视化文件的范围、内容想做一些调整，可以直接操作数据库文件后，重新根据数据库文件生成可视化文件，不需要重新运行 HIP 应用程序。

开发者社区

➢--db-path <db path>

指定包含数据库文件的目录路径，自动处理该目录下所有有效数据库。

➢--db-merge <db file path>

指定目录，将目录内多个数据库文件合并为一个数据库文件，并生成合并后的可视化脚本，用于多节点的性能分析，对比多个结点的性能情况，该指令不需要重新运行 HIP 应用程序。

➢--merge-type <type id>

选择合并类型，0 为默认原始时间，1 为时间对齐。

➢--show-trace-id

和--db 连用，列出指定数据库中包含的所有 trace ID，用于和--set-trace-id 使用。

➢--set-trace-id <trace id>

仅导出指定 trace ID 对应的数据。

➢--index-range <start:end>

指定 HIP API 的索引号范围，导出部分可视化脚本。这个指令可以和--db 联合使用，如果联合--db 则不需要重新执行 HIP 应用。

➢--flush-interval <ms>

设置缓存自动刷新间隔（默认 5000 ms）。适用于程序不能正常结束，又无法填满缓存且运行时间短的场景：缩短间隔可提升数据落盘及时性（最小值 1 ms），牺牲一部分性能从而提高数据的实时性。

➢--buffer-size

为了保证性能，galaxy 产生的性能数据是存在缓存中，只有程序结束或者缓

开发者社区

存满了或者距离上一次刷新超过 5s 才会输出到数据库文件中，缓存大小默认是 100000 条 HIP API 数据。但是如果程序不能正常结束（比如 server 程序，或者中间存在异常结束），又不能填满缓存的情况下，使用这个指令可以缩小缓存空间，最小是 1，牺牲一部分性能从而提高数据的实时性。

➢--exit-cleanup

启用退出前强制刷新缓存。适用于当用户程序的运行时间没有达到默认刷新时间，同时没有填满缓存以及程序没有清理用户空间退出的情况下，通过拦截退出系统调用确保数据完整性。

➢--follow-fork

跟踪子进程。

➢--session <name>

通过该指令启动 session 服务端，通过接收 session 客户端的指令动态控制 hipprof 性能分析。

➢--session-client <name>

通过该指令启动 session 客户端，通过指定 start、stop、flush、exit、show-nodes 等子命令通知 session 服务端动态控制性能分析。

➢--show-nodes

session 子命令，展示 hipprof 集群节点信息指令。

➢--start

session 子命令，性能分析开始指令。

➢--stop

session 子命令，性能分析结束指令。

开发者社区

➢--flush

session 子命令，性能分析数据刷新指令。

➢--exit

session 子命令，控制 hipprof 集群强制退出指令。

2.4 PMC 指令

PMC（性能监控计数器）指令用于收集异构加速器硬件计数器的性能数据，以提供核函数执行的详细硬件层面指标。

由于硬件计数器资源有限，无法一次性输出全部完整的性能数据，因此 PMC 功能按照指标类型分为七组，分别对应 default、read、write、compute、wave、util、memory。 default、read、write 兼容原 pmc 用法，其中--pmc-read、--pmc-write 命令保留并标记为弃用，--pmc 修改用法。每组使用方法相同，指标存在部分重合，您可以根据所关注的性能维度选择相应的组别进行分析。

➢--pmc <type>

1、default：兼容原--pmc 指令，当前可使用--pmc 或--pmc default，输出的性能数据如表 2.1 所示，包含特有的缓存命中相关指标。

表 2.1 基础 PMC 指标

参数	说明
kernel time	核函数执行时间。
processed ALU instructions	计算单元负载率（%）。
performance	计算峰值（Gflops）
shared memory operation	LDS 操作次数
shared memory bank conflict	LDS Bank 冲突时间占比（%）
L1 cache unit is active	L1 缓存操作时间占比（%）
L1 cache unit is stalled	L1 缓存等待时间占比（%）
L2 cache write unit is stalled	L2 缓存写入操作占比（%）

开发者社区

L2 cache hit rate	L2 缓存命中率（%）

2、read：兼容原--pmc-read 指令，当前可使用--pmc read，输出的性能数据如表 2.2 所示，包含特有的缓存读相关指标。

表 2.2 读相关 PMC 指标

参数	说明
kernel time	核函数执行时间。
processed ALU instructions	计算单元负载率（%）
performance	计算峰值（Gflops）
shared memory operation	LDS 操作次数
shared memory bank conflict	LDS Bank 冲突时间占比（%）
L1 cache unit is active	L1 缓存操作时间占比（%）
L1 cache unit is stalled	L1 缓存等待时间占比（%）
number of L2 cache read requests	L2 缓存读请求次数
size of L2 cache read	L2 缓存读字节数

3、write：兼容原--pmc-write 指令，当前可使用--pmc write，输出的性能数据如表 2.3 所示，包含特有的缓存写相关指标。

表 2.3 写相关 PMC 指标

参数	说明
kernel time	核函数执行时间。
processed ALU instructions	计算单元负载率（%）
performance	计算峰值（Gflops）
shared memory operation	LDS 操作次数
shared memory bank conflict	LDS Bank 冲突时间占比（%）
L1 cache unit is active	L1 缓存操作时间占比（%）
L1 cache unit is stalled	L1 缓存等待时间占比（%）
number of L2 cache write requests	L2 缓存写请求数

size of L2 cache write	L2 缓存写字节数

4、compute：分析算力利用是否充分，仅支持 gfx938 及其后架构，当前可使用--pmc compute，输出的性能数据如表 2.4 所示，包含各类精度矩阵运算吞吐量。此数值不包括来自 VALU 指令的任何浮点操作。

表 2.4 矩阵运算相关 PMC 指标

参数	说明
MMOP FLOPs (F64)	每秒执行的 FP64 矩阵乘加运算次数（GFLOP/s）
MMOP FLOPs (F32)	每秒执行的 FP32 矩阵乘加运算次数（GFLOP/s）
MMOP FLOPs (TF32)	每秒执行的 TF32 矩阵乘加运算次数（GFLOP/s）
MMOP FLOPs (F16)	每秒执行的 F16 矩阵乘加运算次数（GFLOP/s）
MMOP FLOPs (BF16)	每秒执行的 BF16 矩阵乘加运算次数（GFLOP/s）
MMOP IOPs (I8)	每秒执行的 Int8 矩阵乘加运算次数（GIOP/s）
MMOP FLOPs (F8)	每秒执行的 FP8 矩阵乘加运算次数（GFLOP/s）
MMOP IOPs (I4)	每秒执行的 Int4 矩阵乘加运算次数（GIOP/s）
MMOP FLOPs (MIX)	每秒执行的混合精度矩阵乘加运算次数，仅支持 gfx92a（GFLOP/s）

5、wave：观察 wavefront（warp）执行与并发情况，当前可使用--pmc wave，输出的性能数据如表 2.5 所示，包含 wavefront 相关指标。

表 2.5 Wavefront 相关 PMC 指标

参数	说明
Wavefront Occupancy	核函数运行期间，DCU 上驻留 wavefront 占比（%）
Wave Cycles	每个 wavefront 在 CU 上平均执行周期数（Cycles per Wave）
Dependency Wait Cycles	wavefront 因等待任何类型的内存访问而停顿的周期占比（%）

Issue Wait Cycles	wavefront 因无法发射指令而等待的周期占比（%）
Active Cycles	wavefront 真正处于执行状态的周期占比（%）

6、util：观察各硬件模块利用情况，当前可使用--pmc util，输出的性能数据如表 2.6 所示，包含各硬件管线相关指标。

表 2.6 硬件管线相关 PMC 指标

参数	说明
CU Utilization	计算单元 CU 的整体利用率（%）
SALU Utilization	SALU 在内核执行期间忙于执行指令所占的时间百分比（%）
VALU Utilization	VALU 在内核执行期间忙于执行指令所占的时间百分比（%）
VMEM Utilization	VMEM 在内核执行期间忙于执行指令所占的时间百分比，包括全局/通用内存和溢出/临时存储操作（%）
Branch Utilization	分支在内核执行期间忙于执行指令所占的时间百分比（%）
Vector L1 Data Cache Utilization	vL1D 缓存在内核执行期间的繁忙程度（%）
Local Data Share Utilization	LDS 在内核运行期间实际执行指令所占的时间百分比（%）
L2 Cache Utilization	L2 通道在内核执行期间处于活动状态的时间百分比（%）

7、memory：观察各级缓存利用情况，当前可使用--pmc memory，输出的性能数据如表 2.7 所示，包含 LDS、L1、L2 相关指标。

表 2.7 各级缓存相关 PMC 指标

开发者社区

参数	说明
Local Data Share BankConflict	LDS Bank 冲突时间占比（%）
Local Data Share Bandwidth Utilization	LDS 通路的带宽使用程度（%）
Instruction Cache HitRate	指令缓存命中率（%）
Scalar L1 Data Cache HitRate	标量 L1 数据缓存命中率（%）
Vector L1 Data Cache Bandwidth Utilization	向量 L1 数据缓存的数据吞吐使用程度（%）
Vector L1 Data Cache HitRate	向量 L1 数据缓存命中率（%）
L2 Cache HitRate	L2 缓存命中率（%）
L2 Cache Bandwidth Utilization	L2 缓存带宽占用程度（%）

➢--pmc-type <number>

选择 PMC 输出格式，默认是 0，number 表示 PMC 输出格式，解释意义如下：

0，默认，输出 PMC 参数的全名，展示清晰易懂。

1，输出 PMC 参数的简写，减少输出文本尺寸，兼顾易读性。

2，输出 PMC 原始数据（参考旧版本 PMC 输出）。

3，输出 CSV 表格形式的数据，可用 excel 打开方便统计。

➢--kernel-name <kernel>

指定要收集 PMC 性能数据的核函数名称，可排除其他核函数的干扰，使分析结果更为聚焦。

➢--pmc-off

在启动时关闭 PMC 功能。

➢--kernel-profile-mode <mode>

设置内核性能数据的收集模式。默认值为 0；设置为 1 可支持 OpenCL 程序。

开发者社区

2.5 LeakCheck 指令

内存泄漏检测，用来检测应用程序使用内存、显存、流和事件时出现的内存泄漏问题，并生成内存泄漏问题对应的详细报告，协助用户快速定位问题。

➢--leak-check

启动内存、显存、流和事件的泄漏检查，生成报告内容如下

1、泄漏函数名称、泄漏次数、泄漏内存大小、泄漏函数栈

2、内存占用趋势图，输出 <result_pid>.mem<pid>.html 文件

3、内存泄漏堆栈图，输出 <result_pid>.leakcheck.html 文件

4、记录内存泄漏详情的 sqlite 数据库文件

➢--check-minsize <size>

size 默认值是 1，内存检测时，泄漏小于 size 时，就不会提取内存泄漏处的函数堆栈。size 值越小，检测堆栈越多，程序运行越慢。

➢--memory-usage

启动内存、显存、流和事件的泄漏检查，生成报告内容如下

1、内存占用趋势图，输出 <result_pid>.mem<pid>.html 文件

2、记录内存泄漏详情的 sqlite 数据库文件

➢--time-range <start:end>

使用指定时间范围过滤内存泄露信息。

➢--record-memfreeinfo

记录内存释放信息，默认不记录。记录内存释放信息后可从数据库中查找到运行周期泄漏及未泄漏的内存申请信息。

➢--orderby-totalsize

开发者社区

设置根据泄漏大小排列泄漏函数堆栈，默认根据泄漏次数排列泄漏函数堆栈。

➢---show-limit <limit_num>

限制展示泄漏堆栈的最大数量，默认最多展示 5 个。

2.6 寄存器压力分析指令

寄存器压力分析能够追踪核函数运行各阶段的寄存器占用状态及压力水平，这对于特定代码段的资源优化配置至关重要，进而能够显著提升优化工作的效率和针对性。

➢--codeobj-analyze <elf file>

选择 elf 文件中的核函数进行寄存器压力分析并生成分析报告文件。

2.7 CoreDump 分析指令

Core Dump 分析用于 HIP 程序出现异常时的现场保存与分析。可以分析崩溃进程的 AQL 报文、Kernel 信息、GC 状态信息以及内存信息等数据。

➢--input-core <core file>

分析指定的 CoreDump 文件并输出对相关的数据文件。

2.8 集群指令

集群场景简单介绍：

➢--master

用于启动**主节点（服务端）**，必须作为命令行的第一个参数。默认使用 db 传输模式（参考--data-mode），子节点的数据会先存入 db，通过网络将 db 传输到主节点，默认通过 eno1 网卡传输，数据会在主节点展示。

➢--port

指定通信端口，仅用于主节点使用。当出现端口占用时可以使用该指令切换端口。主要用于多个 docker 使用同一个宿主机 ip，每个 docker 中都开启 hipprof

开发者社区

集群的情况，--master --port 2345 为指定通过主节点的 2345 端口通信，后续子节点使用--master-address 指令配对指定 2345 端口。

➢--data-mode

data-mode 可以指定子节点数据传输的方式，支持三种模式 0：db 传输; 1：实时传输；2：不传输；默认为 db 传输模式。

（a）db 传输模式下，子节点抓取的数据先存入本节点的 db 文件，子节点程序运行结束后会将 db 文件通过网络发送到主节点，子节点 db 文件在子节点的命名规则：hip-prof-[子节点 hipprof 的 pid]-[子节点通信网卡 ip].db，主节点接收文件后不改变 db 文件命名规则。

（b）实时传输模式下，子节点会将性能数据直接发送到主节点，在主节点生成 db 文件并将子节点的性能数据存入 db 文件，此时 db 命名规则：hip-prof-[主节点 hipprof 的 pid]-[子节点通讯网卡 ip].db。

（c）不传输模式下，子节点抓取的数据存入本节点的 db 文件，但是程序结束后不发送到主节点，因此主节点不会展示子节点的数据。

➢--node-list

允许收集子节点（ip 或者节点名字）的数据，不在指定范围的节点不允许启动。例如--node-list 1.1.1.1：允许 ip 地址为 1.1.1.1 的节点启动 hipprof 客户端；--node-list b08r3n19：允许节点名字为 b08r3n19 的节点启动 hipprof 客户端。

➢--slave

指定启动子节点，必须作为命令行的第一个参数。

➢--master-address

仅用于子节点（--slave 连用）指定连接的主节点 ip，使用方式为 hipprof --slave --master-address <ip:port>，默认可以不指定 port，使用 1234 端口。例如 --master-address 1.1.1.1:2345，指定主节点的 1.1.1.1 网卡，2345 端口通信。

➢--netcard

集群模式默认使用 eno1 网卡进行通信，当集群没有 eno1 网卡或者 eno1 网

卡没有 ip 时，可以通过--netcard 指定一个有 ip 的网卡进行通信，例如--netcard ib1 为指定通过 ib1 网卡通信。当前网卡信息可通过 ip address 查看。

➢--pid-file

指定前后端通信的全局 pid 文件，当 pid 文件存在导致无法启动集群时，可以通过该指令指定一个新的 pid 文件用于启动集群。

3 hipprof 功能演示

3.1 单进程 HIP 接口跟踪可视化

首先需要准备一个性能分析的 HIP 应用程序。在正确安装 DTK 以后，可以在 hip/sample 目录中选择一个应用样例进行演示。本次演示使用 hip/samples/2_Cookbook/0_MatrixTranspose 程序，注意如果没有权限，可以把程序代码复制到自己的用户目录下操作。

➢make 生成可执行程序 MatrixTranspose。

➢使用 hipprof 进行性能分析，可获得下述结果。

hipprof --hip-trace ./MatrixTranspose

```
[root@j17r4n00 matrixTranspose]$ hipprof --hip-trace ./MatrixTranspose
HIP_PROF:on Fri Jan 30 15:42:23 2026
HIP_PROF:process id '14788'
HIP_PROF:db file 'hip-prof-14788.db'
HIP_PROF:out put file 'result_14788'
HIP_PROF:profiling ' ./MatrixTranspose'
HIP_PROF:process begin
Device name Device 66a1
PASSED!
HIP_PROF:process end run total cost:0(s)
--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP_PROF:IP:11.10.17.61 HIP API statistics
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Name                                                                                                                |Calls          |TotalDurationNs      |AverageNs            |Percentage           |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|hipMalloc                                                                                                           |2              |17634501             |8817250              |75.185137181051      |
|hipMemcpy                                                                                                           |2              |4550674              |2275337              |19.4019127026187     |
|hipLaunchKernel                                                                                                     |1              |1085993              |1085993              |4.63015838569297     |
|hipFree                                                                                                             |2              |161361               |80680                |0.687966669466381    |
|__hipPopCallConfiguration                                                                                           |1              |10495                |10495                |0.0447456956516734   |
|__hipPushCallConfiguration                                                                                          |1              |7014                 |7014                 |0.029904364869065    |
|hipGetDeviceProperties                                                                                              |1              |4732                 |4732                 |0.020175000650187 6  |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Total                                                                                                               |10             |23454770             |2345477              |100.0                |
--------------------------------------------------------------------------------------------------------------------------------------------------------

--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP_PROF:IP:11.10.17.61 HIPOPS API statistics
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Name                                                                                                                |Calls          |TotalDurationNs      |AverageNs            |Percentage           |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|matrixTranspose(float*, float*, int)                                                                                |1              |206812               |206812               |100.0                |
--------------------------------------------------------------------------------------------------------------------------------------------------------
|Total                                                                                                               |1              |206812               |206812               |100.0                |
--------------------------------------------------------------------------------------------------------------------------------------------------------
HIP_PROF:dump perfetto trace (15/15)
HIP_PROF:finish
[root@j17r4n00 matrixTranspose]$
```

分析完成后会输出上图中的信息，包括 HIP 接口的统计表和核函数的统计表。Name 列是函数或者接口的名称；Calls 表示调用的次数；TotalDurationNs 是函数运行过程中总共消耗的时间，以纳秒为单位；AverageNs 是每次调用平均耗

时，以纳秒为单位；Percentage 是该函数占用的总时间的百分比；最后一行 total 是表格的统计总计信息。

以上表格除了输出在终端外会另外形成两个 csv 格式的文件：

result_14788.hiptrace.csv

result_14788.hipkernel.csv

文件名称中 14788 是本次性能分析的进程号，将文件下载到本地后可以使用 Excel 等支持 csv 的图表工具打开。

表 3.1 result_8237.hiptrace.csv 文件内容

Name	Calls	TotalDurationNs	AverageNs	Percentage
hipMalloc	2	17494607	8747303	78.22034
hipMemcpy	2	3494594	1747297	15.62472
hipLaunchKernel	1	1202751	1202751	5.377634
hipFree	2	152840	76420	0.683365
__hipPopCallConfiguration	1	9500	9500	0.042476
__hipPushCallConfiguration	1	6980	6980	0.031208
hipGetDeviceProperties	1	4530	4530	0.020254
Total	10	22365802	2236580	100

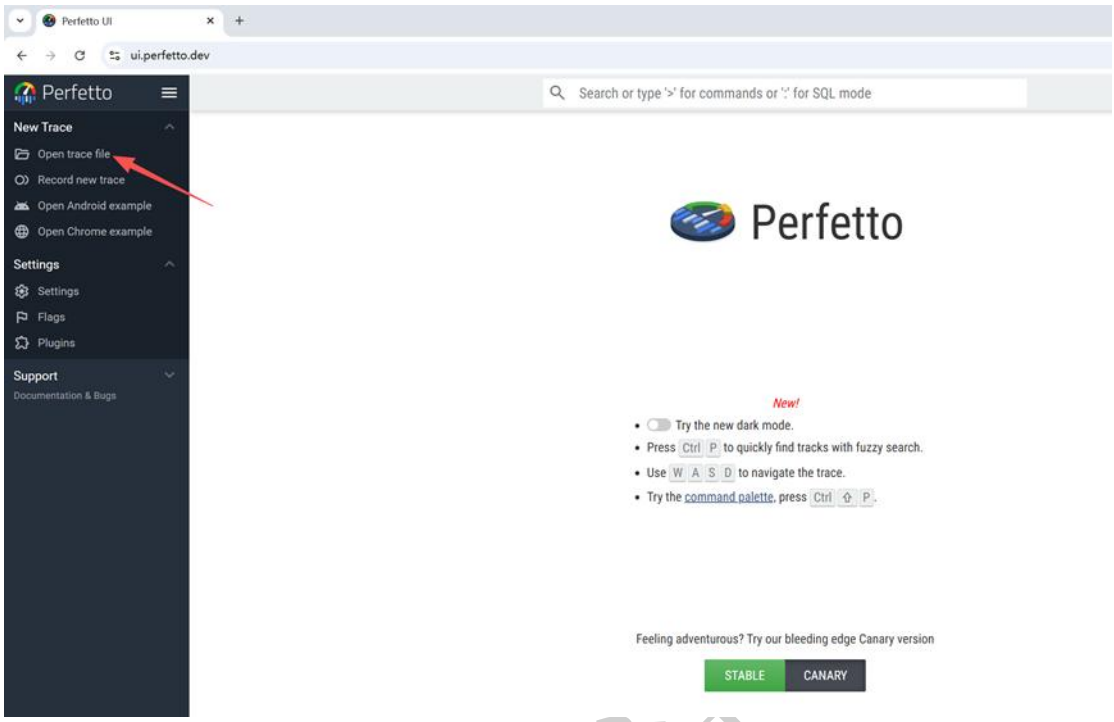
表 3.2 result_14788.kernel.csv 文件内容

Name	Calls	TotalDurationNs	AverageNs	Percentage
matrixTranspose(float*, float*, int)	1	207351	207351	100
Total	1	207351	207351	100

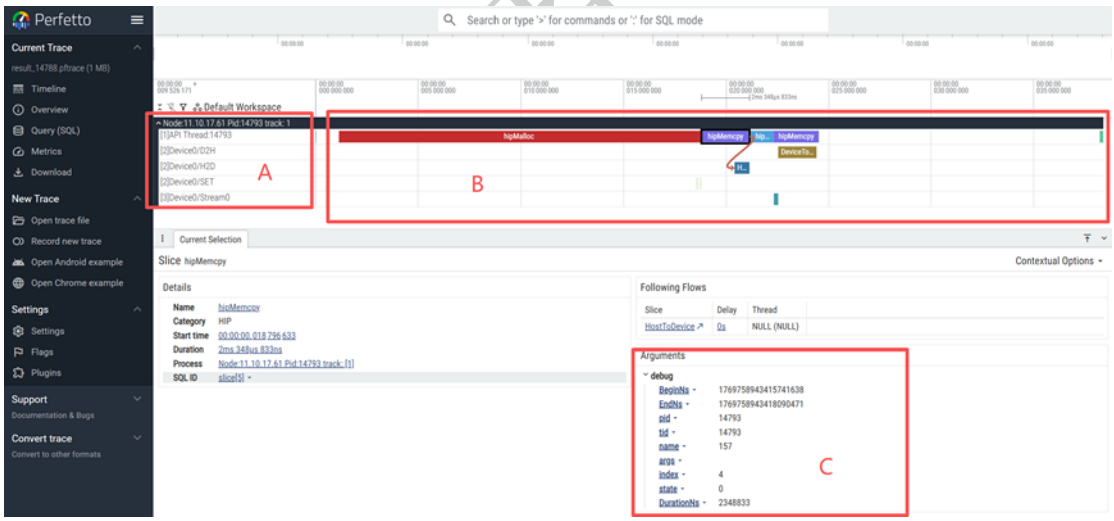
除了输出统计表格以外，性能分析过程还会产生一个 perfetto 格式的文件：

result_14788.pftrace

把文件下载到本地后在 chrome 浏览器中输入 https://ui.perfetto.dev/打开可视化工具：



点击 Open trace file 按钮，选择下载好的 pftrace 文件，打开可视化的程序运行过程和性能信息：



打开后左侧 A 区域显示的是性能数据的目录结构：

Node:11.5.10.60 Pid:14793 track:1 表示当前程序的进程号是 14793，运行的节点 IP 是 11.5.10.60，Perfetto 轨道序号是 1；

[1]API Thread:14793 表示当前行是运行在线程 14793 上的 API 接口；

[2]Device0/D2H 表示当前行是运行在卡 0 上的数据拷贝（显存拷贝到内存）；

[2]Device0/H2D 表示当前行是运行在卡 0 上的数据拷贝（内存拷贝到显存）；

[2]Device0/SET 表示当前行是运行在卡 0 上的数据设置；

[3]Device0/Stream0 表示当前行是运行在卡 0 上的流 0 对应的 kernel 函数；

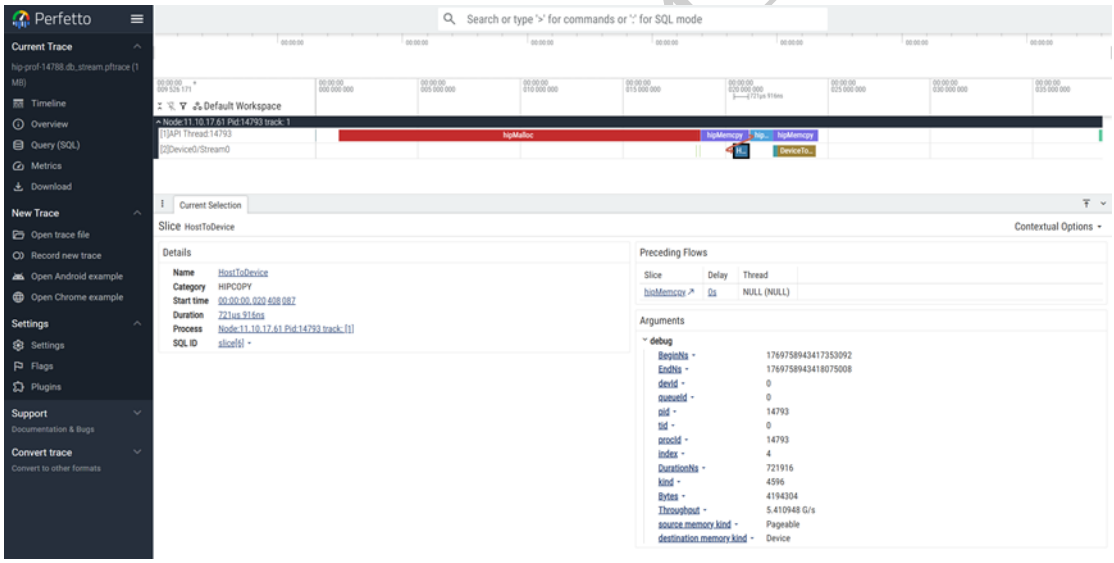
右侧 B 区域是具体每个操作所占用的时间和流程，称之为 timeline，可以通过键盘上的（w，s，a，d）按键进行放大缩小和前后滚动查看的操作，点击彩色单个 timeline 可以在下方 C 区域显示出函数的详细信息。

API 和 copy、kernel 之间存在异步操作，可以通过点击 API、copy、kernel 按钮来显示 API 和 copy、kernel 的对应关系。

默认情况下，memcpy 和核函数是分开显示的，memcpy 的信息并没有按照流进行显示，如果需要和核函数一样按照流进行显示，在性能分析过程中增加指令--group-stream，也可以通过已经生成的数据库文件重新生成可视化文件，指令：

```
hipprof --db hip-prof-8237.db --group-stream
```

重新生成的 perfetto 文件打开后效果如下，少了 memcpy 的行，memcpy 的数据与核函数一起显示在流中。



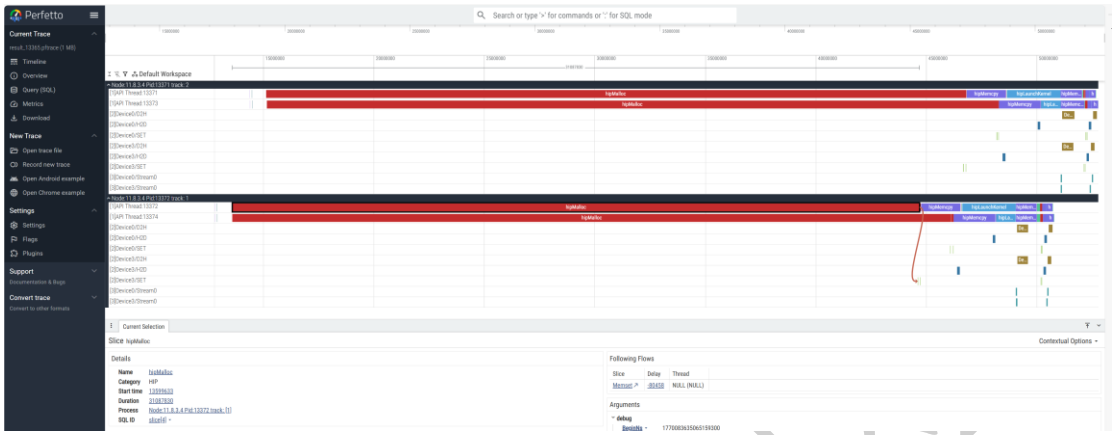
3.2 多进程 HIP 接口跟踪可视化

多进程的性能分析分为普通的多进程应用分析和 MPI 程序多进程分析，这里分别进行演示，MPI 多进程在下一个小节中。

hipprof 本身具有多进程的管理模块，可以和单进程 HIP 程序一样直接使用 hipprof 进行分析。本次示例使用一个普通的 HIP 程序，通过 fork 产生多个进程，指令：

hipprof --hip-trace ./mutiprocess

可视化效果如下图，与单进程相比，多进程的 API 会分别显示在不同的 trace 进程框中：



跟踪到的 trace 数据会存放到 db 文件中，通过 hipprof 可以查看 db 中 trace 数据对应的 trace-id (ip-pid-start_time)

hipprof --show-trace-id --db hip-prof-13365.db

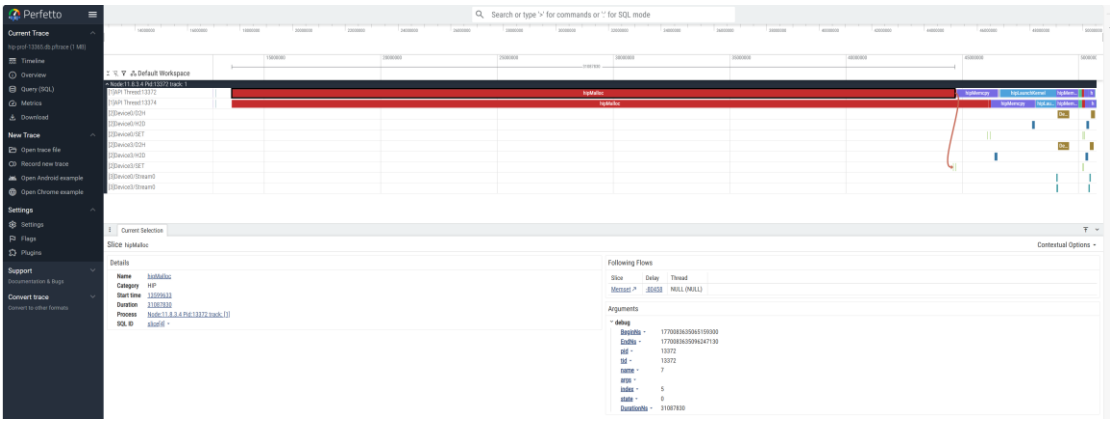
```
root@h03r1n03 trace]$ hipprof --show-trace-id --db hip-prof-13365.db
HIP_PROF:db filename:hip-prof-13365.db
-----------------------------------------------------------------------------------
|trace-id                    |ip         |pid     |start-time                  |
-----------------------------------------------------------------------------------
|11_8_3_4_13372_1770083635   |11.8.3.4   |13372   |2026-02-03 09:53:55.51559667  |
-----------------------------------------------------------------------------------
|11_8_3_4_13371_1770083635   |11.8.3.4   |13371   |2026-02-03 09:53:55.52968551  |
-----------------------------------------------------------------------------------
HIP_PROF:finish
root@h03r1n03 trace]$
```

当前 db 中包含了两个 trace-id，hipprof 支持指定 trace-id 导出对应的 trace 数据

hipprof --set-trace-id 11_8_3_4_13372_1770083635 --db hip-prof-13365.db

```
root@h03r1n03 trace]$ hipprof --set-trace-id 11_8_3_4_13372_1770083635 --db hip-prof-13365.db
-----------------------------------------------------------------------------------------------------------------
HIP_PROF:IP:11.8.3.4 HIP API statistics
-----------------------------------------------------------------------------------------------------------------
|Name                                      |Calls        |TotalDurationNs        |AverageNs          |Percentage          |
-----------------------------------------------------------------------------------------------------------------
|hipMalloc                                 |8            |63970508               |7996313            |86.3522517551087    |
|hipMemcpy                                 |8            |6467443                |808430             |8.73024591500533    |
|hipLaunchKernel                           |4            |3305544                |826386             |4.46207442460187    |
|hipFree                                   |8            |302721                 |37840              |0.408635804542279   |
|__hipPopCallConfiguration                 |4            |14296                  |3574               |0.019297826915663   |
|__hipPushCallConfiguration                |4            |9494                   |2373               |0.0128157224914175  |
|hipGetDeviceProperties                    |2            |7714                   |3857               |0.0104129432587734  |
|hipSetDevice                              |2            |3160                   |1580               |0.00426560807592998 |
-----------------------------------------------------------------------------------------------------------------
|Total                                     |40           |74080880               |1852022            |100.0               |
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
HIP_PROF:IP:11.8.3.4 HIPOPS API statistics
-----------------------------------------------------------------------------------------------------------------
|Name                                      |Calls        |TotalDurationNs        |AverageNs          |Percentage          |
-----------------------------------------------------------------------------------------------------------------
|matrixTranspose(float*, float*, int)      |4            |223622                 |55905              |100.0               |
-----------------------------------------------------------------------------------------------------------------
|Total                                     |4            |223622                 |55905              |100.0               |
-----------------------------------------------------------------------------------------------------------------
HIP_PROF:dump perfetto trace (60/60)
HIP_PROF:finish
root@h03r1n03 trace]$
```

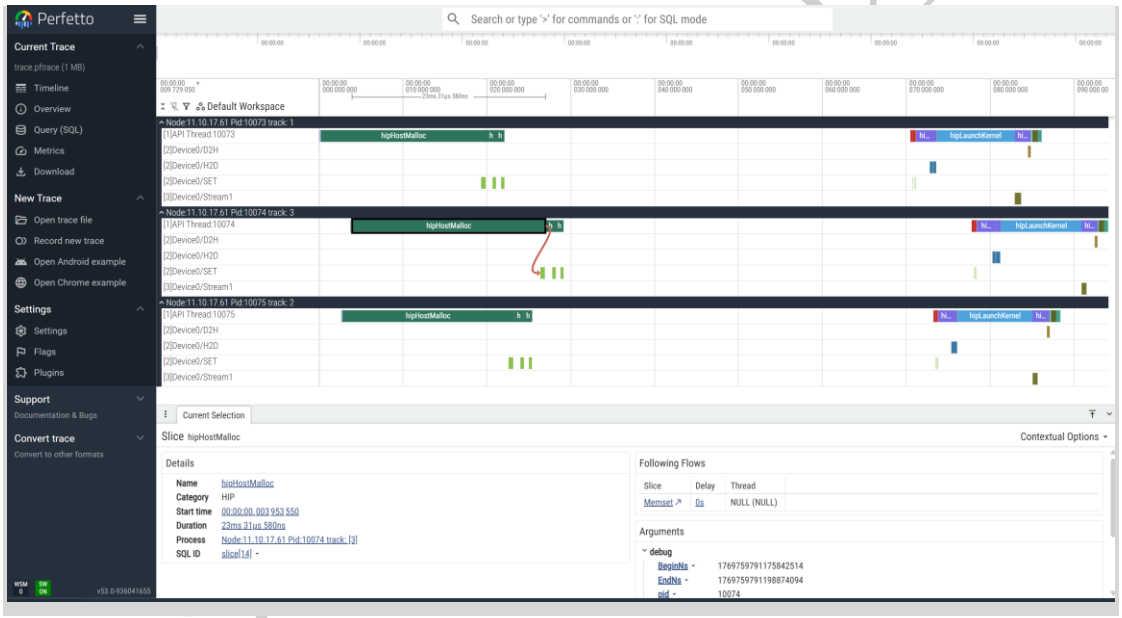
过滤后的 trace 数据可视化图：



如果是单节点多进程的 MPI 程序，也可以和普通的多进程程序一样做分析：

```
hipprof --hip-trace -o trace mpirun -x LD_PRELOAD=mpitracer.so -np 4 ./test_mpi
```

显示效果如下：



3.3 HIP 接口跟踪信息打印

该功能是为了将性能数据打印在终端，可配合--trace-args 或者--db 进行使用。开启指令如下：

```
hipprof --print-trace ./test_trace
```

执行成功后,可在终端显示 HIP API 的性能数据，比如返回值状态和函数调用持续时间和输入参数等信息。

```
[root@j17r4n00 matrixTranspose]$ hipprof --print-trace ./MatrixTranspose
HIP_PROF:on Fri Jan 30 16:01:44 2026
HIP_PROF:process id '20370'
HIP_PROF:db file 'hip-prof-20370.db'
HIP_PROF:out put file 'result_20370'
HIP_PROF:profiling ' ./MatrixTranspose'
HIP_PROF:process begin
HIPAPI: start_time:16:01:14.072368461, pid:20374, tid:20374, index:1, stat:0, extId:0, dur:4230ns, hip_name:hipGetDeviceProperties
Device name Device 66a1
COPY: start_time:16:01:14.091319134, dev_id:0, que:0, kind:SET, index:2, dur:13598ns
HIPAPI: start_time:16:01:14.074354841, pid:20374, tid:20374, index:2, stat:0, extId:0, dur:17098950ns, hip_name:hipMalloc
COPY: start_time:16:01:14.091618634, dev_id:0, que:0, kind:SET, index:3, dur:12638ns
HIPAPI: start_time:16:01:14.091477751, pid:20374, tid:20374, index:3, stat:0, extId:0, dur:197040ns, hip_name:hipMalloc
COPY: start_time:16:01:14.093272688, dev_id:0, que:0, kind:H2D, index:4, dur:709716ns
HIPAPI: start_time:16:01:14.091710981, pid:20374, tid:20374, index:4, stat:0, extId:0, dur:2341940ns, hip_name:hipMemcpy
HIPAPI: start_time:16:01:14.094118991, pid:20374, tid:20374, index:5, stat:0, extId:0, dur:6960ns, hip_name:__hipPushCallConfiguration
HIPAPI: start_time:16:01:14.094168811, pid:20374, tid:20374, index:6, stat:0, extId:0, dur:10090ns, hip_name:__hipPopCallConfiguration
HIPAPI: start_time:16:01:14.094220821, pid:20374, tid:20374, index:7, stat:0, extId:0, dur:1016080ns, hip_name:hipLaunchKernel
OPS : start_time:16:01:14.095244482, dev_id:0, que:0, index:7, dur:206546ns, kernel_name:matrixTranspose(float*, float*, int)
COPY: start_time:16:01:14.095455188, dev_id:0, que:0, kind:D2H, index:8, dur:826108ns
HIPAPI: start_time:16:01:14.095270871, pid:20374, tid:20374, index:8, stat:0, extId:0, dur:1207470ns, hip_name:hipMemcpy
PASSED!
HIPAPI: start_time:16:01:14.110087051, pid:20374, tid:20374, index:9, stat:0, extId:0, dur:129640ns, hip_name:hipFree
HIPAPI: start_time:16:01:14.110266621, pid:20374, tid:20374, index:10, stat:0, extId:0, dur:25440ns, hip_name:hipFree
HIP_PROF:process end run total cost:0(s)
```

同时可以搭--trace-args 指令来显示 args 信息。开启指令如下：

hipprof --print-trace --trace-args ./test_trace

执行成功后,可在终端显示含有 args 信息的打印信息。

```
[root@j17r4n00 matrixTranspose]$ hipprof --print-trace --trace-args ./MatrixTranspose
HIP_PROF:on Fri Jan 30 16:02:48 2026
HIP_PROF:process id '22435'
HIP_PROF:db file 'hip-prof-22435.db'
HIP_PROF:out put file 'result_22435'
HIP_PROF:profiling ' ./MatrixTranspose'
HIP_PROF:process begin
HIPAPI: start_time:16:02:14.594034421, pid:22439, tid:22439, index:1, stat:0, extId:0, dur:5140ns, hip_name:hipGetDeviceProperties
args:hipGetDeviceProperties(props={pageableMemoryAccessUsesHostPageTables=0, pageableMemoryAccess=0, concurrentManagedAccess=0, directManagedMemAccessFromHost=0, managedMemory=0, asic
Revision=1, isLargeBar=1, cooperativeMultiDeviceUnmatchedSharedMem=1, cooperativeMultiDeviceUnmatchedBlockDim=1, cooperativeMultiDeviceUnmatchedGridDim=1, cooperativeMultiDeviceUnmatc
hedFunc=1, tccDriver=0, ECCEnabled=0, kernelExecTimeoutEnabled=0, texturePitchAlignment=256, textureAlignment=256, memPitch=2147483647, hdpRegFlushCntl=0x2b5992342004, hdpMemFlushCntl
=0x2b5992342000, maxTexture3D=0xef6300, maxTexture2D=0xef62f8, maxTexture1D=16384, maxTexture1DLinear=2147483647, cooperativeMultiDeviceLaunch=1, cooperativeLaunch=1, integrated=0, gc
nArchName="gfx906:sramecc-:xnack-", gcnArch=26273, canMapHostMemory=1, isMultiGpuBoard=0, maxSharedMemoryPerMultiProcessor=65536, pciDeviceID=0, pciBusID=4, pciDomainID=0, concurrentK
ernels=1, arch={}, clockInstructionRate=1000000, computeMode=0, maxThreadsPerMultiProcessor=2560, l2CacheSize=0, multiProcessorCount=64, minor=0, major=9, totalConstMem=2147483647, me
moryBusWidth=4096, memoryClockRate=1000000, clockRate=1700000, maxGridSize=0xef6180, maxThreadsDim=0xef6174, maxThreadsPerBlock=1024, warpSize=64, regsPerBlock=65536, sharedMemPerBloc
k=65536, totalGlobalMem=17163091968, name="Device 66a1"}, device=0)
Device name Device 66a1
COPY: start_time:16:02:14.613383384, dev_id:0, que:0, kind:SET, index:2, dur:12960ns
HIPAPI: start_time:16:02:14.595483781, pid:22439, tid:22439, index:2, stat:0, extId:0, dur:18040040ns, hip_name:hipMalloc
args:hipMalloc(ptr=0x2b5db5000000, size=4194304)
COPY: start_time:16:02:14.613687204, dev_id:0, que:0, kind:SET, index:3, dur:12960ns
HIPAPI: start_time:16:02:14.613569241, pid:22439, tid:22439, index:3, stat:0, extId:0, dur:173490ns, hip_name:hipMalloc
args:hipMalloc(ptr=0x2b5db8600000, size=4194304)
COPY: start_time:16:02:14.615548452, dev_id:0, que:0, kind:H2D, index:4, dur:758834ns
HIPAPI: start_time:16:02:14.613806421, pid:22439, tid:22439, index:4, stat:0, extId:0, dur:2561740ns, hip_name:hipMemcpy
args:hipMemcpy(dst=0x2b5db5000000, src=0x2b5aa3a00010, sizeBytes=4194304, kind=1)
HIPAPI: start_time:16:02:14.616464991, pid:22439, tid:22439, index:5, stat:0, extId:0, dur:6730ns, hip_name:__hipPushCallConfiguration
args:__hipPushCallConfiguration(gridDim={z=1, y=256, x=256}, blockDim={z=1, y=4, x=4}, sharedMem=0, stream=0)
HIPAPI: start_time:16:02:14.616552521, pid:22439, tid:22439, index:6, stat:0, extId:0, dur:11190ns, hip_name:__hipPopCallConfiguration
args:__hipPopCallConfiguration(gridDim={z=1, y=256, x=256}, blockDim={z=1, y=4, x=4}, sharedMem=0, stream=0)
HIPAPI: start_time:16:02:14.616641981, pid:22439, tid:22439, index:7, stat:0, extId:0, dur:1057930ns, hip_name:hipLaunchKernel
args:hipLaunchKernel(function_address=0x200d20, numBlocks={z=1, y=256, x=256}, dimBlocks={z=1, y=4, x=4}, args=0x7ffd6914b028, sharedMemBytes=0, stream=0)
OPS : start_time:16:02:14.617707280, dev_id:0, que:0, index:7, dur:206388ns, kernel_name:matrixTranspose(float*, float*, int)
COPY: start_time:16:02:14.617917828, dev_id:0, que:0, kind:D2H, index:8, dur:1959320ns
HIPAPI: start_time:16:02:14.617762951, pid:22439, tid:22439, index:8, stat:0, extId:0, dur:2308800ns, hip_name:hipMemcpy
args:hipMemcpy(dst=0x2b5db4603010, src=0x2b5db8600000, sizeBytes=4194304, kind=2)
PASSED!
HIPAPI: start_time:16:02:14.633466801, pid:22439, tid:22439, index:9, stat:0, extId:0, dur:136490ns, hip_name:hipFree
args:hipFree(ptr=0x2b5db5000000,size=4194304)
HIPAPI: start_time:16:02:14.633692201, pid:22439, tid:22439, index:10, stat:0, extId:0, dur:31140ns, hip_name:hipFree
args:hipFree(ptr=0x2b5db8600000,size=4194304)
HIP_PROF:process end run total cost:0(s)
```

同时可以搭配--db 来输入数据库，打印数据库信息，开启指令如下：

hipprof --print-trace --db ./hip-prof-4731.db

执行成功后,打印信息和使用单指令 hipprof --print-trace ./test_trace 一样的输出效果。

使用--kernel-stack 指令对核函数接口的跟踪还进行了额外的堆栈信息显示。该功能是方便查找相关 kernel 出现优化问题时，通过堆栈信息查看显示该核函数由哪个库提供，方便追踪。

hipprof --kernel-stack ./test_trace

执行成功后,会生成 result_<pid> .stack.csv 文件。

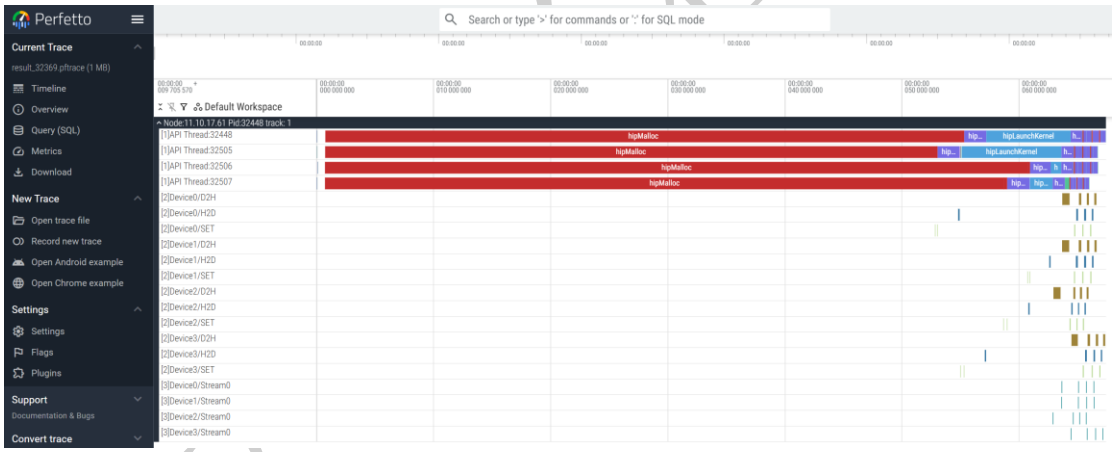
如下所示，由 pid:进程；index:kernel 索引值；str_name:kernel name；stack_sq:堆栈索引；addr：地址；addrname：地址信息 6 列数据。

	A	B	C	D	E	F
1	pid	index	str_name	stack_sq	addr	addrname
2	24516		7 matrixTranspose(float*, float*, int)	1	4.73E+13	/public/home/guoyl01/local/dtk/hip/lib/libgalaxyhip.so.5(hipLaunchKernel+0xc80) [0x2b0cb9a66800]
3	24516		7 matrixTranspose(float*, float*, int)	2	2238873	./MatrixTranspose() [0x222999]
4	24516		7 matrixTranspose(float*, float*, int)	3	4.73E+13	/lib64/libc.so.6(__libc_start_main+0xf5) [0x2b0cba7b93d5]
5	24516		7 matrixTranspose(float*, float*, int)	4	2237769	./MatrixTranspose() [0x222549]
6						
7						
8						

3.4 HIP 接口跟踪过滤分析

HIP 接口跟踪过滤分为通过指定设备、API 的配置信息进行过滤方式以及 session 会话机制动态控制三种方式。

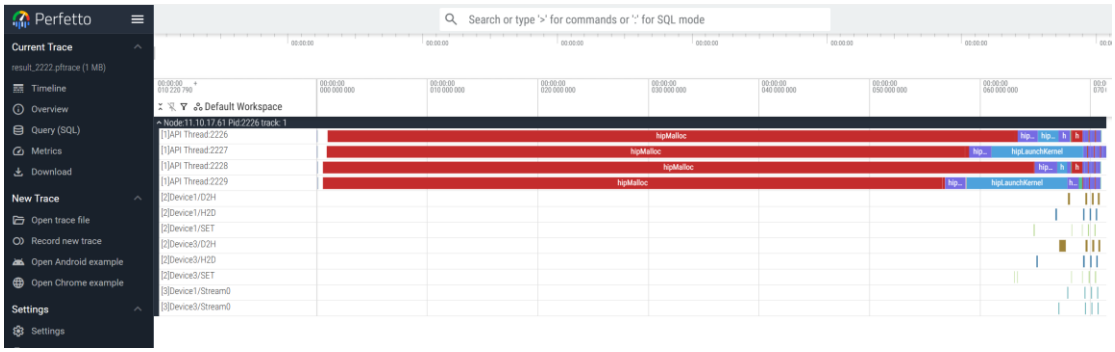
第一种方式为指定设备方式，通过--devices 指令来指定设备 ID,然后只输出相关 ID 设备下的 COPY 和 OPS 信息。分配多设备后，不指定设备时，默认全部输出，如下图所示。



开启指令如下

```
hipprof --devices 1,3 ./test_devices
```

执行成功后，只采集设备 ID 为 1 和 3 的性能数据。



第二种方式为使用 API 的配置信息进行过滤，开启指令

```
hipprof --mask-trace ./example.txt ./test_trace
```

其中./example.txt 表示 HIP API 配置文件，HIP API 之间以逗号隔开。

```
test_hipprof > test_trace > example.txt
1    hipMalloc,hipMemcpy
```

执行成功后，会输出配置文件中想采集的 HIP API 统计信息。

```
[root@j17r4n00 test-trace-performance]$ hipprof --mask-trace ./example.txt ./MatrixTranspose 1 4 4
HIP_PROF:on Fri Jan 30 16:15:28 2026
HIP_PROF:process id '15998'
HIP_PROF:db file 'hip-prof-15998.db'
HIP_PROF:out put file 'result_15998'
HIP_PROF:profiling ' ./MatrixTranspose 1 4 4'
HIP_PROF:process begin
HIP_PROF:process end run total cost:1(s)
HIP_PROF:IP:11.10.17.61 HIP API statistics
|Name                                                
```

```
Device name Z100SM
PASSED!
Press ENTER to perform matrix transpose (loop 1)...

Device name Z100SM
PASSED!
Press ENTER to perform matrix transpose (loop 2)...

Device name Z100SM
PASSED!
Press ENTER to perform matrix transpose (loop 3)...
q
Named pipe /tmp/hipprof_luosf553_10155 deleted successfully.
SubThread thread stop
HIP_PROF:process end run totol cost:45(s)
Named pipe /tmp/1234 deleted successfully.
Monitoring thread stop
--------------------------------------------------------------------------------------------------------------------------------
HIP PROF:hip API statistics
--------------------------------------------------------------------------------------------------------------------------------
|Name                                                  |Calls         |TotalDurationNs       |AverageNs             |Percentage            |
--------------------------------------------------------------------------------------------------------------------------------
|hipMalloc                                             |4             |424338046             |106084511             |98.4099938594237      |
|hipMemcpy                                             |4             |5639161               |1409790               |1.30780118495974      |
|hipLaunchKernel                                       |2             |938248                |469124                |0.217592979910683     |
|hipFree                                               |4             |238541                |59635                 |0.0553210313487205    |
|__hipPopCallConfiguration                             |2             |19300                 |9650                  |0.0044759429407536    |
|__hipPushCallConfiguration                            |2             |12741                 |6370                  |0.0029548180833234    |
|hipGetDeviceProperties                                |2             |8021                  |4010                  |0.00186018333304584   |
--------------------------------------------------------------------------------------------------------------------------------
|Total                                                 |20            |431194058             |21559702              |100.0                 |
--------------------------------------------------------------------------------------------------------------------------------
```

3.5 MPI 多节点多进程 HIP 接口跟踪可视化

在多节点多进程的 MPI 程序性能分析过程需要两个步骤：

1. 每个进程启动一个 hipprof 分别对每个进程进行分析，生成数据库文件，这里需要把 mpirun 放在 hipprof 前面，再由 hipprof 来启动 HIP 程序。
2. 将多个数据库文件进行合并，生成合并后的可视化脚本。

如果 MPI 开启了日志，hipprof 可以通过增加--mpi-trace 指令对 MPI 日志进行解析并显示在可视化页面中，本次示例使用两个节点，每个节点使用两个进程，一共 4 个进程，1 个进程用于等待，其它 3 个进程执行计算后发送结束信号给等待进程。这是主要代码逻辑：

开发者社区

```
MPI_Init(NULL,NULL);
MPI_Comm_size(MPI_COMM_WORLD,&comm_sz);
MPI_Comm_rank(MPI_COMM_WORLD,&my_rank);

if(my_rank==1){
    sprintf(greeting,"Greeting from process %d of %d!",my_rank,comm_sz);
    run_kernel();
    MPI_Send(greeting,strlen(greeting)+1,MPI_CHAR,0,0,MPI_COMM_WORLD);
    printf("rank %d has returned from MPI_Send\n",my_rank);
}else if(my_rank==2){
    sprintf(greeting,"Greeting from process %d of %d!",my_rank,comm_sz);
    run_kernel();
    MPI_Send(greeting,strlen(greeting)+1,MPI_CHAR,0,0,MPI_COMM_WORLD);
    printf("rank %d has returned from MPI_Send\n",my_rank);
}else if(my_rank==3){
    sprintf(greeting,"Greeting from process %d of %d!",my_rank,comm_sz);
    MPI_Send(greeting,strlen(greeting)+1,MPI_CHAR,0,0,MPI_COMM_WORLD);
    run_kernel();
    printf("rank %d has returned from MPI_Send\n",my_rank);
}else if(my_rank==0){
    MPI_Recv(greeting,MAX_STRING,MPI_CHAR,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
    printf("%s\n",greeting);
    MPI_Recv(greeting,MAX_STRING,MPI_CHAR,2,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
    printf("%s\n",greeting);
    MPI_Recv(greeting,MAX_STRING,MPI_CHAR,3,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
    printf("%s\n",greeting);
}
MPI_Finalize();
printf("###rank %d finished\n", my_rank);
```

下面做详细步骤演示：

单节点四进程运行，看程序是否能够正常执行：

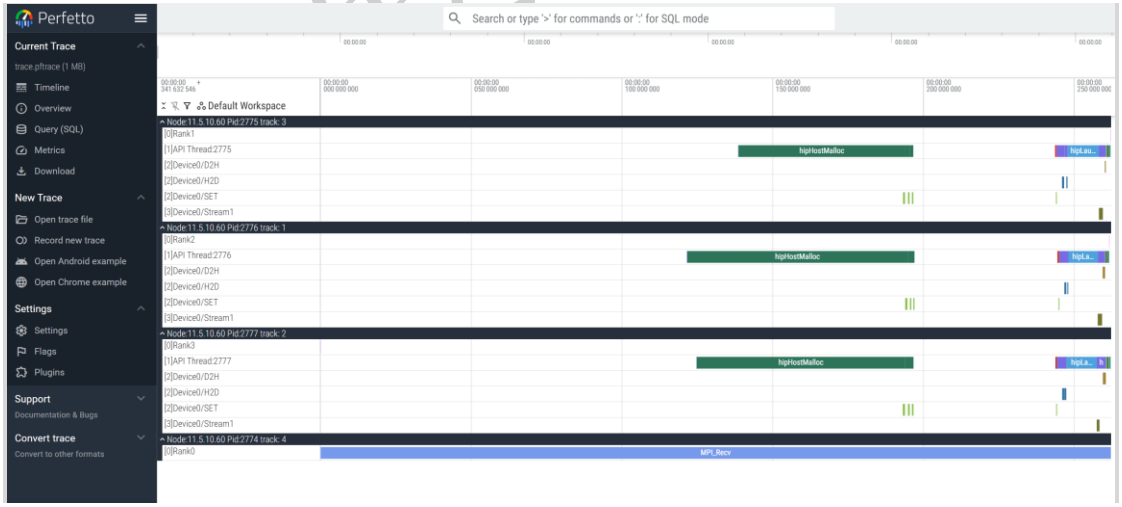
```
mpirun -np 4 ./testmpi
```

开发者

```
[root@j17r4n00 mpi]$ mpirun -np 4 ./test_mpi
> Starting...
> Starting...
> Starting...
> Using Device 0: Device 66a1
> vector size = 1048576
> size nBytes = 4194304
> Using Device 0: Device 66a1
> vector size = 1048576
> size nBytes = 4194304
> Using Device 0: Device 66a1
> vector size = 1048576
> size nBytes = 4194304
> grid (2048,1) block (512,1)
> grid (2048,1) block (512,1)
> grid (2048,1) block (512,1)
###END
rank 3 has returned from MPI_Send
###END
###END
rank 2 has returned from MPI_Send
rank 1 has returned from MPI_Send
Greeting from process 1 of 4!
Greeting from process 2 of 4!
Greeting from process 3 of 4!
###rank 1 finished
###rank 2 finished
###rank 3 finished
###rank 0 finished
[root@j17r4n00 mpi]$
```

单节点四进程加 MPI preload 日志：

mpirun -x LD_PRELOAD=mpitracer.so:/opt/hpc/software/mpi/hpcx/v2.7.4/gcc-7.3.1/lib/libmpi.so.40 -np 4 ./testmpi



双节点四进程，加上性能分析指令：

mpirun
-x LD_PRELOAD=/public/software/compiler/rocm/dtk-26.04/lib/mpitracer.so:/opt/hpc/software/mpi/hpcx/v2.7.4/gcc-7.3.1/lib/libmpi.so.40
-np 4 -H i02r1n08:2,i02r1n04:2
/public/software/compiler/rocm/dtk-26.04/bin/hipprof --hip-trace --mpi-trace ./testmpi

developer.sourcefind.cn

因为是在多节点上执行指令，避免环境和相对路径影响，这里对指令和引用库加上了绝对路径。执行过程中 mpirun 会在两个节点上分别启动两个 hipprof 程序，再由 hipprof 启动 MPI 程序，总共四个进程，分别进行性能数据的搜集，并输出四个数据库文件。

```
[root@login09 ~]$ mpirun -x LD_PRELOAD=mpitracer.so -np 4 -H b13r1n19:2,j17r4n00:2 hipprof --hip-trace --mpi-trace ./test_mpi
WARNING: /opt/hyhal/lib/cmake/rocm_smi doesn't exist, please install the latest driver(version >= 6.3.15)
The rocm_smi_lib will be used in default
WARNING: /opt/hyhal/lib/cmake/rocm_smi doesn't exist, please install the latest driver(version >= 6.3.15)
The rocm_smi_lib will be used in default
HIP_PROF:on Fri Jan 30 16:23:38 2026
HIP_PROF:process id '32218'
HIP_PROF:db file 'hip-prof-32218.db'
HIP_PROF:out put file 'result_32218'
HIP_PROF:profiling ' ./test_mpi'
HIP_PROF:process begin
HIP_PROF:on Fri Jan 30 16:23:38 2026
HIP_PROF:process id '3538'
HIP_PROF:db file 'hip-prof-3538.db'
HIP_PROF:out put file 'result_3538'
HIP_PROF:profiling ' ./test_mpi'
HIP_PROF:process begin
HIP_PROF:on Fri Jan 30 16:23:38 2026
HIP_PROF:process id '3539'
HIP_PROF:db file 'hip-prof-3539.db'
HIP_PROF:out put file 'result_3539'
HIP_PROF:profiling ' ./test_mpi'
HIP_PROF:process begin
HIP_PROF:on Fri Jan 30 16:23:38 2026
HIP_PROF:process id '32217'
HIP_PROF:db file 'hip-prof-32217.db'
HIP_PROF:out put file 'result_32217'
HIP_PROF:profiling ' ./test_mpi'
HIP_PROF:process begin
MPITRACER       Use openmpi!
MPITRACER       Use openmpi!
> Starting...
MPITRACER       Use openmpi!
MPITRACER       Inject MPI to Trace Traffic
MPITRACER       Total ranks: 4
```

终端输出如下：

```
|sumArrays(float*, float*, float*, int)                    |1            |577242                |577242                |100.0                 |

|Total                                                     |1            |577242                |577242                |100.0                 |

HIP_PROF:IP:11.10.17.61 HIP API statistics

|Name                                                      |Calls        |TotalDurationNs       |AverageNs             |Percentage            |
|hipHostMalloc                                             |3            |22485120              |7495040               |59.4785909958068      |
|hipLaunchKernel                                           |1            |8895430               |8895430               |23.5305678912022      |
|hipMemcpy                                                 |3            |4585080               |1528360               |12.1286476568973      |
|hipMalloc                                                 |3            |584330                |194776                |1.54569444488532      |
|hipHostFree                                               |3            |577540                |192513                |1.52773324953206      |
|hipStreamDestroy                                          |2            |439990                |219995                |1.16388016840671      |
|hipFree                                                   |3            |127460                |42486                 |0.337162586115864     |
|hipStreamCreate                                           |2            |55740                 |27870                 |0.147445806920589     |
|hipDeviceReset                                            |1            |30860                 |30860                 |0.0816321779972976    |
|__hipPopCallConfiguration                                 |1            |10420                 |10420                 |0.0275634249751083    |
|__hipPushCallConfiguration                                |1            |6070                  |6070                  |0.0160566208828126    |
|hipGetDeviceProperties                                    |1            |3700                  |3700                  |0.0097873965842515    |
|hipSetDevice                                              |1            |1980                  |1980                  |0.00523757979373459   |

|Total                                                     |25           |37803720              |1512148               |100.0                 |

HIP_PROF:dump perfetto trace (36/36)
HIP_PROF:finish

HIP_PROF:IP:11.10.17.61 HIPOPS API statistics

|Name                                                      |Calls        |TotalDurationNs       |AverageNs             |Percentage            |
|sumArrays(float*, float*, float*, int)                    |1            |584602                |584602                |100.0                 |

|Total                                                     |1            |584602                |584602                |100.0                 |

HIP_PROF:dump perfetto trace (36/36)
HIP_PROF:finish
HIP_PROF:dump perfetto trace (36/36)
HIP_PROF:finish
[root@login09 ~]$
```

如果需要对四个进程的性能数据放在一起做可视化分析，那么执行第二步，对四个数据进行合并，生成统一可视化文件。

```
hipprof --hip-trace --mpi-trace --db hip-prof-32217.db:hip-prof-32218.db:hip-prof-3538.db:hip-prof-3539.db
或者  hipprof --hip-trace --mpi-trace --db-path ./
```

```
[root@login09 ~]$ hipprof --hip-trace --mpi-trace --db hip-prof-32217.db:hip-prof-32218.db:hip-prof-3538.db:hip-prof-3539.db
--------------------------------------------------------------------------------
HIP_PROF:IP:11.10.17.61 HIP API statistics
--------------------------------------------------------------------------------
|Name                                |Calls          |TotalDurationNs        |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|hipHostMalloc                       |3              |22485120               |7495040                |59.4785909958068       |
|hipLaunchKernel                     |1              |8895430                |8895430                |23.53056789122022      |
|hipMemcpy                           |3              |4585080                |1528360                |12.12864765689773      |
|hipMalloc                           |3              |584330                 |194776                 |1.545694444488532      |
|hipHostFree                         |3              |577540                 |192513                 |1.527733249953206      |
|hipStreamDestroy                    |2              |439990                 |219995                 |1.163880016840671      |
|hipFree                             |3              |127460                 |42486                  |0.33716258615864       |
|hipStreamCreate                     |2              |55740                  |27870                  |0.147445806920589      |
|hipDeviceReset                      |1              |30860                  |30860                  |0.08163217799729776    |
|__hipPopCallConfiguration           |1              |10420                  |10420                  |0.02756342497510833    |
|__hipPushCallConfiguration          |1              |6070                   |6070                   |0.01605662088828126    |
|hipGetDeviceProperties              |1              |3700                   |3700                   |0.009787396584251 5    |
|hipSetDevice                        |1              |1980                   |1980                   |0.005237579793734 59   |
--------------------------------------------------------------------------------
|Total                               |25             |37803720               |1512148                |100.0                  |
--------------------------------------------------------------------------------

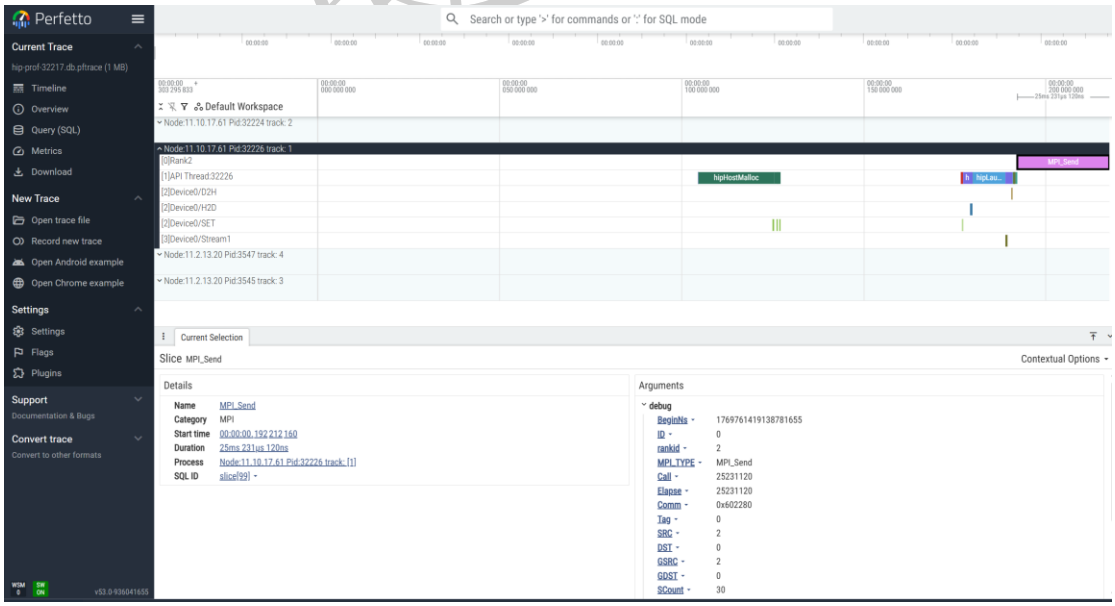
--------------------------------------------------------------------------------
HIP_PROF:IP:11.10.17.61 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                |Calls          |TotalDurationNs        |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|sumArrays(float*, float*, float*, int)|1              |584602                 |584602                 |100.0                  |
```

```
HIP_PROF:IP:11.10.17.61 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                |Calls          |TotalDurationNs        |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|sumArrays(float*, float*, float*, int)|1              |577242                 |577242                 |100.0                  |
--------------------------------------------------------------------------------
|Total                               |1              |577242                 |577242                 |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.20 HIP API statistics
--------------------------------------------------------------------------------
|Name                                |Calls          |TotalDurationNs        |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|hipHostMalloc                       |3              |22668719               |7556239                |62.4620308862576       |
|hipLaunchKernel                     |1              |7949570                |7949570                |21.9044705116538       |
|hipMemcpy                           |3              |3988637                |1329545                |10.9904034492672       |
|hipHostFree                         |3              |590695                 |196898                 |1.627617746479 54      |
|hipMalloc                           |3              |484763                 |161587                 |1.335729710995 8       |
|hipStreamDestroy                    |2              |406273                 |203136                 |1.119456140166 22      |
|hipFree                             |3              |115757                 |38585                  |0.318960118977192      |
|hipStreamCreate                     |2              |35457                  |17728                  |0.097699228424569      |
|hipDeviceReset                      |1              |31656                  |31656                  |0.087225839701633 4    |
|__hipPopCallConfiguration           |1              |9935                   |9935                   |0.027375180611 4395    |
|__hipPushCallConfiguration          |1              |5683                   |5683                   |0.015659099286 8455    |
|hipGetDeviceProperties              |1              |3272                   |3272                   |0.009015761546 11273   |
|hipSetDevice                        |1              |1581                   |1581                   |0.004356332214 05997   |
--------------------------------------------------------------------------------
|Total                               |25             |36291998               |1451679                |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.20 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                |Calls          |TotalDurationNs        |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|sumArrays(float*, float*, float*, int)|1              |572896                 |572896                 |100.0                  |
--------------------------------------------------------------------------------
|Total                               |1              |572896                 |572896                 |100.0                  |
--------------------------------------------------------------------------------
HIP_PROF:dump perfetto trace (111/111)
HIP_PROF:finish
[root@login09 ~]$
```

下载对应的 trace 文件在 chrome 浏览器中进行可视化。



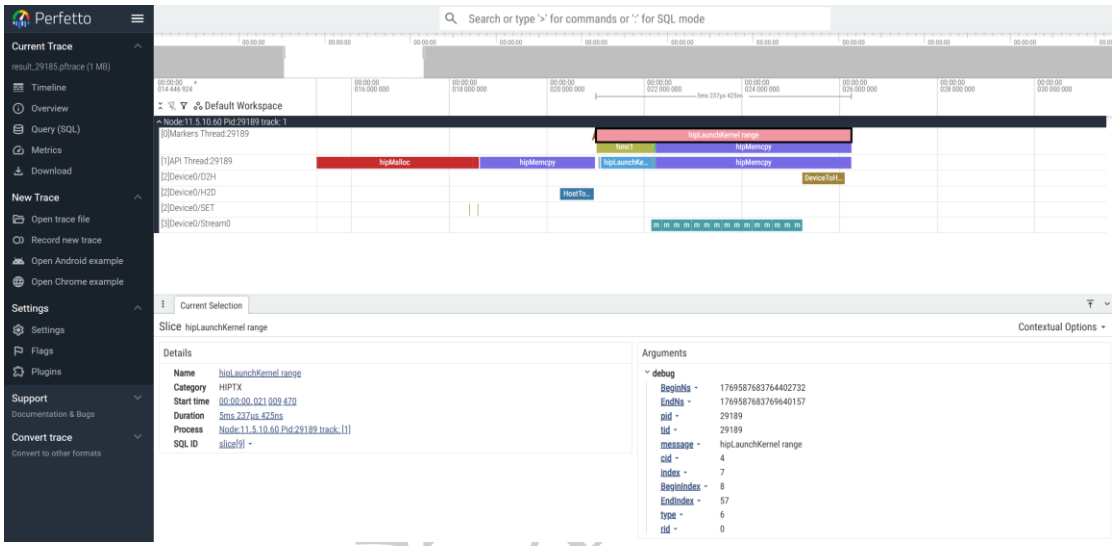
3.6 HIPTX 接口跟踪可视化

hiptx 是 hipprof 提供的外部接口，可以通过对 roctx.h 头文件的引用，添加用户需要的 API 时间点，再与 HIP API 做可视化对比。roctx.h 提供了 mark 和 range 的接口。

正确使用 ROCTX 接口后，在性能分析时候添加--hiptx-trace 指令。

```
hipprof --hip-trace --hiptx-trace ./testroctx
```

可视化结果如下：



其中，Markers Thread 中显示的内容是根据用户程序调用 roctx 中的 API 做的标记生成并显示的。

示例代码如下：

```
// Memory transfer from host to device
HIP_CHECK(hipMemcpy(gpuMatrix, Matrix, NUM * sizeof(float), hipMemcpyHostToDevice));
roctxMark("before hipLaunchKernel");
int rangeId = roctxRangeStart("hipLaunchKernel range");
roctxRangePush("func1");
launchKernel(5, gpuTransposeMatrix, gpuMatrix);
roctxRangePop();    // func1

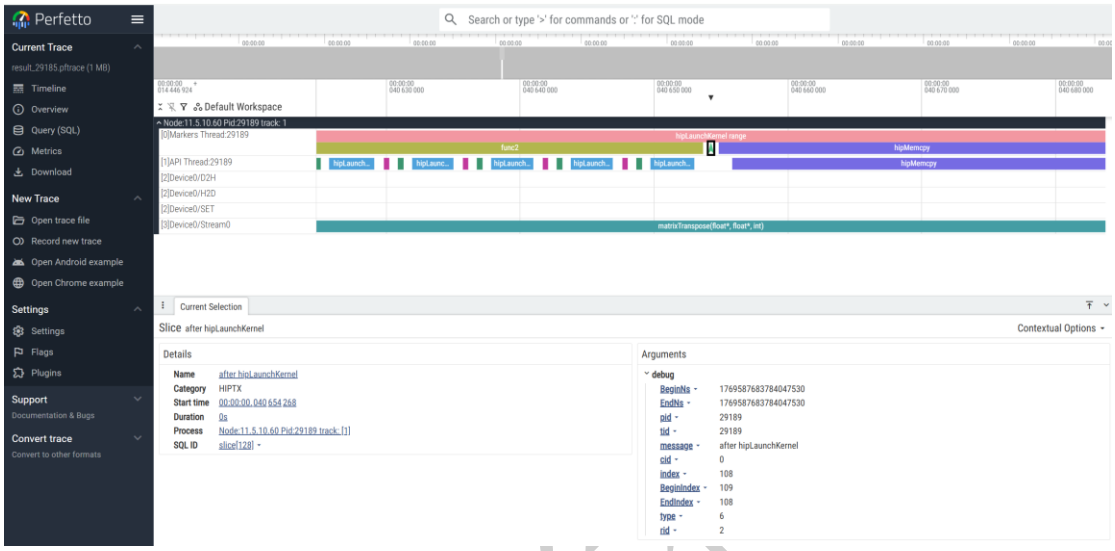
roctxRangePush("func2");
launchKernel(10, gpuTransposeMatrix, gpuMatrix);
roctxRangePop();    // func2

roctxMark("after hipLaunchKernel");
// Memory transfer from device to host
roctxRangePush("hipMemcpy");
```

```
HIP_CHECK(hipMemcpy(TransposeMatrix, gpuTransposeMatrix, NUM * sizeof(float), hipMemcpyDeviceToHost));
roctxRangePop();  // for "hipMemcpy"

roctxRangeStop(rangeId);
```

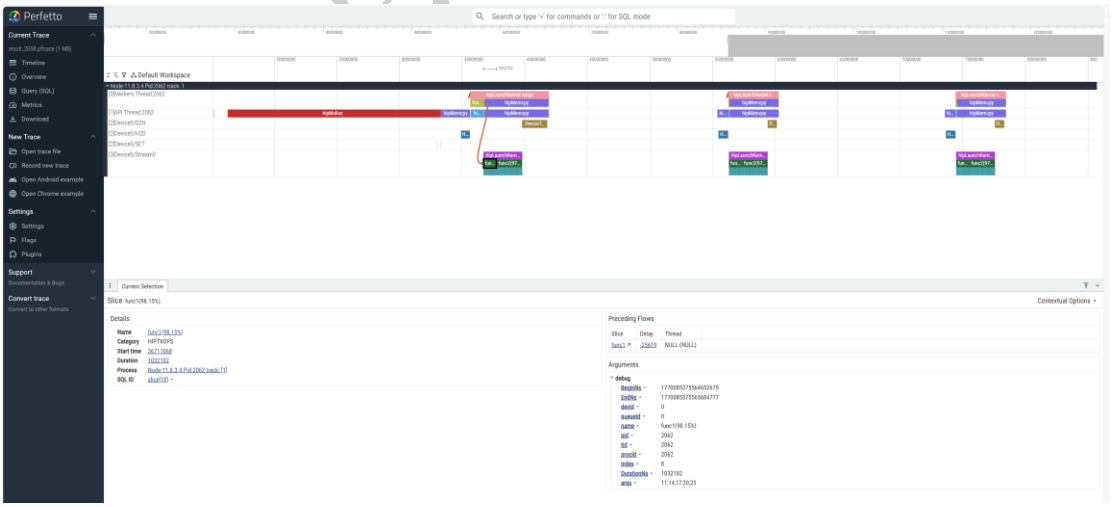
其中 Mark 信息没有时间线，在可视化页面中显示的是箭头。



在性能分析时候添加--hiptx-trace --hiptx-kernel 指令。

```
hipprof --hip-trace --hiptx-trace --hiptx-kernel ./testroctx
```

可视化结果如下：



3.7 OpenMP/OpenACC 接口跟踪可视化

对接 OpenMP/OpenACC 相关函数，OpenMP/OpenACC 相关函数可进行追踪。目前只支持 OpenMP Fortran 语言的跟踪可视化。在正确安装 DTK 以后，可以进行测试，在性能分析时候添加--omp-trace 指令。

演示使用的程序代码 target_data_device.f90 如下：

```
module test
contains
subroutine vec_mult(p, v1, v2, N)
real       ::    p(N), v1(N), v2(N)
integer ::    i,j
call init(v1, v2, N)
p = 0
!$omp target data device(2) map(to:v1,v2) map(from:p)
!$omp target device(1)
                    do i=1,N
                        p(i) = v1(i) * v2(i)
                    end do
!$omp end target
!$omp end target data
if(p(1)/=0)then
write(*,*)"p0 = ",p(1)
endif

p = 0
!$omp target data device(2) map(to:v1,v2) map(from:p)
!$omp target device(2)
                    do i=1,N
                        p(i) = v1(i) * v2(i)
                    end do
!$omp end target
!$omp end target data
if(p(1)/=6.0)then
write(*,*)"p1 = ",p(1)
endif

p = 0
!$omp target data device(2) map(to:v1,v2) map(from:p)
!$omp target device(2)
                    do i=1,N
                        p(i) = v1(i) * v2(i)
                    end do
!$omp end target
!$omp target update from(p) device(2)
!$omp end target data
if(p(1)/=6.0)then
write(*,*)"p2 = ",p(1)
endif
```

开发者社区

```
p = 0
!$omp target data device(2) map(to:v1,v2) map(from:p)
!$omp target device(2)
                    do i=1,N
                        p(i) = v1(i) * v2(i)
                    end do
!$omp end target
!$omp target update from(p) device(2)
if(p(1)/=6.0)then
                    write(*,*)"p3 = ",p(1)
  endif
!$omp end target data
end subroutine
subroutine init(v1, v2, N)
integer,intent(in)::N
real,intent(inout)::v1(N), v2(N)
v1 = 2.
v2 = 3.
end subroutine
end module
program main
use test
integer, parameter :: N=1024
real       ::    p(N), v1(N), v2(N)
call vec_mult(p, v1, v2, N)
end program
```

根据下面指令进行编译程序

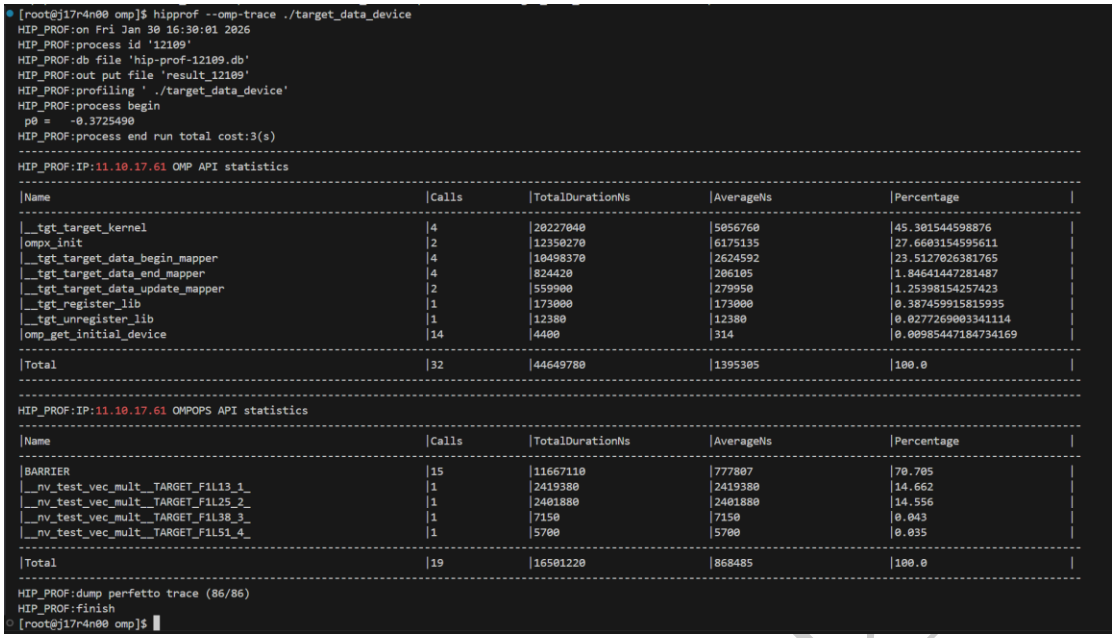
```
flang -fopenmp -fopenmp-targets=amdgcn-amd-amdhsa -Xopenmp-target=amdgcn-amd-amdhsa -march=gfx906 target_data_device.f90 -o target_data_device
```

使用 hipprof 进行 OpenMP 接口跟踪

```
hipprof --omp-trace ./target_data_device
```

执行成功后，可在终端显示 OpenMP API 统计表和核函数统计表。

开发者社区



以上表格除了输出在终端外会另外形成两个 csv 格式的文件：

result_12109.ompkernel.csv

result_12109.omptrace.csv

文件名称中 12109 是本次性能分析的进程号，将文件下载到本地后可以使用 Excel 等支持 csv 的图表工具打开。

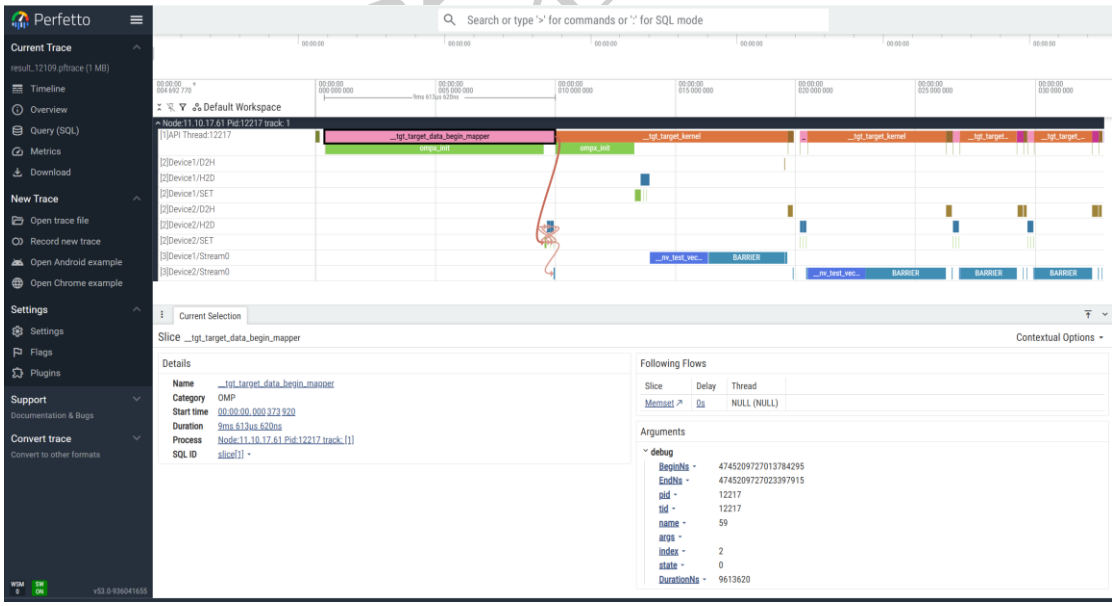
表 10.1 result_12109.omptrace.csv 文件内容

Name	Calls	TotalDurationNs	AverageNs	Percentage
__tgt_target_kernel	4	20227040	5056760	45.30154
ompx_init	2	12350270	6175135	27.66032
__tgt_target_data_begin_mapper	4	10498370	2624592	23.5127
__tgt_target_data_end_mapper	4	824420	206105	1.846414
__tgt_target_data_update_mapper	2	559900	279950	1.253982
__tgt_register_lib	1	173000	173000	0.38746
__tgt_unregister_lib	1	12380	12380	0.027727
omp_get_initial_device	14	4400	314	0.009854
Total	32	44649780	1395305	100

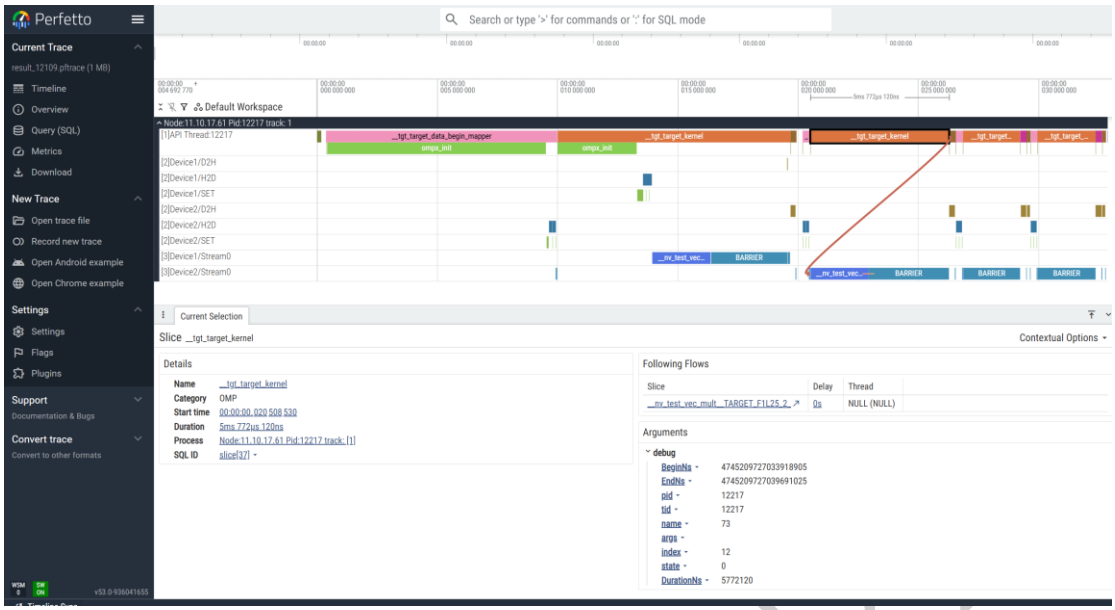
表 10.2 result_12109.ompkernel.csv 文件内容

Name	Calls	TotalDurationNs	AverageNs	Percentage
BARRIER	15	11667110	777807	70.705
__nv_test_vec_mult__TARGET_F1L13_1_	1	2419380	2419380	14.662
__nv_test_vec_mult__TARGET_F1L25_2_	1	2401880	2401880	14.556
__nv_test_vec_mult__TARGET_F1L38_3_	1	7150	7150	0.043
__nv_test_vec_mult__TARGET_F1L51_4_	1	5700	5700	0.035
Total	19	16501220	868485	100

可视化结果如下：



左侧区域显示的是性能数据的目录结构，[1]API Thread 行即 OpenMP 接口信息，[2]Device 表示的是 device 上的 memcpy 信息，[3] device 表示核函数在每个流上的运行信息，右侧区域是具体每个操作所占用的时间和流程。



OpenMP 的核函数可能存在隐形拷贝情况，如果有隐形拷贝，在可视化中一个 OMP Runtime API 会关联多个事件流。

3.8 RCCL 接口跟踪可视化

对接 RCCL 相关函数，RCCL 相关算子函数可进行追踪，hipprof 追踪 HIP API 的指令都可以执行。在正确安装 DTK 以后，可以编译 RCCL_TESTS 进行测试，在性能分析候添加--rccl-trace 指令。

RCCL 接口跟踪分为单节点和多节点两类。

单节点执行指令如下：

```
hipprof --rccl-trace ./all_reduce_perf -b 8 -e 1G -f 2 -g 4
```

执行成功后，除 HIP 接口的统计表和核函数的统计表外，还可在终端显示 RCCL 算子接口统计表和 RCCL 核函数的统计表。

```
HIP_PROF:IP:11.5.10.60 RCCL API statistics
|Name                                                  |Calls         |TotalDurationNs          |AverageNs                |Percentage               |
|ncclAllReduce                                         |4             |86490                    |21622                    |100.0                    |
|Total                                                 |4             |86490                    |21622                    |100.0                    |

HIP_PROF:IP:11.5.10.60 RCCLOPS API statistics
|Name                                                  |Calls |TotalDurationNs |AverageNs |TotalBytes |TotalBWGB/s |AverageBWGB/s |Percentage |
|ncclDevKernel_AllReduce_Ring_LL_Sum_f32(ncclDevKern...|4     |29361687        |7340421   |16384      |0.088661    |0.022165      |100.0      |
|Total                                                 |4     |29361687        |7340421   |16384      |0.088661    |0.022165      |100.0      |
HIP_PROF:dump perfetto trace (3679/3679)
HIP_PROF:finish
```

分析完成后会输出上图中的信息，Name 列是函数或者接口的名称；Calls 表示调用的次数；TotalDurationNs 是函数运行过程中总共消耗的时间，以纳秒为单位；

AverageNs 是每次调用平均耗时，以纳秒为单位；Percentage 是该函数占用的总时间的百分比；TotalBytes 是函数运行过程中总传输字节数，以字节为单位；TotalBWGB/s 为函数运行过程中总传输速率，以 GB/s 为单位；AverageBWGB/s 为每次调用平均传输速率，以 GB/s 为单位；最后一行 total 是表格的统计总计信息。

以上表格除了输出在终端外会另外形成两个 csv 格式的文件：

result_1198.rccltrace.csv

result_1198.rcclkernel.csv

文件名称中 1198 是本次性能分析的进程号，将文件下载到本地后可以使用 Excel 等支持 csv 的图表工具打开。

表 11.1 result_1198.rccltrace.csv 文件内容

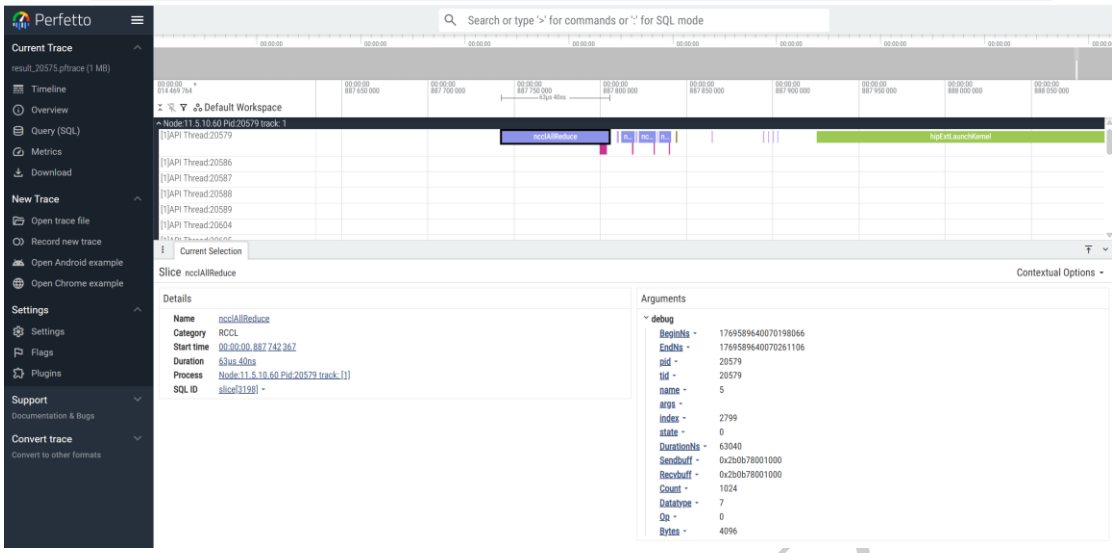
Name	Calls	TotalDurationNs	AverageNs	Percentage
ncclAllReduce	4	86490	21622	100.0
Total	4	86490	21622	100.0

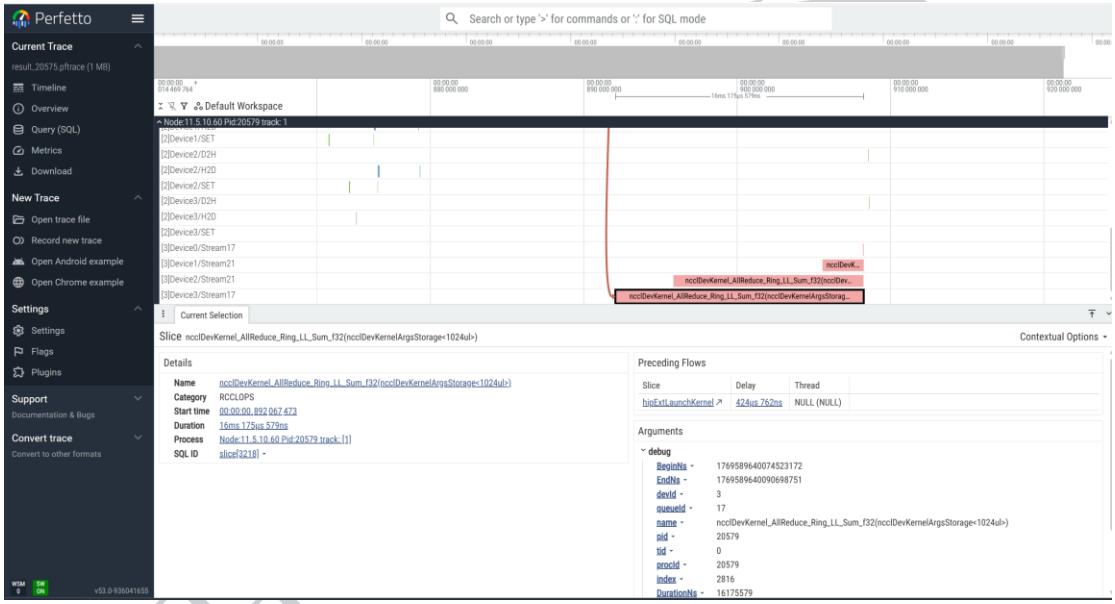
表 11.2 result_1198.rcclkernel.csv 文件内容

Name	Calls	TotalDurationNs	AverageNs	TatalBytes	TotalBWGB/s	AverageBWGB/s	Percentage
ncclDevKernel_AllReduce_Ring_LL_Sum_f32(ncclDevKernelArgsStorage<1024ul>)	4	29361687	7340421	16384	0.088661	0.022165	100.0
Total	4	29361687	7340421	16384	0.088661	0.022165	100.0

可视化结果如下：

开发者社区





左边目录结构没有变化，API Thread 中记录了 RCCL API 的 timeline 显示。Device/Stream 中记录了每个核函数的详细信息，主要变化如下：

(1)核函数名称显示为通讯过程中的真实协议、算法、操作类型、数据类型。

(2)Args 中增加 rank_id、nranks、sendBytes/recvBytes、BwGBPS 信息。

在多节点多进程的 RCCL 程序性能分析过程需要两个步骤：

1. 每个进程启动一个 hipprof 分别对每个进程进行分析，生成数据库文件，这里需要把 mpirun 放在 hipprof 前面，再由 hipprof 来启动 RCCL 程序。
2. 将多个数据库文件进行合并，生成合并后的可视化脚本。

以 rccl-tests 为例，mpi 开启两个节点，每个节点 4 个进程：

mpirun -np 8 -H b13r2n01:4,b13r3n05:4 sh runtest.sh

runtest.sh 脚本如下：

```
#!/bin/bash
source ${ROCM_PATH}/env.sh
hipprof –rccl-trace NCCL_IB_HCA=mlx5_0:1 ${RCCL_TEST}/build/all_reduce_perf -b 8 -e 10M -f 2 -g 1
```

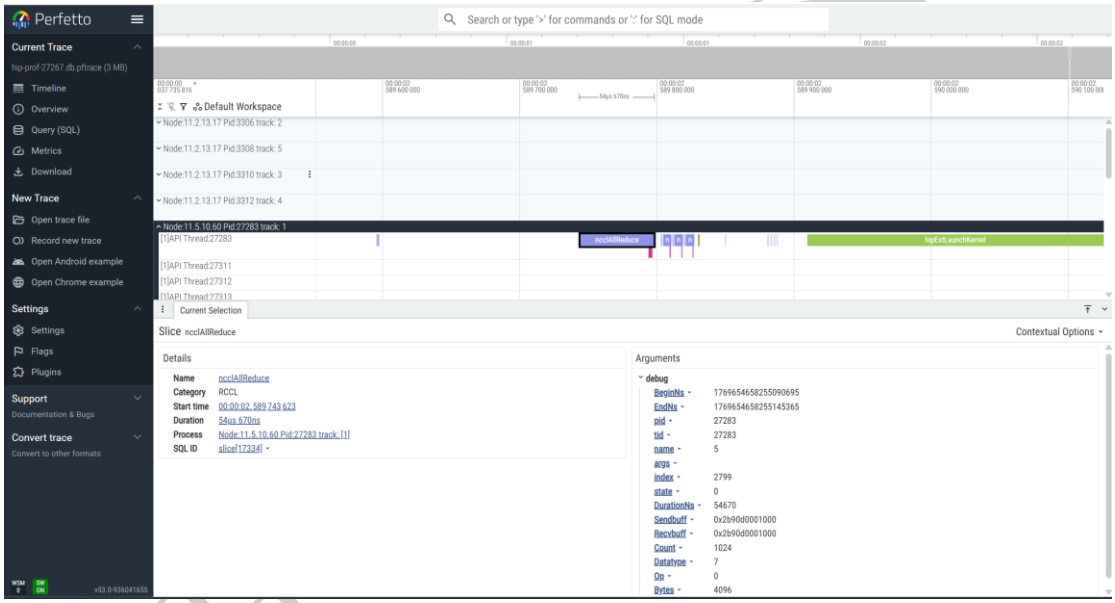
执行合并数据库的指令

```
hipprof --rccl-trace --db hip-prof-27267.db:hip-prof-3293.db:hip-prof-3294.db:hip-prof-3295.db:hip-prof-3296.db
```

或者 hipprof --rccl-trace --db-path ./

将会对这些 db 数据中的 RCCL 数据合并导出到一个 trace 文件。

可视化结果如下：



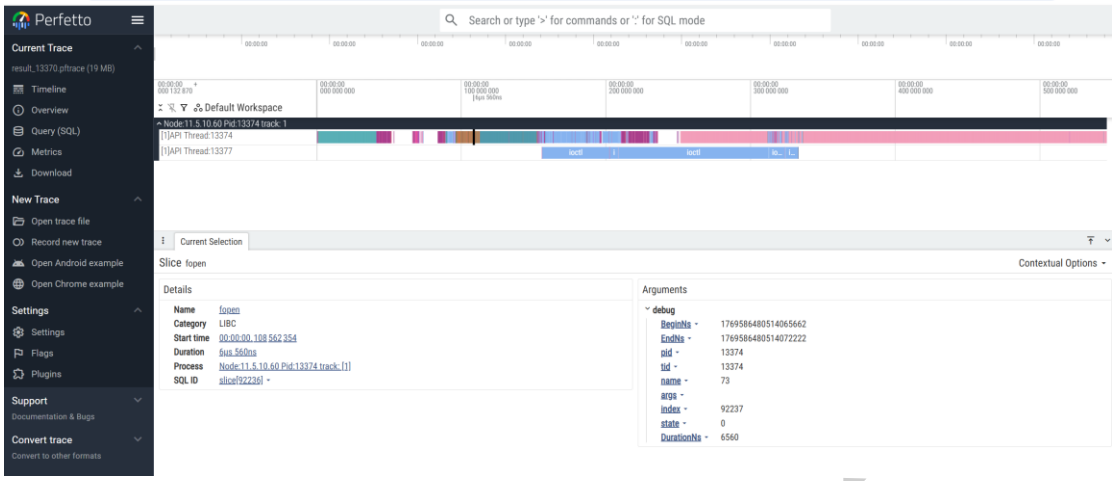
3.9 LIBC 接口跟踪可视化

和--hip-trace 指令类似，用户开启 LIBC API 的性能分析。在正确安装 DTK 以后，在性能分析时添加--libc-trace 指令。

执行指令如下：

```
hipprof --libc-trace ./ MatrixTranspose
```

执行成功后，可获取 LIBC 接口的统计表和 trace 可视化文件。生成的 trace 可视化如下图所示：

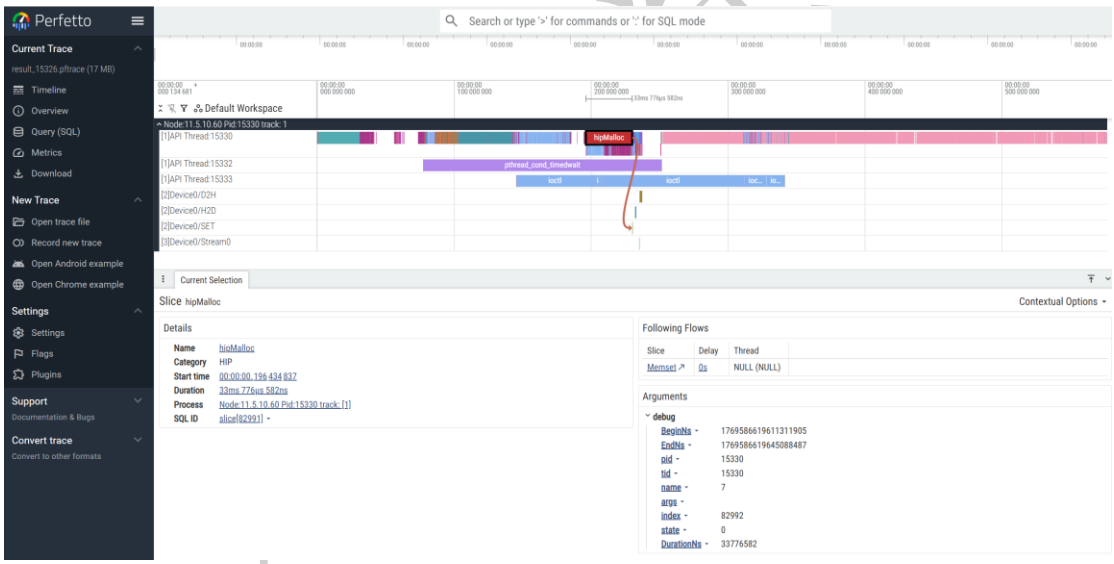


其中 Runtime API 显示 libc api 的具体调用信息。

执行联合--hip-trace 指令如下：

```
hipprof --hip-trace --libc-trace ./ MatrixTranspose
```

生成的 trace 可视化如下图所示，可查看 HIP 接口调用的 libc 函数堆栈。



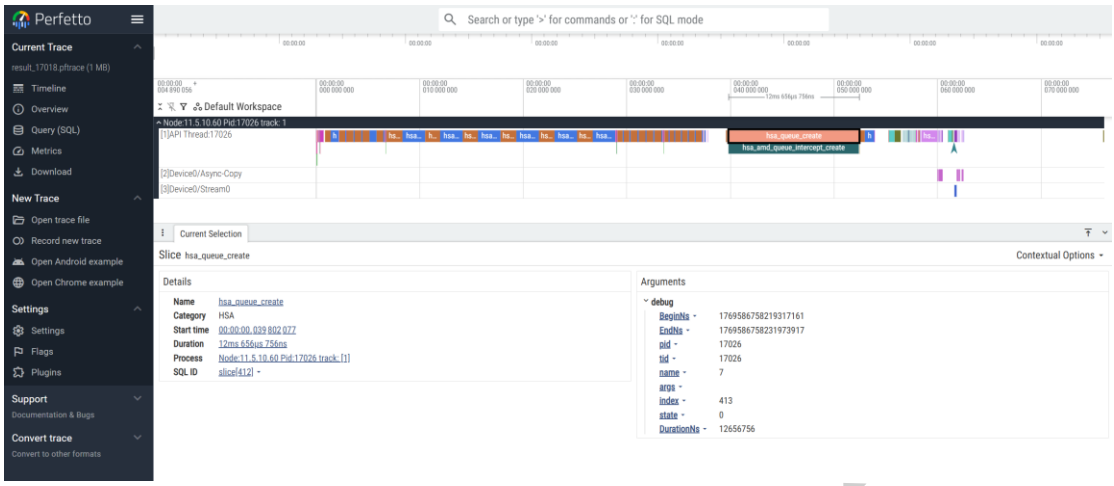
3.10 HSA 接口跟踪可视化

和--hip-trace 指令类似，用户开启 HSA API 的性能分析。在正确安装 DTK 以后，在性能分析时添加--hsa-trace 指令。

执行指令如下：

```
hipprof --hsa-trace ./MatrixTranspose
```

执行成功后，可获取 HSA 接口的统计表和 trace 可视化文件。生成的 trace 可视化如下图所示：

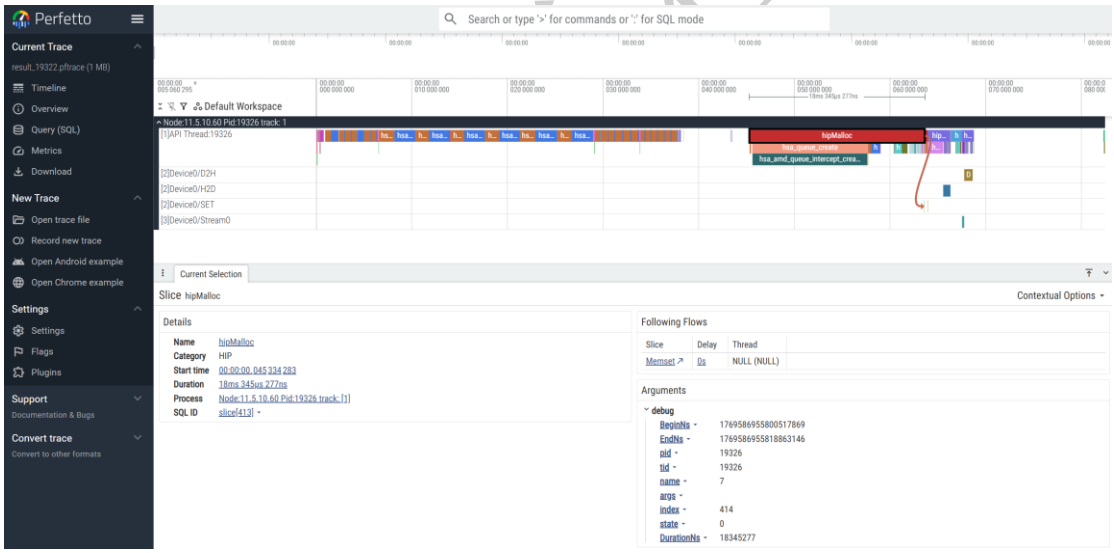


其中 API Thread 显示 hsa api 的具体调用信息，而 Device0/Async-Copy 表示的是 device0 上的 memcpy 信息。

执行联合--hip-trace 指令如下：

```
hipprof --hip-trace --hsa-trace ./MatrixTranspose
```

生成的 trace 可视化如下图所示，可查看 HIP 接口调用的 hsa 函数堆栈。



3.11 OpenCL 接口跟踪可视化

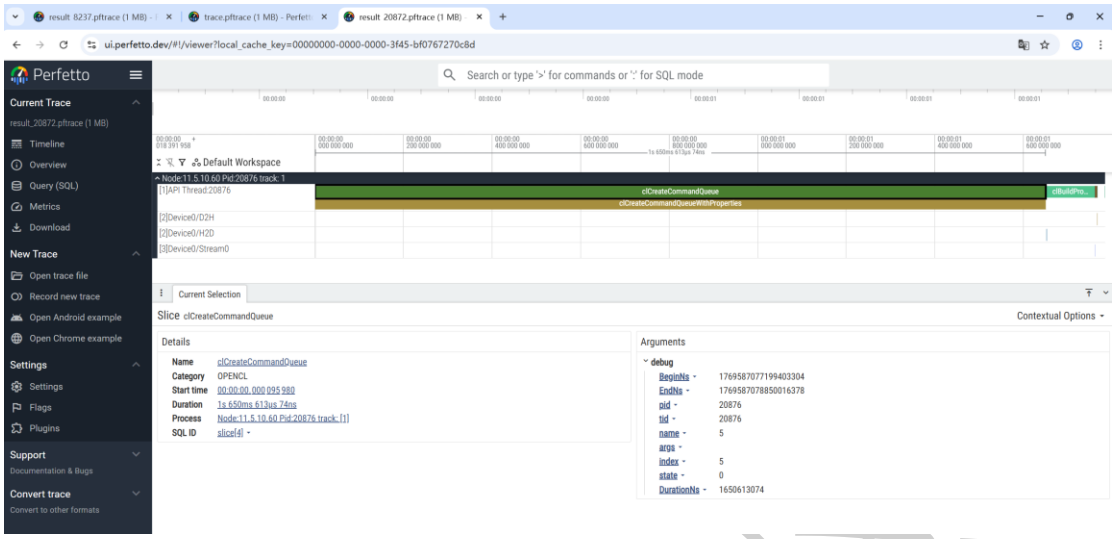
和--hip-trace 指令类似，用户开启 OpenCL API 的性能分析。在正确安装 DTK 以后，在性能分析时添加--cl-trace 指令。

执行指令如下：

```
hipprof --cl-trace ./cl_MatrixTranspose
```

执行成功后，可获取 OpenCL 接口的统计表和可视化文件。生成的可视化如下图

所示：

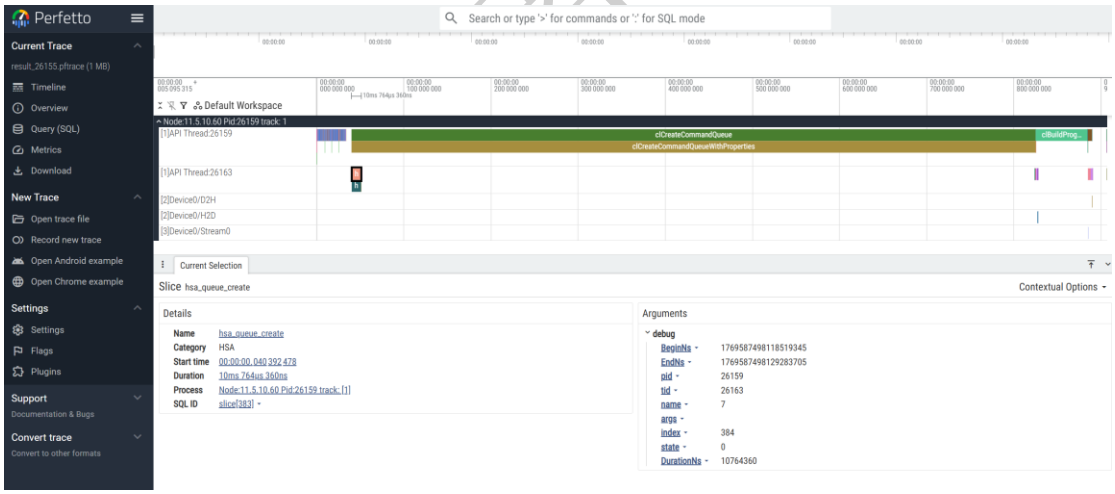


其中 Runtime API 显示 opencl api 的具体调用信息，而 Memcpy on device 表示的是 device 上的 memcpy 信息。

执行联合--hsa-trace 指令如下：

```
hipprof --cl-trace --hsa-trace ./MatrixTranspose
```

生成的 trace 可视化如下图所示，可查看 OpenCL 接口调用的 hsa 函数堆栈。

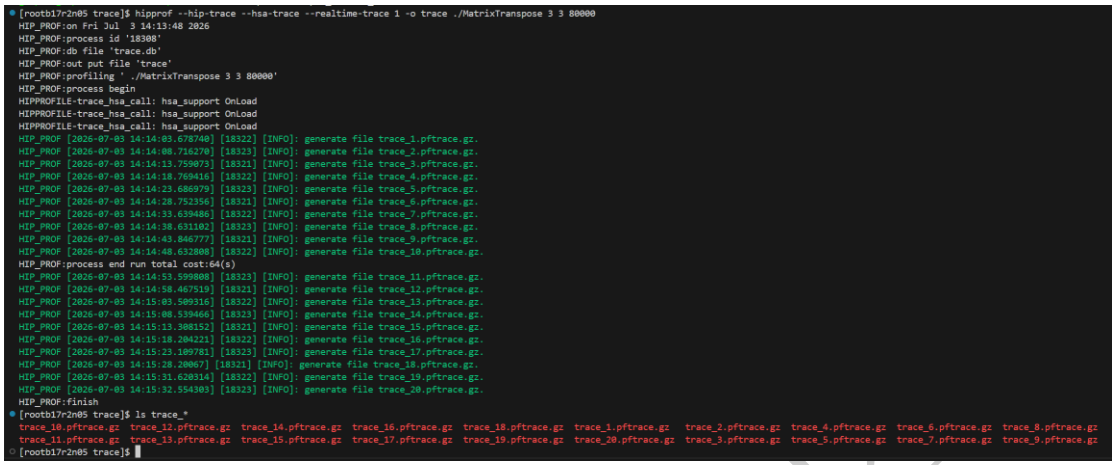


3.12 实时导出 Trace 数据

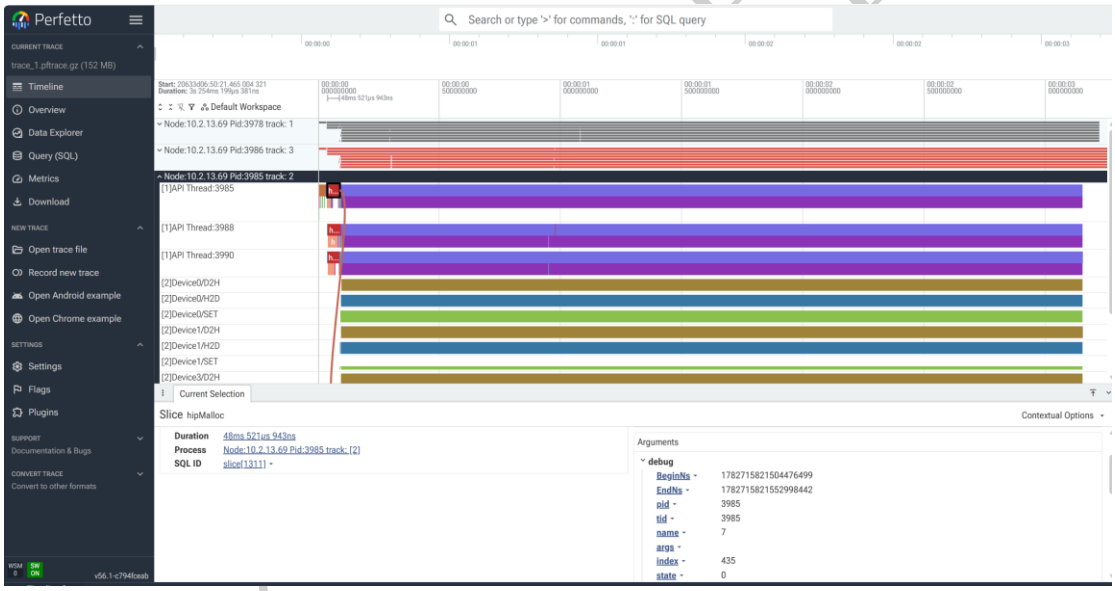
DB 模式（原模式）：内存中的性能数据先存入 DB 文件，再查询导出到性能数据文件。实时模式：内存中的性能数据直接导出到性能数据文件。

支持的 trace 数据类型有：hip、hiptx、hsa、libc、rccl、omp、opencl、kernel、copy、barrier 类型的 trace，不支持的 trace 数据类型有：mpi、hiptx-kernel 类型的 trace（这些数据无法实时传输到 hipprof）。

hipprof 现在模式使用 db 模式导出 trace 数据，需要通过实时模式导出 trace 数据时，示例如下图所示：实时模式下减去了 DB 文件的生成、查询导出部分，磁盘占用能降至 DB 模式的 50%，耗时可降至 DB 模式耗时的 10%。



实时数据使用 https://ui.perfetto.dev/网站打开效果如下：

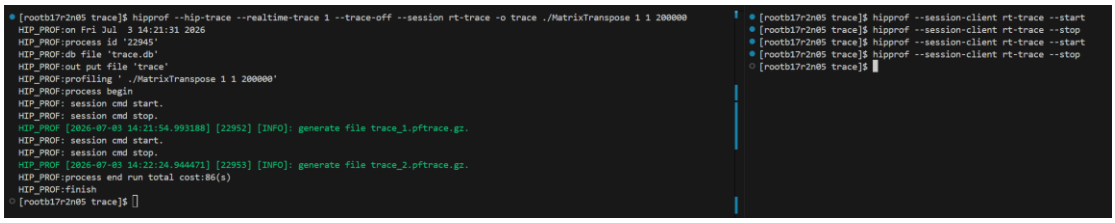


3.13 分段实时导出 Trace 数据

当前功能对应的问题：用户长时间采集运行程序时，采集的数据过多，数据存储成本较高、用户需要仅关注关键几个阶段的性能数据。

分段实时导出 Trace 数据功能如下图所示，实时导出功能和会话功能相结合，通过开始、停止指令控制性能数据的采集动作；实时导出模式下 hipprof 端超过 10 秒未收到新 trace 数据，就会将已接收到的 trace 数据写入到 trace 文件中；

下图中采集了一个进程两个阶段的性能数据，当前功能可有效降低数据存储成本，帮助用户聚焦关键性能分析场景



3.14 PMC 性能分析

通过性能追踪（trace）分析，可以整体把握 HIP API 及 kernel 函数的执行情况，从而进行流程优化和热点定位。如果发现性能瓶颈位于某个 kernel 函数，则可以进一步使用性能监控计数器（PMC）指令进一步对 kernel 函数的进行细粒度性能和行为分析。

```
hipprof --pmc ./demo
```

如果需要针对特定的核函数，可通过--kernel-name 指定函数名来限定输出范围：

```
hipprof --pmc --kernel-name "Cijk_Ailk_Bjlk_SB_MT128x128x16_SE_K1_TT8_8" ./demo
```

命令执行结束后将产生 PMC 数据文件，命名格式为 pmc_results_<PID>.txt（其中<PID>为进程标识号）。文件内容包含 kernel 函数的详细的性能指标信息，可用于定位性能瓶颈并指导优化，例如：

kernel-name:"Cijk_Ailk_Bjlk_SB_MT128x128x16_SE_K1_TT8_8"

kernel dispatch index	6
gpu id	0
dispatch queue id	0
dispatch queue index	13
thread id	50
grid size	65536
work group size	256
shared memory size	32768
scratch size	0
vgpr count	15
sgpr count	2
fbarrier count	0
signal handle	0x0
kernel time	0.001916(s)
processed ALU instructions	84.688568(%)
performance	9150.809840(Gflops)

shared memory operation 8912896
shared memory bank conflict 0.000000(%)
L1 cache unit is active 13.244532(%)
L1 cache unit is stalled5.093312(%)
L2 cache write unit is stalled0.455153(%)
L2 cache hit rate89.157204(%)

除默认的文本格式外，PMC 也支持其他结构化输出。例如，使用 --pmc-type 3 可生成 CSV 格式：

```
hipprof --pmc --pmc-type 3 ./demo
```

生成的 CSV 格式的表格数据示例如下：

kernel-name	dispatch	gpu-	queue	queu	tid	grd	wgr	lds	scr	vgpr	sgpr	fbar	sig	time	VALUBusy	performan	LDSInsts	LDSBankC	L1Memory	L1Memory	L2WriteUn	L2CacheHit
Cijk_Ailk_Bjlk_SB_MT128x	6	0	0	13	62	65536	256	32768	0	15	2	0	0x0	0.001932	83.96563	9072.695	8912896	0	13.76974	5.795308	0.362976	89.29555
Cijk_Ailk_Bjlk_SB_MT128x	7	0	0	16	62	65536	256	32768	0	15	2	0	0x0	0.001942	83.51913	9024.449	8912896	0	13.60839	5.565203	0.294692	89.31583
Cijk_Ailk_Bjlk_SB_MT128x	8	0	0	19	62	65536	256	32768	0	15	2	0	0x0	0.001939	83.67217	9040.985	8912896	0	13.67155	5.805173	0.424664	89.28464
Cijk_Ailk_Bjlk_SB_MT128x	9	0	0	22	62	65536	256	32768	0	15	2	0	0x0	0.001941	83.59825	9032.999	8912896	0	13.19966	5.3462	0.400066	89.27204
Cijk_Ailk_Bjlk_SB_MT128x	10	0	0	25	62	65536	256	32768	0	15	2	0	0x0	0.001945	83.42421	9014.192	8912896	0	13.38785	5.577544	0.400793	89.23628
Cijk_Ailk_Bjlk_SB_MT256x	12	0	0	29	62	131072	256	25600	0	9	13	13467456	0x0	0.001707	93.36368	10088.18	14155776	0	25.96736	1.138163	1.671268	89.47449

CSV 格式适合 kernel 函数数量较多、需要进行批量统计或者对比分析的场景。

当想要获取其他类型 pmc 时，例如期望获取各流水线使用情况，可使用下列命令：

```
hipprof   --pmc util ./demo
```

命令执行完成后将产生 PMC 数据文件，文件内容如下所示：

kernel-name:"_ZN2at6native29vectorized_elementwise_kernelILi4EZNS0_25bfloat16_copy_kernel_cudaERNS_18TensorIteratorBaseEEUlfE_St5arrayIPcLm2EEEEviT0_T1_"
kernel dispatch index 314
gpu id 0
dispatch queue id 0
dispatch queue index 359
process id 5333
thread id 5333
grid size 16384
work group size 256
shared memory size 0
scratch size 0
vgpr count 20
sgpr count 16
signal handle 0x0
kernel time 0.000005(s)
CU Utilization 26.8127801368(%)
SALU Utilization 0.3019580090(%)
VALU Utilization 1.0379806558(%)
VMEM Utilization 0.0754895022(%)

Branch Utilization	0.0566171267(%)
Vector L1 Data Cache Utilization	100.0000000000(%)
Local Data Share Utilization	0.0000000000(%)
L2 Cache Utilization	17.3698100967(%)

结果文件中除一些通用指标外，还展示了各个流水线利用率情况，当前共支持七种类型的指标组合。

3.15 寄存器压力分析

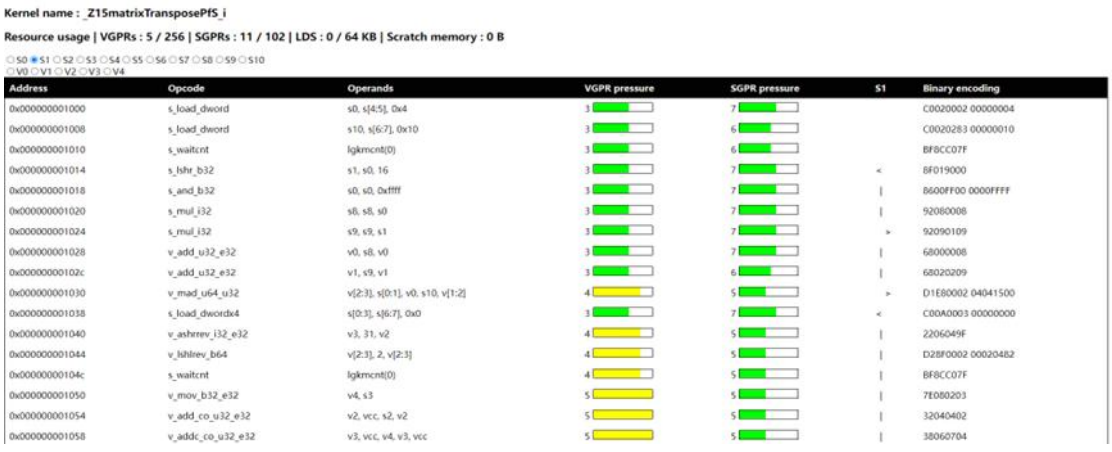
使用寄存器压力分析功能可以分析核函数内每个阶段寄存器的使用情况、压力情况。

执行 hipprof --codeobj-analyze <elf file> 命令后，按照提示选择核函数，分析步骤如下：

```
$ hipprof --codeobj-analyze MatrixTranspose
Found 6 code object(s) in the file:
  [1] amdgcn-amd-amdhsa--gfx906 (31768 bytes)
  [2] amdgcn-amd-amdhsa--gfx926 (33048 bytes)
  [3] amdgcn-amd-amdhsa--gfx928 (33048 bytes)
  [4] amdgcn-amd-amdhsa--gfx936 (33048 bytes)
  [5] amdgcn-amd-amdhsa--gfx938 (33048 bytes)
  [6] amdgcn-amd-amdhsa--gfx92a (33048 bytes)
input and select elf file index: 5
Found 1 symbol(s) in the code object:
  [1] _Z15matrixTransposePfS_ii
input and select symbol index: 1
analyze finish!
```

输出文件：

MatrixTranspose-amdgcn-amd-amdhsa--gfx938.out.obj.html



Kernel name : _Z15matrixTransposePfS_i

Resource usage | VGPRs : 5 / 256 | SGPRs : 11 / 102 | LDS : 0 / 64 KB | Scratch memory : 0 B

Address	Opcode	Operands	VGPR pressure	SGPR pressure	S1	Binary encoding
0x000000001000	s_load_dword	s0, s[4:5], 0x4	3	7		C0020002 00000004
0x000000001008	s_load_dword	s10, s[6:7], 0x10	3	6		C0020283 00000010
0x000000001010	s_waitcnt	lgkmcnt(0)	3	6		BF8CC07F
0x000000001014	s_lshr_b32	s1, s0, 16	3	7	<	8F019000
0x000000001018	s_and_b32	s0, s0, 0xffff	3	7	|	8600FF00 0000FFFF
0x000000001020	s_mul_i32	s8, s8, s0	3	7	|	92080008
0x000000001024	s_mul_i32	s9, s9, s1	3	7	>	92090109
0x000000001028	v_add_u32_e32	v0, s8, v0	3	7	|	68000008
0x00000000102c	v_add_u32_e32	v1, s9, v1	3	6	|	68020209
0x000000001030	v_mad_u64_u32	v[2:3], s[0:1], v0, s10, v[1:2]	4	5	>	D1E80002 04041500
0x000000001038	s_load_dwordx4	s[0:3], s[6:7], 0x0	3	7	<	C00A0003 00000000
0x000000001040	v_ashrrev_i32_e32	v3, 31, v2	4	5	|	2206049F
0x000000001044	v_lshlrev_b64	v[2:3], 2, v[2:3]	4	5	|	D28F0002 00020482
0x00000000104c	s_waitcnt	lgkmcnt(0)	4	5	|	BF8CC07F
0x000000001050	v_mov_b32_e32	v4, s3	5	5	|	7E080203
0x000000001054	v_add_co_u32_e32	v2, vcc, s2, v2	5	5	|	32040402
0x000000001058	v_addc_co_u32_e32	v3, vcc, v4, v3, vcc	5	5	|	38060704

3.16 CoreDump 分析

首先使用环境变量让崩溃的 HIP 程序生成 CoreDump 文件，具体环境变量

设置如下：

环境变量	含义
HSA_ENABLE_COREDUMP	设置为“1”表示需要保存 Core Dump 文件
HSA_ENABLE_COREDUMP_MEMORY	设置为“1”表示需要保存内存快照到文件，其他则表示不保存内存快照文件
HSA_COREDUMP_SAVE_DIR	设置 Core Dump 文件的保存位置，默认保存在“/tmp”

命令示例如下：

```
export HSA_ENABLE_COREDUMP=1
export HSA_ENABLE_COREDUMP_MEMORY=1
./MatrixTranspose
```

Core Dump 文件默认保存在“/tmp/hyd-core-dump_程序名_进程号_时间戳”下。该文件夹下主要包括两部分，分别是核心转储文件“core_0”以及内存快照信息文件夹“mems”。

打开 Core Dump 所在文件夹，执行“hipprof --input-core core_0”解析 CoreDump 文件。

生成的文件如下：

文件	文件内容
agent_*	程序崩溃发生在哪个 HCU 上，及该 HCU 的硬件信息和 Wave 的初略统计。
environment	运行程序时的环境变量。
fault_info	发生 Core Dump 的错误原因汇总。包括产生错误的 Code Object、产生错误的 Queue、产生错误的 Wave 及相关的寄存器数据和反汇编信息。
codes/code_object_*	所有的 Kernel Code Object 的文件。每个 Code Object 对应三个文件，分别是原始文件、汇编文件和信息文件。
queues/*	运行过程中建立的 Queue 信息统计。该目录下又按照编号分为 0，1，2...子文件夹，记录了每个 Queue 的具体统计，在每个 queue 文件夹中，又有不同的文件：aql_rb 和 cwsr 都是

开发者社区

	Queue 的原始二进制数据。其中存在*.kd_args 文件为核函数调用参数。

fault_info 文件输出示例：

```
Code Object:

/tmp/hyd-core-dump_matrix.core.out_1785747_20250617102514659024/codes/code_object_1:  file format elf64-amdgpu

Disassembly of section .text:

0000000000001000 <_Z15matrixTransposePfS_i>:
; _Z15matrixTransposePfS_i():
v_add_co_u32_e32 v0, vcc, s0, v0        // 00000000107C: 32000000
v_addc_co_u32_e32 v1, vcc, v1, v3, vcc          // 000000001080: 38020701
v_add_co_u32_e32 v0, vcc, 0x8cd15000, v0        // 000000001084: 320000FF 8CD15000
v_addc_co_u32_e32 v1, vcc, 4, v1, vcc           // 00000000108C: 38020284
s_waitcnt vmcnt(0)          // 000000001090: BF8C0F70
global_store_dword v[0:1], v2, off offset:1436 // 000000001094: DC70859C 007F0200
s_endpgm        // 00000000109C: BF810000

PC offset: 0X109C
```

4 泄漏检测功能演示

4.1 泄漏检测

启用 leak-check 功能后，hipprof 会记录应用程序整个运行周期内的申请释放内存、显存、流和事件的信息，在程序运行结束后，统计并输出内存泄漏报告。

➢编译程序

```
hipcc -g -O0 hipprof_leak_check_test.cc -o hipprof_leak_check_test
```

-g -O0 编译程序时保存文件名行号信息，用于定位泄漏位置

➢使用 hipprof 进行泄漏检测

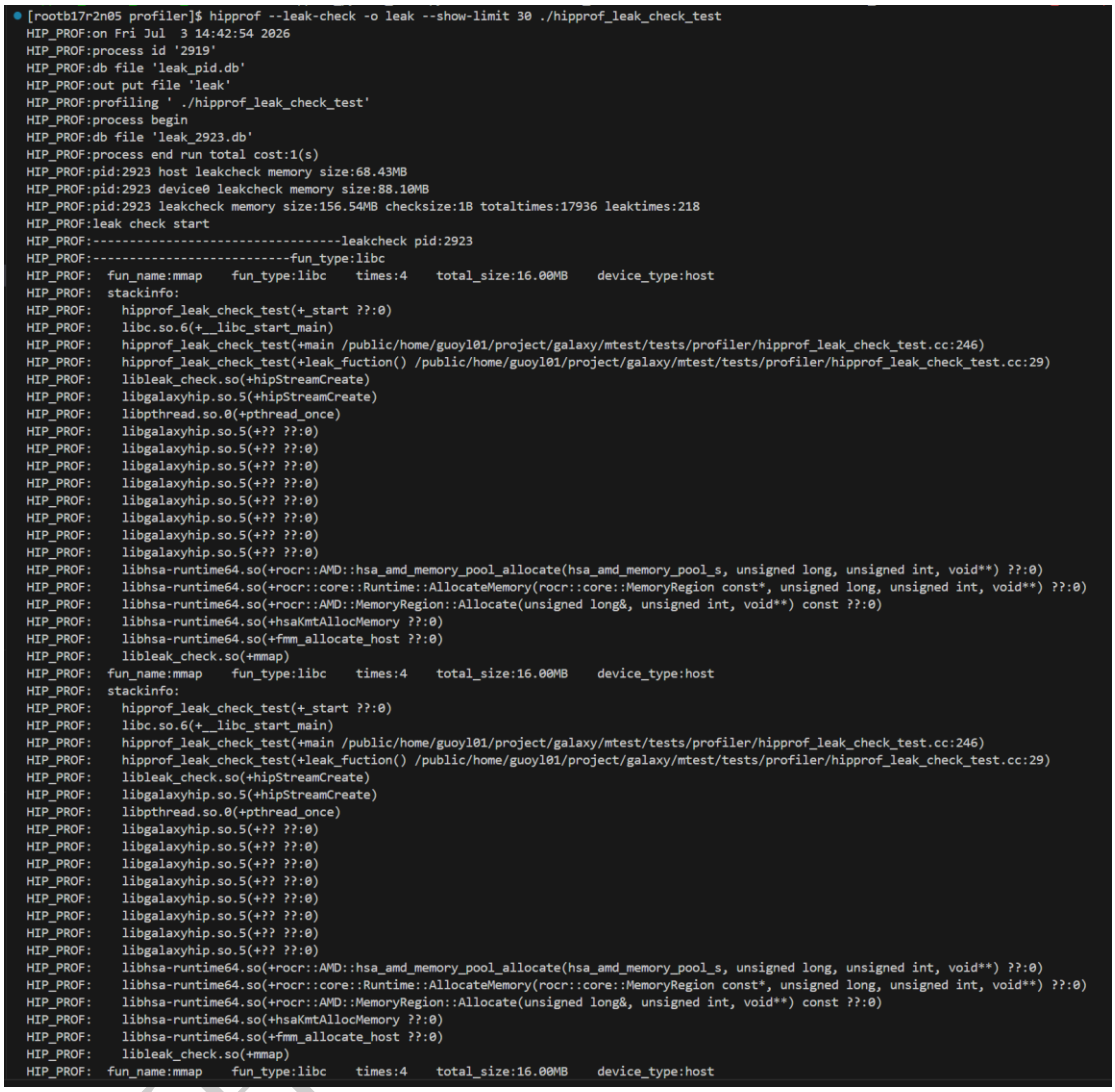
```
hipprof --leak-check -o leak --show-limit 30 ./hipprof_leak_check_test
```

--leak-check 开启泄漏检测功能

--o leak 指定输出泄漏报告的文件名称前缀

--show-limit 30 指定输出泄漏函数堆栈 30 个，默认 5 个。

终端显示效果如下，泄漏次数多的堆栈排在前面，泄漏次数相同时泄漏 size 大的排在前面：



HIP_PROF:db file 'leak_2923.db'

pid 为 2923 的进程的内存泄漏信息被记录在数据库 leak_2923.db

HIP_PROF:pid:2923 host leakcheck memory size:68.42MB

pid 为 2923 的进程内存泄漏 68.42MB

HIP_PROF:pid:2923 device0 leakcheck memory size:88.09MB

pid 为 2923 的进程显卡 0 显存泄漏 88.09MB

HIP_PROF:pid:2923 leakcheck memory size:156.52MB checksize:1B totaltimes:17674 leaktimes:218

pid 为 2923 的进程共泄漏 156.52MB，检测大小是 1 字节，共有 17674 次内存或显存申请，发生泄漏有 218 次

HIP_PROF:leak check start

HIP_PROF:----------leakcheck pid:2923，

下面是 pid 为 2923 的进程的泄漏堆栈信息

fun_name:mmap 发生泄漏的函数名称

fun_type:libc 泄漏函数的类型，支持的类型有 libc、hip

time:4 当前堆栈发生泄漏的次数为 4

total_size:16.00MB 当前堆栈发生泄漏的大小为 16.00MB

device_type:host 泄漏类型为内存，支持的类型有 host(内存)、device(显存)

下图是 hip 类型的泄漏函数堆栈

```
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:new    fun_type:libc    times:1    total_size:8.00KB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:103)
HIP_PROF:    libleak_check.so(+hipGraphCreate)
HIP_PROF:    libgalaxyhip.so.5(+hipGraphCreate)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libleak_check.so(+operator new(unsigned long))
HIP_PROF:-------------------------fun_type:hip
HIP_PROF:  fun_name:hipMalloc3DArray    fun_type:hip    times:1    total_size:40.00MB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:61)
HIP_PROF:    libleak_check.so(+hipMalloc3DArray)
HIP_PROF:  fun_name:hipMalloc3D    fun_type:hip    times:1    total_size:10.00MB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/ho  Open file in editor (ctrl + click)  mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:86)
HIP_PROF:    libleak_check.so(+hipMalloc3D)
HIP_PROF:  fun_name:hipMallocPitch    fun_type:hip    times:1    total_size:4.00MB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:48)
HIP_PROF:    libleak_check.so(+hipMallocPitch)
HIP_PROF:  fun_name:hipMallocMipmappedArray    fun_type:hip    times:1    total_size:4.00MB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:75)
HIP_PROF:    libleak_check.so(+hipMallocMipmappedArray)
HIP_PROF:  fun_name:hipMallocArray    fun_type:hip    times:1    total_size:1024.00KB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:68)
HIP_PROF:    libleak_check.so(+hipMallocArray)
HIP_PROF:  fun_name:hipMalloc    fun_type:hip    times:1    total_size:400.00KB    device_type:device0
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:51)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:  fun_name:hipHostMalloc    fun_type:hip    times:1    total_size:400.00KB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    hipprof_leak_check_test(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    hipprof_leak_check_test(+main /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:246)
HIP_PROF:    hipprof_leak_check_test(+leak_fuction() /public/home/guoyl01/project/galaxy/mtest/tests/profiler/hipprof_leak_check_test.cc:78)
HIP_PROF:    hipprof_leak_check_test(+hipError_t hipHostMalloc<void>(void**, unsigned long, unsigned int) /public/home/guoyl01/local/dtk-26.04.2-
HIP_PROF:    libleak_check.so(+hipHostMalloc)
HIP_PROF:  fun_name:hipMallocManaged    fun_type:hip    times:1    total_size:400.00KB    device_type:device0
HIP_PROF:  stackinfo:
```

hip 接口基于 libc 接口，libc 的泄漏大小包含 hip 的泄漏大小，用户可根据需要选择关注的接口。可参考如下 4.1 和 4.2 表格。

表 4.1 支持的 libc 接口

malloc	calloc	memalign	realloc	new	new[]	mmap

表 4.2 支持的 hip 接口

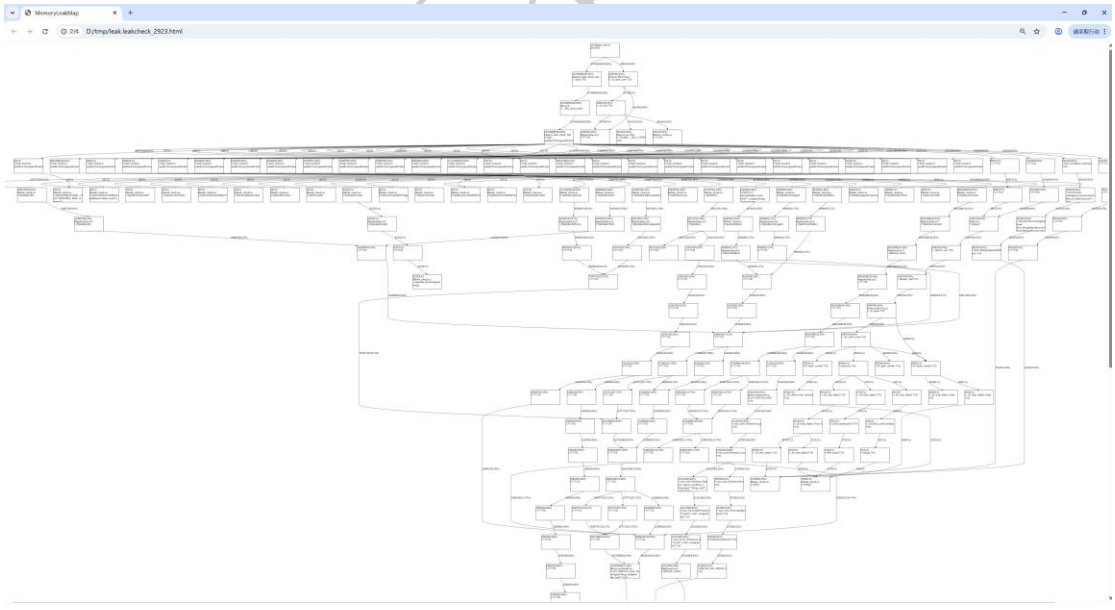
hipHostMalloc	hipExtHostMalloc	hipMalloc
hipMalloc3D	hipMallocArray	hipMalloc3DArray
hipMallocPitch	hipMemAllocPitch	hipMallocManaged
hipMallocMipmappedArray		
hipStreamCreate	hipStreamCreateWithFlags	hipStreamCreateWithPriority
hipEventCreate	hipEventCreateWithFlags	
hipGraphCreate	hipArrayCreate	hipCtxCreate
hipMipmappedArrayCreate	hipTexRefCreate	hipUserObjectCreate
hipCreateTextureObject	hipGreenCtxCreate	hipMemPoolCreate
hipSurfObjectCreate	hipCreateSurfaceObject	hipMemCreate

--leak-check 内存泄漏检测完成后除了生成泄漏函数堆栈，还会生成文件：

leak_2923.db　leak.leakcheck_2923.html　leak.mem_2923.html

leak_2923.db 是 sqlite 数据库，记录申请释放相关的数据；

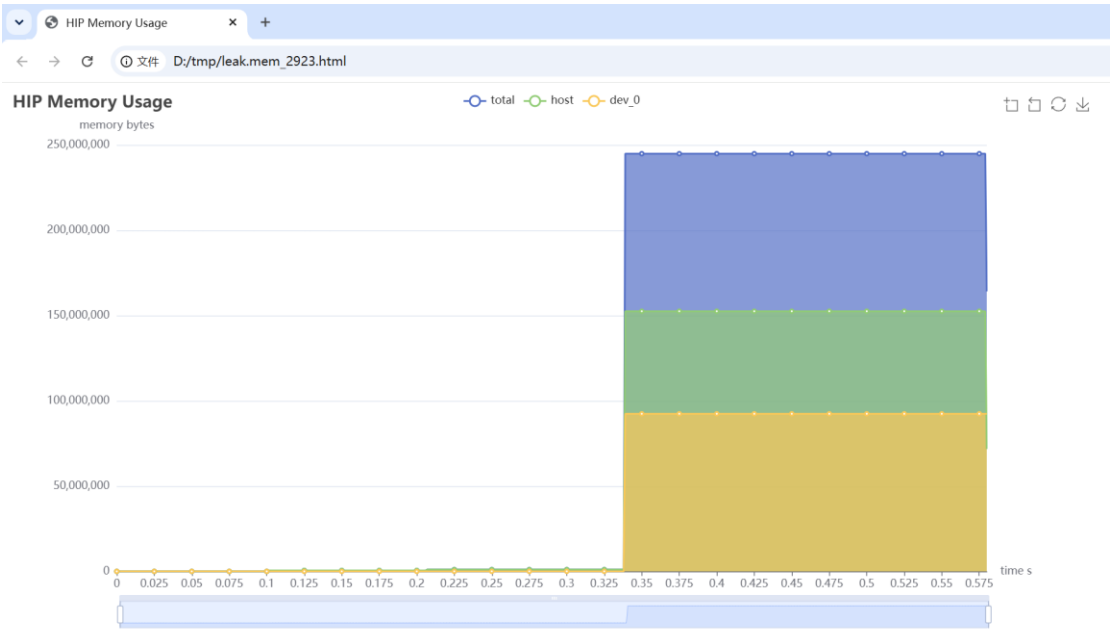
leak.leakcheck_2923.html 是内存泄漏堆栈图，使用浏览器打开如下图：



图中标注了程序内存泄漏函数堆栈、内存泄漏大小、内存泄漏大小占比、内存泄漏位置等信息。

leak.mem_2923.html 是内存占用趋势图，记录着程序运行期间内存大小变化，

使用浏览器打开如下图：



图中横坐标表示执行时间，单位是秒。纵坐标表示内存使用的情况，单位是字节。黄色 dev_0 对应 HCU_0 显卡内存使用情况，绿色 host 对应内存使用情况，紫色 total 对应显存、内存使用总和。横坐标下方是滚动条，除了左右滚动以外可以放大缩小。右上角区域 4 个按钮功能分别是区域缩放，还原上一步，重置和保存为图片。

4.2 基于时间过滤泄漏堆栈

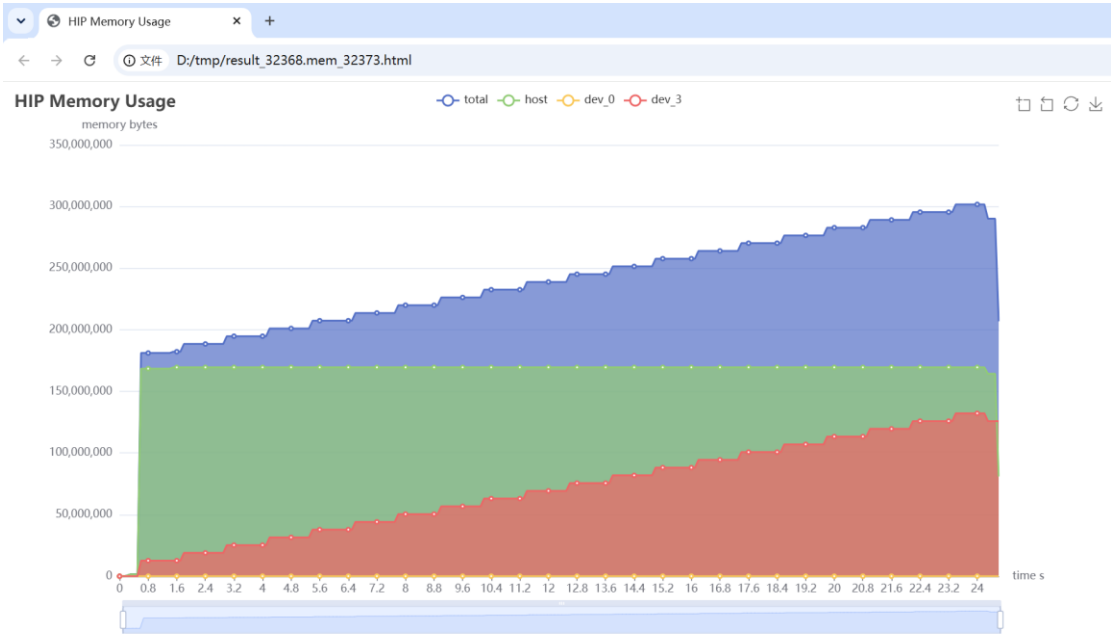
内存泄漏检测时，需要重点关注随着时间增长泄漏的大小持续增长的泄漏函数堆栈。使用 leak-check 功能后，会生成内存变化趋势图和完整泄漏堆栈，通过本节的功能可以过滤出指定时间区间内的内存泄漏函数堆栈。

下图是测试程序例子：

开发者社区

```
[rootb17r2n05 leak]$ hipcc MatrixTranspose.cpp -g -O0 -o MatrixTranspose
[rootb17r2n05 leak]$ hipprof --leak-check ./MatrixTranspose 1 1 20
HIP_PROF:on Fri Jul  3 14:36:46 2026
HIP_PROF:process id '32368'
HIP_PROF:db file 'hip-prof-32368_pid.db'
HIP_PROF:out put file 'result_32368'
HIP_PROF:profiling ' ./MatrixTranspose 1 1 20'
HIP_PROF:process begin
HIP_PROF:db file 'hip-prof-32368_32373.db'
HIP_PROF:process end run total cost:24(s)
HIP_PROF:pid:32373 host leakcheck memory size:77.20MB
HIP_PROF:pid:32373 device0 leakcheck memory size:32.00KB
HIP_PROF:pid:32373 device3 leakcheck memory size:120.07MB
HIP_PROF:pid:32373 leakcheck memory size:197.31MB checksize:1B totaltimes:23610 leaktimes:238
HIP_PROF:leak check start
HIP_PROF:--------------------------------leakcheck pid:32373
HIP_PROF:--------------------------fun_type:libc
HIP_PROF:  fun_name:mmap    fun_type:libc    times:20    total_size:120.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:80)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::hsa_amd_memory_pool_allocate(hsa_amd_memory_pool_s, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::core::Runtime::AllocateMemory(rocr::core::MemoryRegion const*, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::SimpleHeap<rocr::AMD::MemoryRegion::BlockAllocator>::alloc(unsigned long) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+hsaKmtAllocMemory ??:0)
HIP_PROF:    libhsa-runtime64.so(+fmm_allocate_device ??:0)
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:new    fun_type:libc    times:15    total_size:9.02KB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/MatrixTranspose.cpp:106)
HIP_PROF:    libgalaxyhip.so.5(+hipMemcpy)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libleak_check.so(+operator new(unsigned long, std::nothrow_t const&))
HIP_PROF:  fun_name:mmap    fun_type:libc    times:4    total_size:16.00MB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
```

下图是内存泄漏变化趋势图，根据内存趋势图可看出，显存持续增长。



完整的终端输出显示的泄漏函数堆栈信息比较多，仅需要关注某时间段的泄漏函数堆栈时，就可使用本功能过滤出对应堆栈信息。

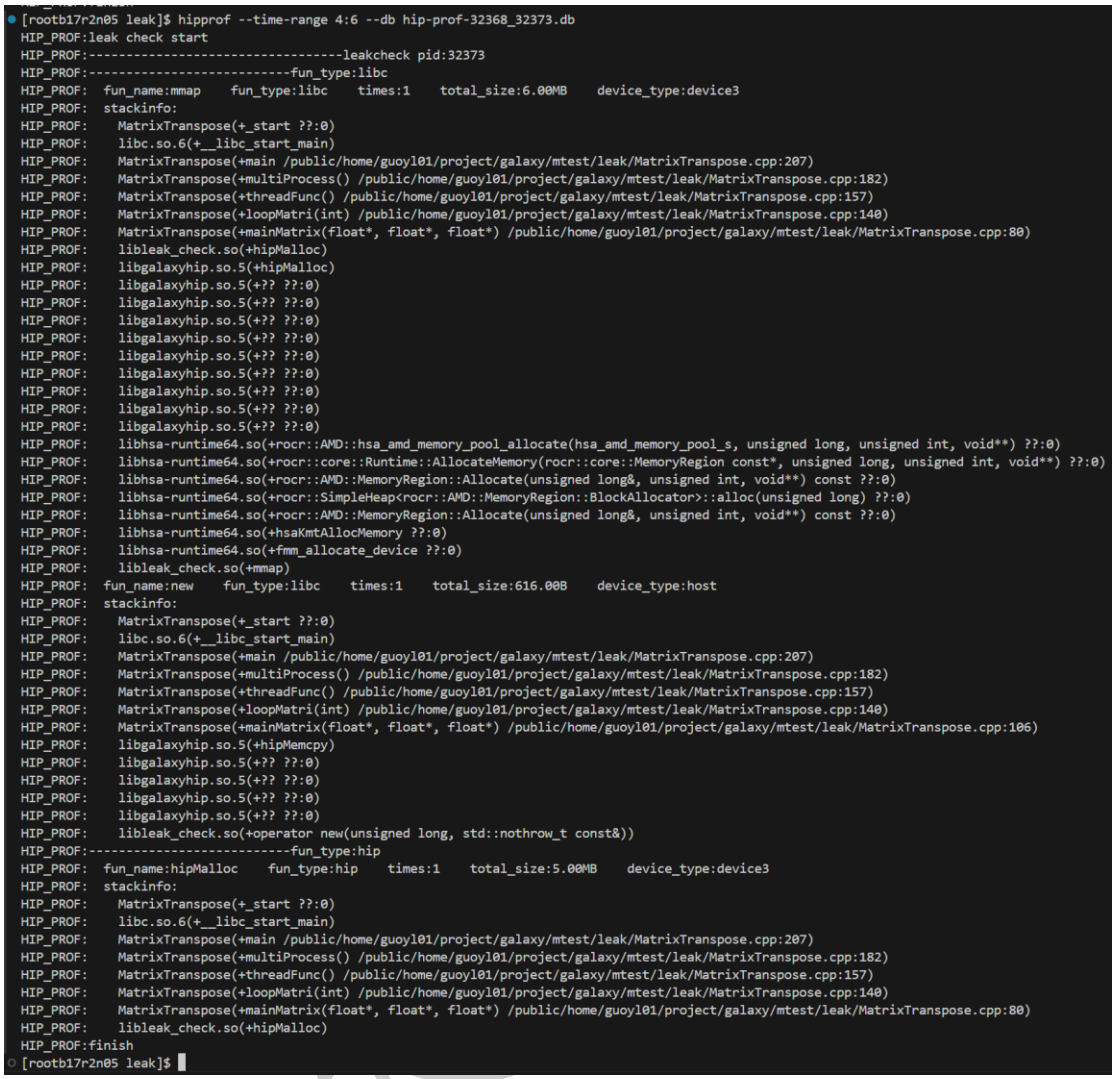
➢使用 hipprof 进行泄漏检测按时间过滤

```
hipprof --time-range 4:6 --db hip-prof-32368_32373.db
```

--time-range 4:6 过滤出数据库中时间区间[4:6]秒内的泄漏函数堆栈；

--db hip-prof-32368_32373.db 指定记录泄漏信息的数据库。

开发者社区



4.3 查找内存波动原因

当未发生内存泄漏时，想找到内存波动对应的内存申请函数堆栈，可添加指令--record-memfreeinfo，数据库中就会记录所有的内存申请函数堆栈信息。

➢使用 hipprof 进行泄漏检测堆栈显示

```
hipprof --leak-check --record-memfreeinfo -o leak ./MatrixTranspose 1 1 3
```

```
[rootb17r2n05 leak]$ hipprof --leak-check --record-memfreeinfo -o leak ./MatrixTranspose 1 1 5
HIP_PROF:on Fri Jul  3 14:28:05 2026
HIP_PROF:process id '26976'
HIP_PROF:db file 'leak_pid.db'
HIP_PROF:out put file 'leak'
HIP_PROF:profiling ' ./MatrixTranspose 1 1 5'
HIP_PROF:process begin
HIP_PROF:db file 'leak_26980.db'
HIP_PROF:process end run total cost:16(s)
HIP_PROF:pid:26980 host leakcheck memory size:77.20MB
HIP_PROF:pid:26980 device0 leakcheck memory size:32.00KB
HIP_PROF:pid:26980 device3 leakcheck memory size:76.00KB
HIP_PROF:pid:26980 leakcheck memory size:77.30MB checksize:1B totaltimes:23386 leaktimes:184
HIP_PROF:leak check start
HIP_PROF:--------------------------------leakcheck pid:26980
HIP_PROF:---------------------------fun_type:libc
HIP_PROF:  fun_name:mmap    fun_type:libc    times:4    total_size:16.00MB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:128)
HIP_PROF:    libgalaxyhip.so.5(+hipSetDevice)
HIP_PROF:    libpthread.so.0(+pthread_once)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::hsa_amd_memory_pool_allocate(hsa_amd_memory_pool_s, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::core::Runtime::AllocateMemory(rocr::core::MemoryRegion const*, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+hsaKmtAllocMemory ??:0)
HIP_PROF:    libhsa-runtime64.so(+fmm_allocate_host ??:0)
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:mmap    fun_type:libc    times:4    total_size:16.00MB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:128)
HIP_PROF:    libgalaxyhip.so.5(+hipSetDevice)
```

测例的内存趋势图如下，内存出现升降波动，使用指令--time-range 可过滤出内存升高对应的函数堆栈，方便查找内存升高原因。通过下图红色部分可看出不存在显存泄漏问题，且期间有多次显存升高和降低的场景。



需要查看[3:5]秒的显存升高的原因可使用的下图中的指令获取调用信息。

```
[rootb17r2n05 leak]$ hipprof --time-range 3:5 --db leak_26980.db
HIP_PROF:leak check start
HIP_PROF:--------------------------------leakcheck pid:26980
HIP_PROF:-------------------------fun_type:libc
HIP_PROF:  fun_name:mmap    fun_type:libc    times:1    total_size:6.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:82)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::hsa_amd_memory_pool_allocate(hsa_amd_memory_pool_s, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::core::Runtime::AllocateMemory(rocr::core::MemoryRegion const*, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::SimpleHeap<rocr::AMD::MemoryRegion::BlockAllocator>::alloc(unsigned long) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+hsaKmtAllocMemory ??:0)
HIP_PROF:    libhsa-runtime64.so(+fmm_allocate_device ??:0)
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:mmap    fun_type:libc    times:1    total_size:6.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:85)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+hipMalloc)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::hsa_amd_memory_pool_allocate(hsa_amd_memory_pool_s, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::core::Runtime::AllocateMemory(rocr::core::MemoryRegion const*, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::SimpleHeap<rocr::AMD::MemoryRegion::BlockAllocator>::alloc(unsigned long) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+hsaKmtAllocMemory ??:0)
HIP_PROF:    libhsa-runtime64.so(+fmm_allocate_device ??:0)
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:mmap    fun_type:libc    times:1    total_size:6.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
```

```
HIP_PROF:    libhsa-runtime64.so(+rocr::core::Runtime::AllocateMemory(rocr::core::MemoryRegion const*, unsigned long, unsigned int, void**) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::SimpleHeap<rocr::AMD::MemoryRegion::BlockAllocator>::alloc(unsigned long) ??:0)
HIP_PROF:    libhsa-runtime64.so(+rocr::AMD::MemoryRegion::Allocate(unsigned long&, unsigned int, void**) const ??:0)
HIP_PROF:    libhsa-runtime64.so(+hsaKmtAllocMemory ??:0)
HIP_PROF:    libhsa-runtime64.so(+fmm_allocate_device ??:0)
HIP_PROF:    libleak_check.so(+mmap)
HIP_PROF:  fun_name:new    fun_type:libc    times:1    total_size:616.00B    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:105)
HIP_PROF:    libgalaxyhip.so.5(+hipMemcpy)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libgalaxyhip.so.5(+?? ??:0)
HIP_PROF:    libleak_check.so(+operator new(unsigned long, std::nothrow_t const&))
HIP_PROF:--------------------------fun_type:hip
HIP_PROF:  fun_name:hipMalloc    fun_type:hip    times:1    total_size:5.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:82)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:  fun_name:hipMalloc    fun_type:hip    times:1    total_size:5.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:85)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:  fun_name:hipMalloc    fun_type:hip    times:1    total_size:5.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:88)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:  fun_name:hipMalloc    fun_type:hip    times:1    total_size:5.00MB    device_type:device3
HIP_PROF:  stackinfo:
HIP_PROF:    MatrixTranspose(+_start ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    MatrixTranspose(+main /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:207)
HIP_PROF:    MatrixTranspose(+multiProcess() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:182)
HIP_PROF:    MatrixTranspose(+threadFunc() /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:157)
HIP_PROF:    MatrixTranspose(+loopMatri(int) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:140)
HIP_PROF:    MatrixTranspose(+mainMatrix(float*, float*, float*) /public/home/guoyl01/project/galaxy/mtest/leak/./MatrixTranspose.cpp:90)
HIP_PROF:    libleak_check.so(+hipMalloc)
HIP_PROF:finish
[rootb17r2n05 leak]$
```

4.4 会话控制泄漏检测

内存泄漏检测时，会降低被检测程序的性能，有些程序的泄漏发生在程序启动较长时间之后。这种场景下，如果一直开启内存泄漏检测将会浪费大量的时间。使用会话功能可以动态的开启、关闭泄漏检测功能。

指令功能介绍：

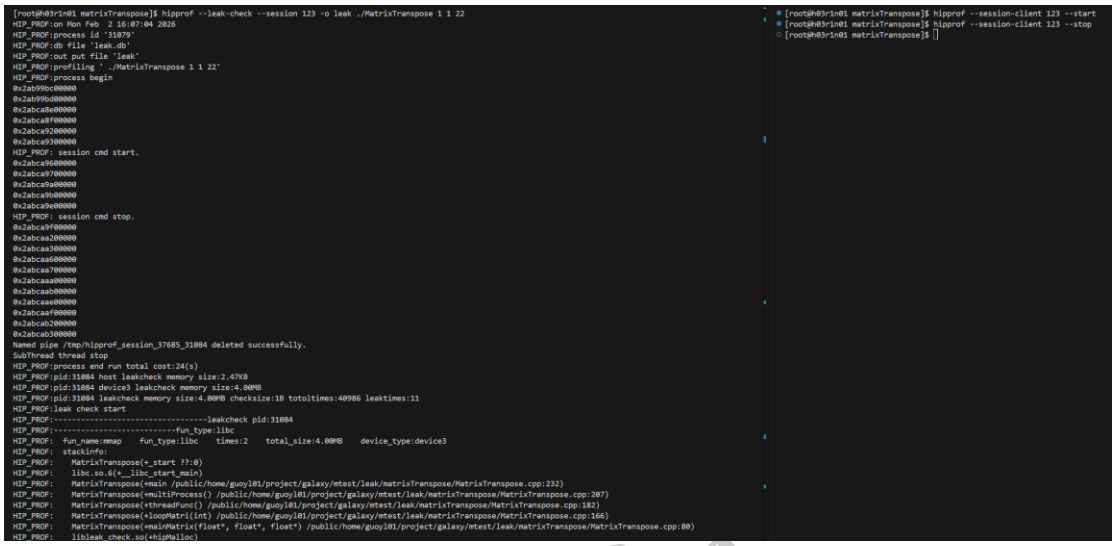
--session 123 开启内存泄漏检测时的会话通信服务器，123 是 key 值，key 值可以是随机值，保持客户端服务器的 key 值相同即可；

--session-client 123 --start 开启会话客户端模式，123 是 key 值，通信内容是启动内存泄漏检测；

--session-client 123 --stop 开启会话客户端模式，123 是 key 值，通信内容是

停止内存泄漏检测。

下图测例中发生了 22 次资源泄漏，通过会话功能仅记录了 5 次资源泄漏堆栈。用户可根据具体时机进行开启、停止泄漏检测。





4.5 泄漏堆栈排序方法

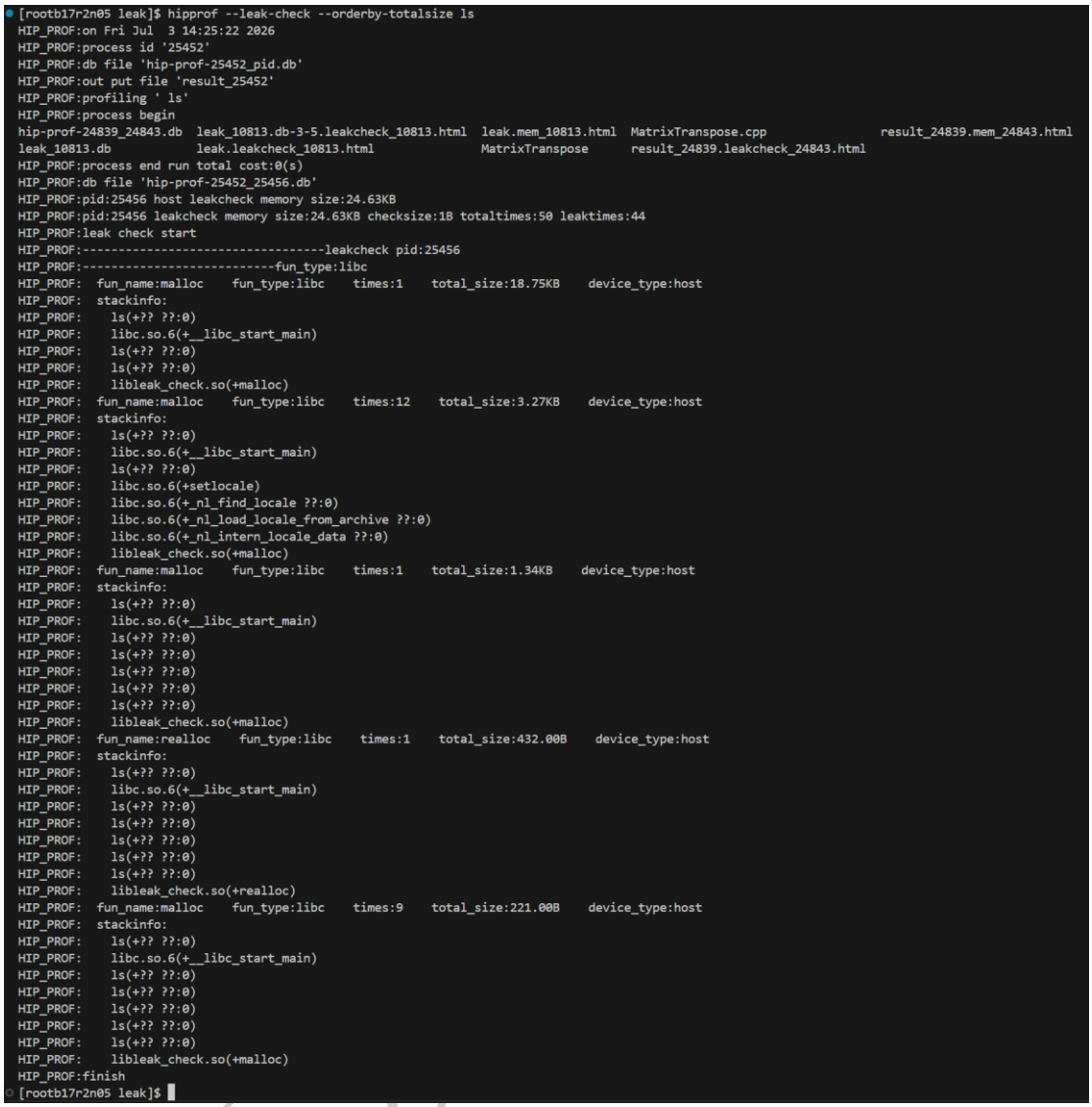
内存泄漏检测时，出现内存泄漏问题时，一般是堆栈调用次数很多，总的泄漏大小不一定很多。26.10 以前的版本默认根据总的泄漏大小来排序，可能会将重点泄漏堆栈排在后面。

从 26.10 版本开始，默认根据泄漏次数来排序，泄漏次数多的堆栈排在前面，如果用户需要根据泄漏大小来排序，可使用--orderby-totalsize 指令将排序方式改为根据泄漏大小排序。

默认函数堆栈的排序方式的示例如下，泄漏次数多的堆栈排在前面：

```
[rootb17r2n05 leak]$ hipprof --leak-check ls
HIP_PROF:on Fri Jul  3 14:24:40 2026
HIP_PROF:process id '24839'
HIP_PROF:db file 'hip-prof-24839_pid.db'
HIP_PROF:out put file 'result_24839'
HIP_PROF:profiling ' ls'
HIP_PROF:process begin
leak_10813.db  leak_10813.db-3-5.leakcheck_10813.html  leak.leakcheck_10813.html  leak.mem_10813.html  MatrixTranspose  MatrixTranspose.cpp
HIP_PROF:process end run total cost:0(s)
HIP_PROF:db file 'hip-prof-24839_24843.db'
HIP_PROF:pid:24843 host leakcheck memory size:23.61KB
HIP_PROF:pid:24843 leakcheck memory size:23.61KB checksize:1B totaltimes:47 leaktimes:41
HIP_PROF:leak check start
HIP_PROF:--------------------------------leakcheck pid:24843
HIP_PROF:--------------------------fun_type:libc
HIP_PROF:  fun_name:malloc    fun_type:libc    times:12    total_size:3.27KB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+setlocale)
HIP_PROF:    libc.so.6(+_nl_find_locale ??:0)
HIP_PROF:    libc.so.6(+_nl_load_locale_from_archive ??:0)
HIP_PROF:    libc.so.6(+_nl_intern_locale_data ??:0)
HIP_PROF:    libleak_check.so(+malloc)
HIP_PROF:  fun_name:malloc    fun_type:libc    times:12    total_size:144.00B    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+setlocale)
HIP_PROF:    libc.so.6(+__strdup)
HIP_PROF:    libleak_check.so(+malloc)
HIP_PROF:  fun_name:malloc    fun_type:libc    times:6    total_size:135.00B    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libleak_check.so(+malloc)
HIP_PROF:  fun_name:malloc    fun_type:libc    times:1    total_size:18.75KB    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libleak_check.so(+malloc)
HIP_PROF:  fun_name:malloc    fun_type:libc    times:1    total_size:624.00B    device_type:host
HIP_PROF:  stackinfo:
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libc.so.6(+__libc_start_main)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    ls(+?? ??:0)
HIP_PROF:    libleak_check.so(+malloc)
HIP_PROF:finish
[rootb17r2n05 leak]$
```

将函数堆栈的排序方式调整为根据泄漏大小来排序的示例如下，泄漏大小多的堆栈排在前面：


[rootb17r2n05 leak]$ hipprof --leak-check --orderby-totalsize ls
HIP_PROF:on Fri Jul 3 14:25:22 2026
HIP_PROF:process id '25452'
HIP_PROF:db file 'hip-prof-25452_pid.db'
HIP_PROF:out put file 'result_25452'
HIP_PROF:profiling ' ls'
HIP_PROF:process begin
hip-prof-24839_24843.db leak_10813.db-3-5.leakcheck_10813.html leak.mem_10813.html MatrixTranspose.cpp result_24839.mem_24843.html
leak_10813.db leak.leakcheck_10813.html MatrixTranspose result_24839.leakcheck_24843.html
HIP_PROF:process end run total cost:0(s)
HIP_PROF:db file 'hip-prof-25452_25456.db'
HIP_PROF:pid:25456 host leakcheck memory size:24.63KB
HIP_PROF:pid:25456 leakcheck memory size:24.63KB checksize:1B totaltimes:50 leaktimes:44
HIP_PROF:leak check start
HIP_PROF:---------------------------------leakcheck pid:25456
HIP_PROF:--------------------------fun_type:libc
HIP_PROF: fun_name:malloc fun_type:libc times:1 total_size:18.75KB device_type:host
HIP_PROF: stackinfo:
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+__libc_start_main)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libleak_check.so(+malloc)
HIP_PROF: fun_name:malloc fun_type:libc times:12 total_size:3.27KB device_type:host
HIP_PROF: stackinfo:
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+__libc_start_main)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+setlocale)
HIP_PROF: libc.so.6(+_nl_find_locale ??:0)
HIP_PROF: libc.so.6(+_nl_load_locale_from_archive ??:0)
HIP_PROF: libc.so.6(+_nl_intern_locale_data ??:0)
HIP_PROF: libleak_check.so(+malloc)
HIP_PROF: fun_name:malloc fun_type:libc times:1 total_size:1.34KB device_type:host
HIP_PROF: stackinfo:
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+__libc_start_main)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libleak_check.so(+malloc)
HIP_PROF: fun_name:realloc fun_type:libc times:1 total_size:432.00B device_type:host
HIP_PROF: stackinfo:
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+__libc_start_main)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libleak_check.so(+realloc)
HIP_PROF: fun_name:malloc fun_type:libc times:9 total_size:221.00B device_type:host
HIP_PROF: stackinfo:
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libc.so.6(+__libc_start_main)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: ls(+?? ??:0)
HIP_PROF: libleak_check.so(+malloc)
HIP_PROF:finish
[rootb17r2n05 leak]$

5 集群场景功能演示

hipprof 支持在万卡集群规模场景上采集性能数据。该章节使用一个主节点，两个子节点进行介绍。

5.1 集群追踪可视化

集群追踪可视化是启动 hipprof 集群功能收集性能数据。集群间通信默认使用“eno1”网卡通信，如果集群不存在“eno1”网卡，则会自动查询/etc/hosts 文件中对应节点的通信网卡 IP。目前集群功能支持--hip-trace、--omp-trace、--barrier、--hiptx-trace、--cl-trace、--pmc <type>，默认开启--hip-trace 功能。其中 pmc 功能不回传数据。集群可以支持只开启主节点收集主节点性能数据（单节点模式）。

集群模式下，启动主节点 hipprof 指令格式如下：

hipprof --master <OPTIONS>

启动子节点 hipprof 指令格式如下：

hipprof --slave --master-address <ip> <OPTIONS>

例如集群的通讯网卡其 ip 为 11.8.3.5。

集群使用步骤如下：

- 启动主节点的 hipprof

```
hipprof --master --hip-trace
```

- 启动子节点的 hipprof

```
hipprof --slave --master-address 11.8.3.5
```

- 启动应用程序，进行性能数据采集(使用 mpirun 在三个节点执行程序，模仿在多节点执行任务)

```
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 ./MatrixTranspose
```

主节点上查看性能数据结果。

执行结果如下：

➢主节点启动成功：

```
hipprof --master --hip-trace
```

- 启动子节点的 hipprof

```
hipprof --slave --master-address 11.8.3.5
```

开发者社区

➢ 启动应用程序，进行性能数据采集(使用 mpirun 在三个节点执行程序，模仿在多节点执行任务)

```
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 ./MatrixTranspose
```

主节点上查看性能数据结果

执行结果如下：

➢主节点启动成功：

```
[        test01@h03r1n04 11:32:49 master-test]# hipprof --master --hip-trace
HIP_PROF:Master node initialized successfully
```

➢子节点启动成功:

```
[        test01@b13r2n07 11:32:40 slave-test]# hipprof --slave --master-address 11.8.3.5
HIP_PROF:Slave node initialized successfully
```

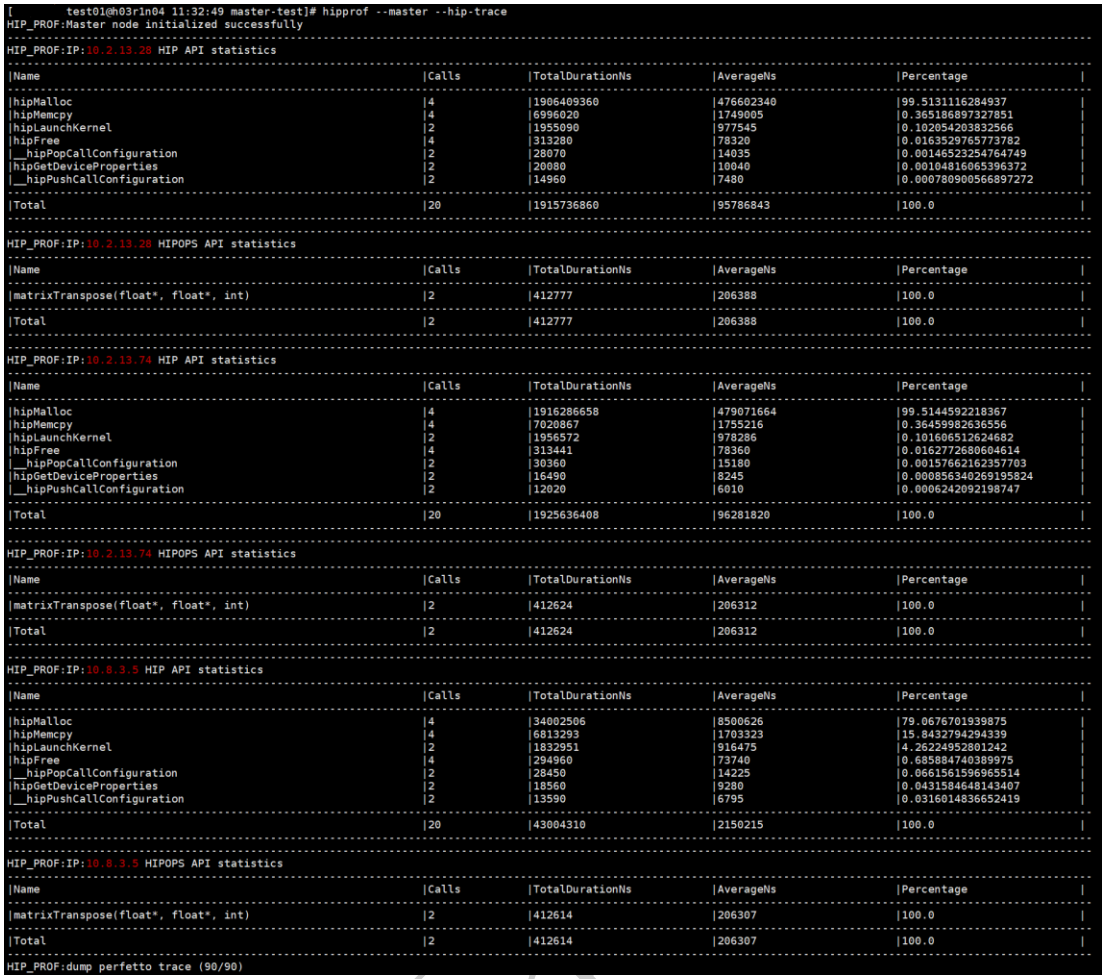
➢用户程序界面执行结果：

```
[        test01@h03r1n04 11:32:05 cluster-test]# mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 ./MatrixTranspose
Device name Device 66a1
Device name Device 66a1
Device name Device 66a1
Device name Device 66a1
Device name Device 66a1
Device name Device 66a1
PASSED!
PASSED!
Named pipe /tmp/hipprof_session_22071_21841 deleted successfully.
SubThread thread stop
Named pipe /tmp/hipprof_session_22071_21842 deleted successfully.
SubThread thread stop
PASSED!
Named pipe /tmp/hipprof_session_22071_12589 deleted successfully.
SubThread thread stop
PASSED!
PASSED!
Named pipe /tmp/hipprof_session_22071_16993 deleted successfully.
SubThread thread stop
Named pipe /tmp/hipprof_session_22071_12590 deleted successfully.
SubThread thread stop
PASSED!
Named pipe /tmp/hipprof_session_22071_16994 deleted successfully.
SubThread thread stop
```

➢子节点运行结束结果:

```
[        test01@b13r4n13 11:19:51 slave-test]#  hipprof --slave --master-address 11.8.3.5
HIP_PROF:Slave node initialized successfully
HIP_PROF:Successfully sent entire DB file using mmap: hip-prof-15320-10.2.13.74.db (24576 bytes)
```

➢主节点接收数据后的展示结果：



从上图可以看到，分别展示了三组数据，每组数据代表一个节点，一共展示在三个节点上的运行结果。每组的展示效果与单节点效果一致。

以上表格除了输出在终端外会另外形成 6 个 csv 格式的文件，命名规则为

result_[主节点 hipprof pid]-[对应节点 ip]：

result_21774-10.2.13.28.hiptrace.csv

result_21774-10.2.13.28.hipkernel.csv

result_21774-10.2.13.74.hiptrace.csv

result_21774-10.2.13.74.hipkernel.csv

result_21774-10.8.3.5.hiptrace.csv

result_21774-10.8.3.5.hipkernel.csv

除了输出统计表格以外，一个 pftrace 格式的文件：

result_21774.pftrace

该 pftrace 文件以主节点 hipprof 的 pid 命名，其中包含多个节点的性能数据。



以上为手动启动 hipprof 集群的执行步骤，用户也可以使用脚本启动。以下为 mpirun 指令使用集群工具示例：

```
# 1) 启动主节点 hipprof
hipprof --master --hip-trace &
HEAD_PID=$!
sleep 1
```

```
# 2) 启动子节点 hipprof
mpirun -x LD_LIBRARY_PATH -x PATH -np 2 -H b13r2n07:1,b13r4n13:1 hipprof --slave --master-address 11.8.3.5 &
WORKER_PID=$!
sleep 2

# 3) 执行用户进程
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 hip/samples/2_Cookbook/0_MatrixTranspose

# 4) 等待完成
wait
```

执行结果如下：

开发者社区

```
test01@h03r1n04 12:05:55 cluster-test]# ./mpirun_cluster.sh
HIP_PROF:Master node initialized successfully
HIP_PROF:Slave node initialized successfully
HIP_PROF:Slave node initialized successfully
Device name Device 66a1
Device name Device 66a1
PASSED!
Device name Device 66a1
Device name Device 66a1
Named pipe /tmp/hipprof_session_22071_19782 deleted successfully.
SubThread thread stop
PASSED!
Named pipe /tmp/hipprof_session_22071_19786 deleted successfully.
SubThread thread stop
Device name Device 66a1
Device name Device 66a1
PASSED!
PASSED!
Named pipe /tmp/hipprof_session_22071_2638 deleted successfully.
SubThread thread stop
Named pipe /tmp/hipprof_session_22071_8640 deleted successfully.
SubThread thread stop
PASSED!
Named pipe /tmp/hipprof_session_22071_8639 deleted successfully.
SubThread thread stop
PASSED!
Named pipe /tmp/hipprof_session_22071_2628 deleted successfully.
SubThread thread stop
HIP_PROF:Successfully sent entire DB file using mmap: hip-prof-8216-11.2.13.74.db (24576 bytes)
HIP_PROF:Successfully sent entire DB file using mmap: hip-prof-2498-11.2.13.28.db (24576 bytes)
--------------------------------------------------------------------------------
HIP_PROF:IP:11.8.3.5 HIP API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|hipMalloc                                           |4              |32780145                 |8195036                |75.8505205271009       |
|hipMemcpy                                           |4              |7188283                  |1797070                |16.6330871094716       |
|hipLaunchKernel                                     |2              |2905492                  |1452746                |6.7230660690283        |
|hipFree                                             |4              |306040                   |76510                  |0.708151025631948      |
|__hipPushCallConfiguration                          |2              |19410                    |9705                   |0.0449131205316825     |
|__hipPopCallConfiguration                           |2              |9170                     |4585                   |0.0212186149034275     |
|hipGetDeviceProperties                              |2              |8230                     |4115                   |0.0190435333320838     |
--------------------------------------------------------------------------------
|Total                                               |20             |43216770                 |2160838                |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.8.3.5 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|matrixTranspose(float*, float*, int)                |2              |412774                   |206387                 |100.0                  |
--------------------------------------------------------------------------------
|Total                                               |2              |412774                   |206387                 |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.28 HIP API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|hipMalloc                                           |4              |2886313110               |721578277              |99.2512999830696       |
|hipLaunchKernel                                     |2              |13802380                 |6901190                |0.474620772470635      |
|hipMemcpy                                           |4              |7614860                  |1903715                |0.261851270248735      |
|hipFree                                             |4              |314330                   |78582                  |0.0108088277101989     |
|__hipPushCallConfiguration                          |2              |23520                    |11760                  |0.000808779396633721   |
|__hipPopCallConfiguration                           |2              |9500                     |4750                   |0.000326675351531477   |
|hipGetDeviceProperties                              |2              |8250                     |4125                   |0.000283691752645757   |
--------------------------------------------------------------------------------
|Total                                               |20             |2908085950               |145404297              |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.28 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|matrixTranspose(float*, float*, int)                |2              |412938                   |206469                 |100.0                  |
--------------------------------------------------------------------------------
|Total                                               |2              |412938                   |206469                 |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.74 HIP API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|hipMalloc                                           |4              |2871924988               |717981247              |99.188096115176        |
|hipMemcpy                                           |4              |20624889                 |5156222                |0.712324827787889      |
|hipLaunchKernel                                     |2              |2520964                  |1260482                |0.0870669048041649     |
|hipFree                                             |4              |320051                   |80012                  |0.0110536485048885     |
|__hipPushCallConfiguration                          |2              |24500                    |12250                  |0.000846160106888492   |
|__hipPopCallConfiguration                           |2              |9710                     |4855                   |0.000335355695905602   |
|hipGetDeviceProperties                              |2              |8020                     |4010                   |0.000276987920703906   |
--------------------------------------------------------------------------------
|Total                                               |20             |2895433122               |144771656              |100.0                  |
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
HIP_PROF:IP:11.2.13.74 HIPOPS API statistics
--------------------------------------------------------------------------------
|Name                                                |Calls          |TotalDurationNs          |AverageNs              |Percentage             |
--------------------------------------------------------------------------------
|matrixTranspose(float*, float*, int)                |2              |412296                   |206148                 |100.0                  |
--------------------------------------------------------------------------------
|Total                                               |2              |412296                   |206148                 |100.0                  |
--------------------------------------------------------------------------------
HIP_PROF:dump perfetto trace (90/90)
```

5.2 集群追踪控制

集群追踪控制主要控制集群的行为，例如当集群节点太多，只想指定某一些节点收集性能数据，或者用户程序运行时间过长，只想获取其中一段时间的运行数据时，可以通过添加指令控制集群。

1、通过--node-list 指令指定在某些节点上收集性能数据，不在指定范围的节

点不允许开启 hipprof。

```
# 1) 启动主节点 hipprof，指定 b13r2n07 可以启动 hipprof
hipprof --master --hip-trace --node-list b13r2n07 &
HEAD_PID=$!
sleep 1

# 2) 启动子节点 hipprof
mpirun -x LD_LIBRARY_PATH -x PATH -np 2 -H b13r2n07:1,b13r4n13:1 hipprof --slave --master-address 11.8.3.5 &
WORKER_PID=$!
sleep 2

# 3) 执行用户进程
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 hip/samples/2_Cookbook/0_MatrixTranspose

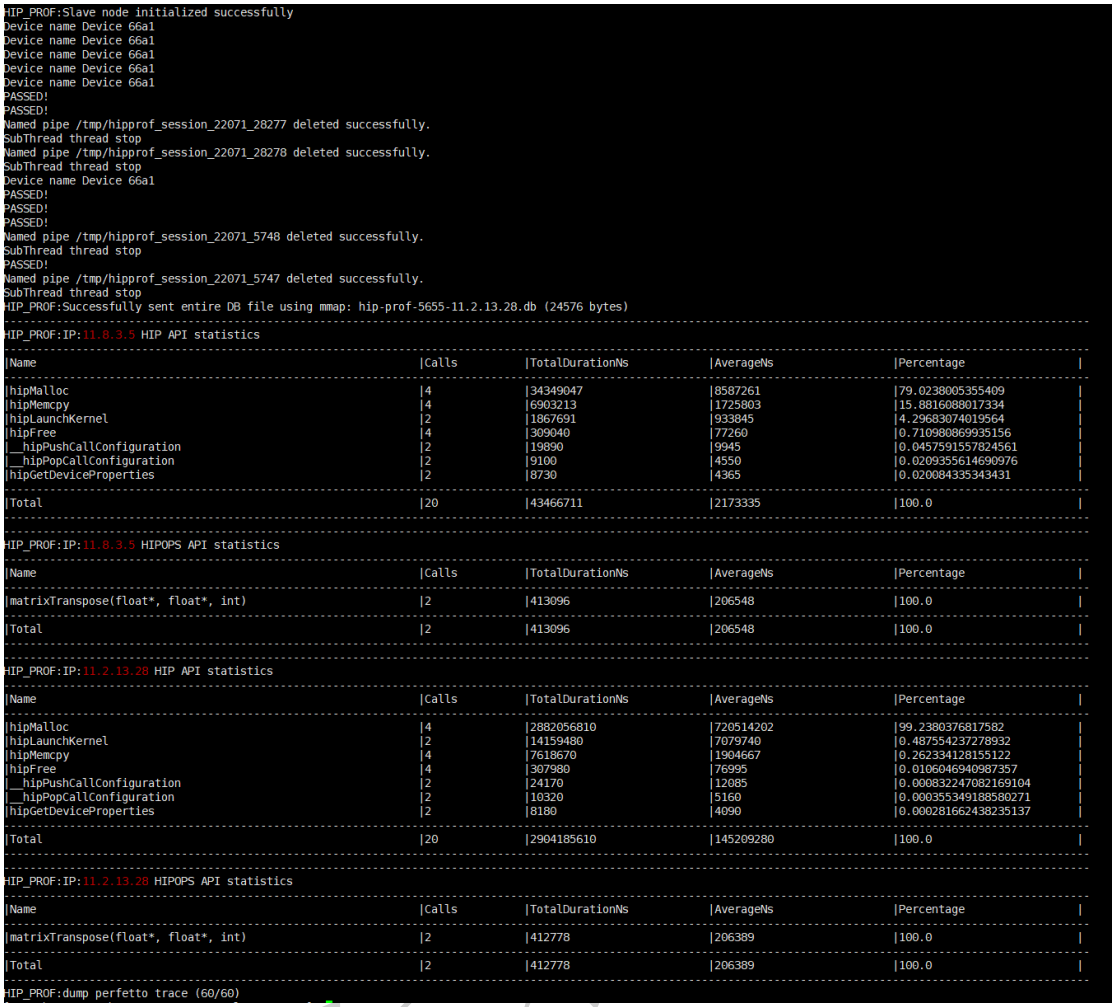
# 4) 等待完成
wait
```

执行结果如下：



可以看到在两个子节点上启动 hipprof 时，出现提示节点退出。主节点只指定 b13r2n07，即只允许 b13r2n07 节点启动 hipprof，b13r4n13 启动时会提示不允许启动并自动退出，此时只会收集主节点和 b13r2n07 节点的数据。

开发者社区



以上为只收集主节点和 b13r2n07 的数据展示，可以看到只有两个节点的统计数据。

2、通过 session 控制开启集群的性能数据收集、关闭集群的性能数据收集，以及退出集群的性能数据收集。test_hipLaunchKernel 测例为循环执行 25000 个 kernel，正常结束时 ops 统计数据应该为 25000 个。

test_hipLaunchKernel 示例代码如下：

```
#include <iostream>
#include <hip/hip_runtime.h>
#include <chrono>
#include <vector>
#include <climits>

#define M 1024
#define N 1024
#define ITERATIONS 25000

__global__ void set_C_to_1024(float *C_value, int n) {
```

```
int a = 1;
}

int main() {
hipError_t ret = hipInit(0);
if (ret != hipSuccess) {
std::cerr << "HIP initialization failed" << std::endl;
return 1;
}

float *d_C_value;
hipMalloc((void **)&d_C_value, sizeof(float) * M * N);

float *h_C_value = (float *)malloc(sizeof(float) * M * N);

dim3 blocksize(128, 1);
dim3 gridsize(M * N / 128, 1);
int number = M * N;

hipStream_t stream;
hipStreamCreate(&stream);

using clock = std::chrono::high_resolution_clock;
using ns = std::chrono::nanoseconds;
uint64_t total_time = 0;
uint64_t min_time = ULLONG_MAX;
uint64_t max_time = 0;
std::vector<uint64_t> timings;
timings.reserve(ITERATIONS - 1);// Reserve for ITERATIONS-1 (excluding first)

bool overall_success = true;
bool first_iteration = true;

for (int i = 0; i < ITERATIONS; i++) {
auto start = clock::now();

void* args[] = {&d_C_value, &number};
ret = hipLaunchKernel((const void *)set_C_to_1024,
                                    gridsize, blocksize, args, 0, stream);
auto end = clock::now();
auto duration = std::chrono::duration_cast<ns>(end - start).count();

if (!first_iteration) {
                    // Record timing (excluding first iteration)
```

```
                timings.push_back(duration);
                total_time += duration;
                if (duration < min_time) min_time = duration;
                if (duration > max_time) max_time = duration;
}
first_iteration = false;

if (ret != hipSuccess) {
                std::cerr << "Iteration " << i << ": Kernel launch failed" << std::endl;
                overall_success = false;
                continue;
}

// Synchronize and verify (optional for timing purity)
hipStreamSynchronize(stream);
hipMemcpy(h_C_value, d_C_value, sizeof(float) * M * N, hipMemcpyDeviceToHost);
}

// Calculate and print timing statistics (excluding first iteration)
double avg_time = static_cast<double>(total_time) / (ITERATIONS - 1);
std::cout << "\nKernel Launch Timing Statistics (ns) - Excluding first launch:\n";
std::cout << "----------------------------------------\n";
std::cout << "Average time: " << avg_time << " ns\n";
std::cout << "Minimum time: " << min_time << " ns\n";
std::cout << "Maximum time: " << max_time << " ns\n";
std::cout << "Total time:" << total_time << " ns (" << (ITERATIONS - 1) << " iterations)\n";
std::cout << "----------------------------------------\n";

// Cleanup
hipFree(d_C_value);
free(h_C_value);
hipStreamDestroy(stream);

return overall_success ? 0 : 1;
}
```

➢编译 test_hipLaunchKernel

```
hipcc -o test_hipLaunchKernel test_hipLaunchKernel.cpp
```

➢主节点启动 hipprof

```
hipprof --master --hip-trace --session 1234
```

➢启动子节点 hipprof

```
 hipprof --slave --master-address 11.8.3.5
```

➢执行用户进程

mpirun -x LD_LIBRARY_PATH -np 3 -H h03r1n04:1,b13r2n07:1,b13r4n13:1 ./test_hipLaunchKernel

➢主节点 session 通知退出集群

hipprof --session-client 1234 --exit

等待完成

主节点运行结果如下：

```
[       test01@h03r1n04 12:51:21 master-test]# hipprof --master --hip-trace --session 1234
HIP_PROF:Master node initialized successfully

HIP_PROF:session cmd exit.

HIP_PROF:IP:10.2.13.28 HIP API statistics
|Name                                                
```

```
[        test01@h03r1n04 12:36:27 cluster-test]# mpirun -x LD_LIBRARY_PATH -np 3 -H h03r1n04:1,b13r2n07:1,b13r4n13:1 ./test_hipLaunchKer
nel

Named pipe /tmp/hipprof_session_22071_15680 deleted successfully.
SubThread thread stop
Named pipe /tmp/hipprof_session_22071_10759 deleted successfully.
SubThread thread stop
Named pipe /tmp/hipprof_session_22071_2460 deleted successfully.
SubThread thread stop

Kernel Launch Timing Statistics (ns) - Excluding first launch:
----------------------------------------
Average time: 3999.53 ns
Minimum time: 2800 ns
Maximum time: 29251 ns
Total time:   99984135 ns (24999 iterations)
----------------------------------------

Kernel Launch Timing Statistics (ns) - Excluding first launch:
----------------------------------------
Average time: 4370.58 ns
Minimum time: 3310 ns
Maximum time: 18930 ns
Total time:   109260050 ns (24999 iterations)
----------------------------------------

Kernel Launch Timing Statistics (ns) - Excluding first launch:
----------------------------------------
Average time: 4540.98 ns
Minimum time: 3310 ns
Maximum time: 122581 ns
Total time:   113520078 ns (24999 iterations)
----------------------------------------
```

以上是使用 session 控制集群退出的执行结果。当主节点 hipprof 接收到了 exit 指令后等待子节点传输结束后将统计数据进行导出；子节点 hipprof 接收到 exit 指令后，强制退出并将性能数据传输到主节点，用户程序接收到退出指令后停止抓取性能数据并退出 session 线程，继续执行后续代码。主节点 hipprof 导出的统计数据每个节点都未执行完 25000 个 kernel，这是因为使用 session --exit 指令会强制 hipprof 集群退出，不会等待用户程序最终运行结束。同理，集群也支持 session 功能的--stop、--start 等功能。

5.3 集群传输控制

hipprof 集群默认使用 db 传输模式，子节点抓取的数据先存入本节点的 db 文件，子节点程序运行结束后会将 db 文件通过网络发送到主节点。

--data-mode 可以指定子节点数据传输的方式，0：db 传输; 1：实时传输；2：不传输，默认 db 传输模式。db 传输的结果参考 5.1 节。

实时传输步骤如下：

➢主节点启动 hipprof 时指定传输模式

```
hipprof --master --hip-trace --data-mode 1
```

➢子节点正常启动

```
 hipprof --slave --master-address 11.8.3.5
```

➢执行用户程序

```
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 ./MatrixTranspose
```

等待用户程序结束，并在主节点上查看性能数据结果。

主节点 hipprof 执行结果如下：

```
[       test01@h03r1n04 13:59:57 master-test]# hipprof --master --hip-trace --data-mode 1
HIP_PROF:Master node initialized successfully

HIP_PROF:IP:10.10.17.68 HIP API statistics

|Name                                                
```

result_572710.8.3.5.hiptrace.csv

result_5727-10.10.17.68.hipkernel.csv

result_5727-10.8.1.80.hipkernel.csv

result_5727-10.8.3.5.hipkernel.csv

result_5727.pftrace

5.4 集群网络控制

集群模式默认使用“eno1”网卡，1234 端口进行数据通信，当集群没有 eno1 网卡或 eno1 网卡没有 ip 或当出现端口占用时，集群会退出。可以通过以下方式启动集群：

1）可以通过--netcard 指定一个有 ip 的网卡进行通信，以下为通过指定网卡启动集群：

➢启动主节点 hipprof 并指定网卡 ib0

```
hipprof --master --netcard ib0
```

➢启动子节点 hipprof 并指定网卡 ib0，11.8.3.5 为主节点 ib0 网卡的 ip

```
hipprof --slave --master-address 11.8.3.5 --netcard ib0
```

运行用户程序；

等待用户程序结束，并在主节点上查看性能数据结果。

2）--port 功能是指定通信端口，当出现默认端口占用时，可以使用该指令切换端口，主要场景为多个 docker 使用同一个宿主机 ip，每个 docker 中都开启 hipprof 集群的情况，以下是通过指定端口启动集群的示例：

➢启动主节点 hipprof 并指定端口

```
hipprof --master --port 3456
```

➢启动子节点 hipprof 并指定端口

```
hipprof --slave --master-address 11.8.3.5:3456
```

运行用户程序；

等待用户程序结束，并在主节点上查看性能数据结果。

5.5 展示信息过滤控制

集群功能增加一系列的展示信息过滤功能，可以通过以下方式过滤：

1）可以通过--show-pid 指定根据 pid 分组打印和导出统计数据。

➢启动主节点 hipprof 并指定--show-pid

```
hipprof --master --show-pid
```

➢启动子节点 hipprof 并指定端口

```
hipprof --slave --master-address 11.8.3.5
```

➢运行用户程序

```
mpirun -x LD_LIBRARY_PATH -np 6 -H h03r1n04:2,b13r2n07:2,b13r4n13:2 ./MatrixTranspose
```

等待用户程序结束，并在主节点上查看性能数据结果

```
[       test01@h03r1n04 14:10:38 master-test]# hipprof --master --hip-trace --show-pid
HIP_PROF:Master node initialized successfully

HIP_PROF:IP:10.10.17.68 PID:14954 HIP statistics

|Name                                                
```

csv 文件中的统计结果也会按 pid 进行分组。下表只显示其中两个 pid 结果。

# PID: 14954				
Name	Calls	TotalDurationNs	AverageNs	Percentage
hipMalloc	2	986158660	493079330	49.62197
hipMemcpy	2	3334131	1667065	0.167768
hipLaunchKernel	1	1048781	1048781	0.052773
hipFree	2	154711	77355	0.007784
__hipPopCallConfiguration	1	15460	15460	0.000777
hipGetDeviceProperties	1	8500	8500	0.000427
__hipPushCallConfiguration	1	7260	7260	0.000365
Total	10	990727503	99072750	100
# PID: 14955				
Name	Calls	TotalDurationNs	AverageNs	Percentage
hipMalloc	2	991986443	495993221	49.915223
hipMemcpy	2	3463441	1731720	0.174274
hipLaunchKernel	1	979511	979511	0.049287
hipFree	2	154500	77250	0.007774
__hipPopCallConfiguration	1	14910	14910	0.000750
hipGetDeviceProperties	1	9410	9410	0.000473
__hipPushCallConfiguration	1	6760	6760	0.000340
Total	10	996614975	99661497	100

2）可按照 trace ID 过滤 DB 中 Trace 数据，可参考 3.2 节使用方式。

开发者社区

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区